

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор литературы	8
1.1 Нейронная сеть	8
1.2 Google MediaPipe.....	9
1.3 Telegram Bot API	12
1.4 Обзор аналогов	13
2 Системное проектирование.....	17
2.1 Перечень структурных блоков проекта	17
2.2 Блок взаимодействия с пользователем	17
2.3 Блок интеграции с телеграм-ботом	18
2.4 Блок модели нейронной сети	18
2.5 Блок обучения нейронной сети.....	19
2.6 Блок тестирования.....	20
2.7 Блок подсчета потерь.....	20
2.8 Блок набора данных	21
2.9 Блок подготовки набора данных	22
3 Функциональное проектирование	23
3.1 Архитектура программного модуля.....	23
3.2 Блок модели нейронной сети	23
3.3 Блок обучения нейронной сети.....	25
3.4 Блок взаимодействия с пользователем	26
3.5 Блок интеграции с Telegram-ботом	32
3.6 Блок подготовки набора данных	35
3.7 Блок набора данных	36
4 Разработка программных модулей	37
4.1 Разработка блока модели нейронной сети.....	37
4.2 Реализация блока обучения нейронной сети.....	40
4.3 Реализация блока взаимодействия с пользователем	44
4.4 Реализация блока интеграции с Telegram-ботом.....	46
4.5 Реализация блока подготовки набора данных	48
4.6 Блок набора данных	48
5 Программа и методика испытаний.....	52
5.1 Методика тестирования.....	52
5.2 Тестирование графического интерфейса	52
5.3 Тестирование функциональности приложения.....	56
6 Руководство пользователя.....	64
6.1 Инструкция по запуску проекта	64
6.2 Руководство пользователя.....	69
7 Техничко-экономическое обоснование разработки и реализации на рынке телеграм-бота с поддержкой языка глухонемых	75
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	75
7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке.....	75

7.3 Расчет экономического эффекта от реализации программного	78
7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	79
7.5 Анализ полученных результатов	79
Заключение	81
Список использованных источников	82
Приложение А	84
Приложение Б	122
Приложение В.....	123

ВВЕДЕНИЕ

В настоящее время мессенджер Telegram активно используется широким кругом пользователей для обмена информацией, голосовые сообщения становятся все более популярным средством коммуникации. Многие пользователи предпочитают отправлять голосовое сообщение за его оперативность и эмоциональную окраску. Однако для людей с нарушениями слуха и речи традиционные аудиосообщения недоступны, что создает серьезный информационный барьер. Современные достижения в области обработки изображений и видеопотока, а также глубокого обучения, открывают новые возможности для разработки систем, способных адаптировать коммуникационные сервисы под нужды данной категории пользователей.

За последнее десятилетие нейронные сети зарекомендовали себя как эффективный инструмент для обработки и анализа больших объемов данных, в том числе и изображений. Их способность автоматически извлекать сложные признаки из визуальной информации существенно повышает точность распознавания объектов, что критически важно для задач компьютерного зрения. Современные архитектуры позволяют нейросетевым моделям адаптироваться к разнообразию входных данных, обеспечивая надежное распознавание даже в условиях плохой освещенности.

Целью данного дипломного проекта является разработка Telegram-бота с поддержкой языка глухонемых, который обеспечит доступность коммуникации для людей с нарушениями слуха и речи путем анализа видеопотока и трансляции его в текст. Проект предназначен для людей с нарушением речи и слуха и разрабатывается для уравнивания возможностей между различными социальными группами.

В соответствии с поставленной целью можно выделить следующий набор задач данного дипломного проекта:

- выбор нейросетевой модели для реализации алгоритма распознавания жестов в видеопотоке;
- подготовка набора данных для обучения модели;
- обучение нейросетевой модели на основе созданного набора данных;
- разработка телеграм-бота для взаимодействия с моделью;
- интеграция модели с ботом в мессенджере Telegram;
- проверка качества работоспособности и распознавания жестов в видеопотоке и создание качественного пользовательского опыта.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 83% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Нейронная сеть

Нейронные сети представляют собой вычислительные модели, созданные по аналогии с биологическими нейронными системами, лежащими в основе работы мозга живых существ. Они играют ключевую роль в современных подходах к машинному и глубокому обучению [1], обеспечивая возможность решения разнообразных сложных задач, включая классификацию, регрессию, генерацию данных, распознавание образов и обработку естественного языка. Благодаря высокой гибкости и способности адаптироваться к различным типам информации, нейронные сети широко используются в компьютерном зрении, анализе временных рядов, медицине и системах автономного управления.

Нейронные сети состоят из множества нейронов – взаимосвязанных узлов, организованных в структурированные слои. Каждый слой выполняет свою роль в обработке входной информации, преобразуя ее шаг за шагом до получения окончательного результата. Важным аспектом является возможность обучения сети: она способна корректировать свои внутренние параметры на основе ошибок, выявленных в ходе обучения, что позволяет ей адаптироваться к новым данным и обнаруживать скрытые закономерности.

Современные нейронные сети строятся следующим образом:

1. Нейрон. Является основой нейронной сети и представляет собой базовую единицу сети, аналогичную клетке нервной системы. Каждый нейрон принимает входные сигналы, осуществляет над ними серию линейных преобразований с использованием весовых коэффициентов, а затем применяет нелинейную функцию активации. Эта функция позволяет ввести элемент нелинейности, благодаря чему сеть способна моделировать сложные зависимости. В процессе обучения веса нейронов корректируются, что повышает точность модели в решении поставленной задачи.

2. Слои нейронной сети. Нейроны в сети связываются в слои – структурное разделение, позволяющее организовать процесс обработки информации в несколько этапов:

- входной слой: принимает исходные данные, такие как пиксели изображения, звуковые сигналы или текстовые последовательности;
- скрытые слои: осуществляют обработку информации, глубина сети, то есть количество скрытых слоев и нейронов в каждом из них, определяет вычислительную сложность модели и ее способность выявлять высокоуровневые абстракции;
- выходной слой: формирует конечный результат работы сети в зависимости от решаемой задачи.

3. Функции активации. Функция активации – это математическое преобразование, вводящее нелинейность в модель. Эти функции позволяют сети гибко адаптироваться к разнообразным входным данным и моделировать сложные зависимости.

4. Алгоритмы обучения. Алгоритм обучения – это процесс настройки весов нейронной сети для минимизации ошибки предсказания. Основным методом является алгоритм обратного распространения ошибки, в ходе которого вычисляется градиент функции потерь, а затем происходит корректировка весовых коэффициентов с использованием оптимизационных методов. Эти методы позволяют модели постепенно улучшать свою способность к обучению на новых данных.

Существует множество архитектур нейронных сетей, каждая из которых оптимизирована для решения определенного класса задач, ниже приведены наиболее распространенные из них:

1. Полносвязные нейронные сети (от англ. Fully Connected Neural Networks, FNN). В таких моделях каждый нейрон одного слоя соединен с каждым нейроном следующего слоя. Такие сети подходят для задач с небольшим числом признаков, однако при работе с изображениями или последовательностями они могут сталкиваться с проблемами масштабируемости и вычислительной затратности.

2. Сверточные нейронные сети (от англ. Convolutional Neural Networks, CNN). Эти сети особенно эффективны при обработке изображений и видео. Сверточные слои автоматически извлекают признаки из данных, сохраняя пространственную иерархию, что позволяет эффективно решать задачи классификации, сегментации и детектирования объектов.

3. Рекуррентные нейронные сети (от англ. Recurrent Neural Networks, RNN). Используются для работы с последовательными данными – временными рядами, текстом или аудиосигналами. Благодаря внутренней памяти, учитывающей предыдущие состояния, RNN способны моделировать контекст и зависимости, что делает их незаменимыми для задач перевода, синтеза речи и анализа последовательностей.

4. Генеративно-состязательные сети (от англ. Generative Adversarial Networks, GAN). Эти сети представляют собой две конкурирующие модели: генератор, создающий искусственные данные, и дискриминатор, оценивающий их достоверность. Такой состязательный процесс позволяет генерировать высококачественные изображения, видео и другие типы данных, приближенные к реальным.

Благодаря разнообразию архитектур и методов обучения, нейронные сети продолжают совершенствоваться, находя новые области применения. Их развитие способствует созданию все более точных, эффективных и адаптивных моделей, способных решать широкий спектр задач в различных сферах науки и технологий – именно поэтому в данном проекте было решено их задействовать.

1.2 Google MediaPipe

Google MediaPipe [2] – это открытый кросс-платформенный фреймворк, разработанный Google для создания интеллектуальных приложений, способных обрабатывать мультимедийные данные в реальном времени. Его

уникальность заключается в способности объединять сложные алгоритмы машинного обучения с эффективной потоковой обработкой, что делает его незаменимым инструментом для разработчиков, работающих над проектами в области компьютерного зрения, анализа движений и интерактивных систем. Архитектура MediaPipe построена вокруг модульности, позволяя гибко подстраиваться под задачи любой сложности – от распознавания эмоций до управления роботами с помощью жестов.

На рисунке 1.1 [3] приведены примеры типичных задач, решаемых с помощью модуля MediaPipe.

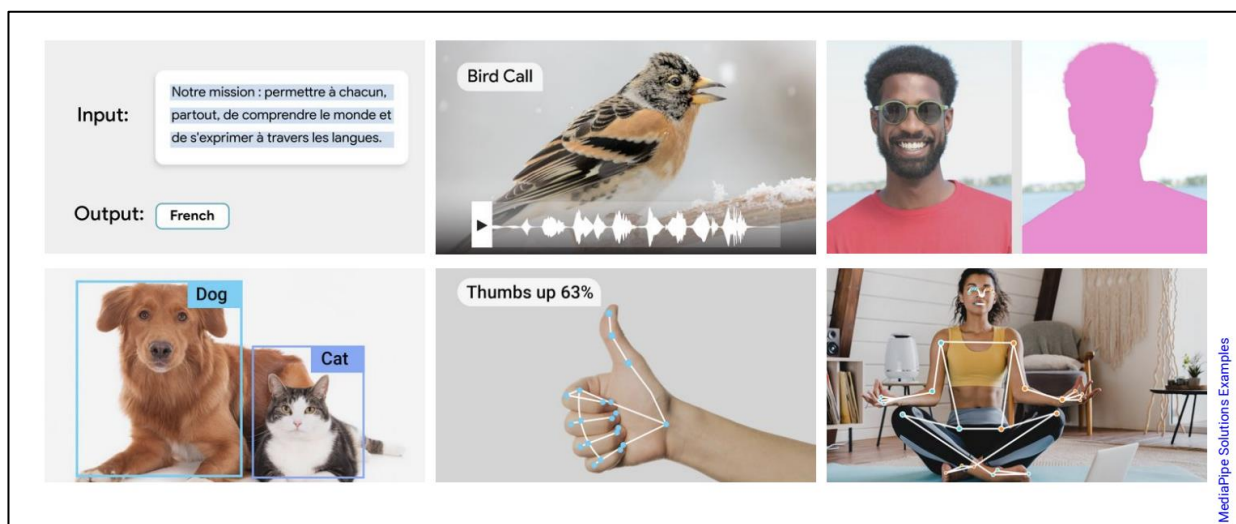


Рисунок 1.1 – Пример задач, решаемых с помощью MediaPipe

Архитектура и принципы работы. В основе MediaPipe лежит концепция графов обработки данных – цепочек взаимосвязанных узлов, где каждый узел выполняет конкретную операцию: от декодирования видео до запуска нейросетевых моделей. Например, при анализе видеопотока кадры последовательно проходят через фильтры шумоподавления, детекцию объектов, трекинг движений и визуализацию результатов. Эти этапы объединяются в единый конвейер, который работает как слаженный механизм, минимизируя задержки даже на маломощных устройствах.

Для достижения высокой производительности MediaPipe использует аппаратное ускорение через интеграцию с графическими API, такими как OpenGL или Vulkan, и мобильными платформами, например, Android NNAPI и iOS Core ML. Это позволяет обрабатывать данные со скоростью до 60 кадров в секунду на смартфонах, что критически важно для приложений дополненной реальности или систем видеонаблюдения.

Детекция рук. Одной из ключевых возможностей MediaPipe является отслеживание рук – технология, которая не только обнаруживает кисти в кадре, но и отслеживает положение 21 ключевой точки на каждой руке, включая суставы и кончики пальцев. Это открывает двери для создания интуитивных интерфейсов, где жест становится заменой мыши или сенсорного экрана. Процесс трекинга делится на два этапа:

1. Детекция ладони. MediaPipe использует легковесную нейросеть, оптимизированную для работы в реальном времени. Модель анализирует кадр и определяет предполагаемые границы вокруг всей руки, даже если она частично скрыта или повернута под углом.

2. Определение ключевых точек. На втором этапе другая модель строит 3D-карту руки, предсказывая координаты суставов. Алгоритм учитывает перспективу и наложение пальцев, что позволяет точно определять самые различные жесты.

Пример детекции рук приведен на рисунке 1.2 [4].

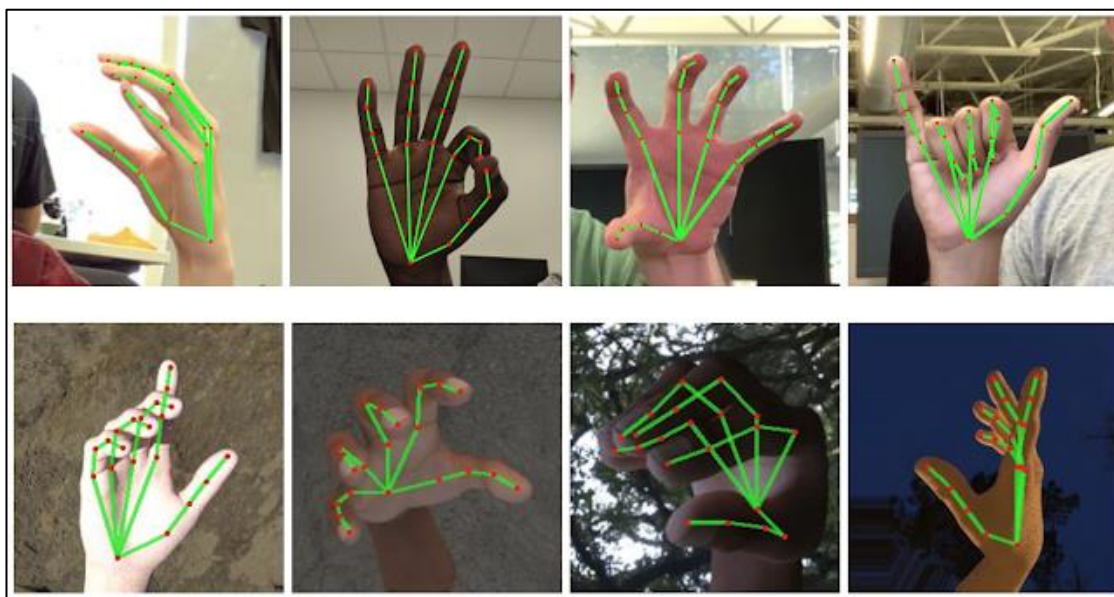


Рисунок 1.2 – Пример детекции рук с помощью MediaPipe

MediaPipe применяет машинное обучение на основе синтетических данных – вместо ручной разметки тысяч изображений, разработчики генерируют реалистичные 3D-модели рук в различных позах и условиях освещения. Это ускоряет обучение моделей и повышает их устойчивость к шумам.

Преимущества подхода MediaPipe в отслеживании рук:

- низкая задержка – алгоритм обрабатывает кадр за 5-10 миллисекунд, что незаметно для пользователя;
- устойчивость к перекрытиям – система сохраняет трекинг даже при скрещенных пальцах или быстрых движениях;
- кроссплатформенность – решение одинаково эффективно работает на Android-смартфонах, iOS-устройствах и ПК.

На практике это позволяет создавать приложения, где пользователь может «нажимать» виртуальные кнопки в воздухе, управлять 3D-объектами в пространстве дополненной реальности или обучаться жестовому языку через интерактивные тренажеры. Например, в образовательных проектах MediaPipe используется для анализа правильности хирургических швов в симуляторах для медиков.

MediaPipe находит применение в неожиданных областях в том числе и в реальном мире:

- фитнес-трекеры – системы анализируют позу во время упражнений, подсчитывают повторения и дают обратную связь через смартфон;
- цифровые ассистенты – распознавание жестов «громче», «тише» или «пауза» в умных колонках;
- инклюзивные технологии – управление компьютером без касаний для людей с ограниченной моторикой;
- робототехника – дроны, следующие за указательным пальцем оператора, или промышленные манипуляторы, реагирующие на жесты.

В отличие от специализированных решений, MediaPipe не требует дополнительного оборудования – достаточно камеры ноутбука или телефона. Это упрощает доступ к технологиям, позволяя студентам и исследователям внедрять сложные функции без больших бюджетов.

1.3 Telegram Bot API

Telegram Bot API [5] – это интерфейс для разработки программных агентов или ботов в мессенджере Telegram, который позволяет автоматизировать взаимодействие с пользователями, обрабатывать входящие сообщения и отправлять ответы в режиме реального времени. Данный инструмент предназначен для упрощения создания сервисов, способных оперативно реагировать на запросы, выполнять различные задачи и интегрироваться с внешними системами, будь то информационные сервисы, системы поддержки клиентов или развлекательные проекты.

Основная идея Telegram Bot API заключается в использовании протокола HTTP для обмена данными между сервером Telegram и сервером разработчика. Разработчик получает обновления – новые сообщения, запросы или события – и обрабатывает их согласно заложенной логике приложения. Для получения обновлений предусмотрены два способа: периодический опрос сервера и автоматическая отправка обновлений на заранее настроенный адрес посредством веб-перехватчиков. Такой подход позволяет организовать надежное и гибкое взаимодействие, отвечающее требованиям самых разнообразных проектов.

Среди возможностей API можно выделить широкий спектр функций:

- отправку текстовых сообщений, медиа-файлов, документов и местоположения;
- использование встроенных и настраиваемых клавиатур для упрощения взаимодействия с пользователем;
- поддержку групповых чатов и каналов;
- реализация сложных сценариев взаимодействия, таких как обработка запросов при вводе данных непосредственно в поле ввода.

Кроме того, API позволяет работать с анимациями, опросами и даже организовывать платежные операции, что делает его универсальным

инструментом для создания интерактивных и функциональных приложений.

Одним из значимых преимуществ Telegram Bot API является его простота интеграции с языком Python. Благодаря множеству специализированных библиотек, разработка ботов становится интуитивно понятной и быстрой. Эти библиотеки оборачивают все сложности взаимодействия с API, предоставляя высокоуровневые функции для отправки сообщений, обработки команд и работы с различными типами данных. Поддержка асинхронного программирования позволяет создавать масштабируемые решения, способные обрабатывать большое количество одновременных запросов без потери производительности.

Работа с Telegram Bot API на Python часто строится вокруг двух основных подходов: постоянного опроса сервера и использования веб-перехватчиков. При первом подходе приложение регулярно запрашивает у серверов Telegram новые обновления, что удобно для разработки и тестирования, а также для небольших проектов. Второй метод, основанный на автоматической доставке обновлений на сервер разработчика, позволяет значительно снизить задержки и эффективно использовать ресурсы при создании коммерческих и высоконагруженных систем.

Кроме того, Python-среда предоставляет богатые возможности для отладки, тестирования и масштабирования ботов. Широкая экосистема инструментов и активное сообщество разработчиков способствуют быстрому решению возникающих задач, обмену опытом и постоянному обновлению библиотек в соответствии с изменениями Telegram Bot API. Это позволяет создавать надежные и безопасные сервисы, отвечающие самым высоким стандартам качества и оперативности.

Telegram Bot API является мощным и универсальным инструментом для разработки ботов, способных автоматизировать взаимодействие с пользователями и решать широкий спектр задач. Его тесная интеграция с Python делает процесс создания и поддержки таких систем доступным даже для небольших команд, позволяя быстро внедрять инновационные решения в самые разные сферы деятельности.

1.4 Обзор аналогов

На текущий момент на рынке представлено весьма ограниченное число коммерческих аналогов разрабатываемому приложению – как правило, большинство подобных реализации являются научными или учебными. В связи с этим, для обзора аналогов были подобраны наиболее подходящие решения, представленные на рынке:

1. SignAll [6]. SignAll – автоматическая система перевода языка жестов в текст. Представляет собой современное решение для обеспечения коммуникации между людьми с нарушениями слуха и окружающими. Используя комбинацию компьютерного зрения и методов NLP, платформа достигает высоких показателей точности и скорости, оставаясь при этом доступной через облачные сервисы и мобильные устройства. SignAll

поддерживает перевод как в реальном времени – с камеры, так и по предзаписанным фото и видеофайлам, автоматически выделяя ключевые точки рук, лица и тела для последующего анализа. Система синхронизирует поток сигналов из нескольких видеокамер и глубинного датчика, объединяя их для реконструкции 3D-движений жестов. Пример работы SignAll представлен на рисунке 1.3 [7].

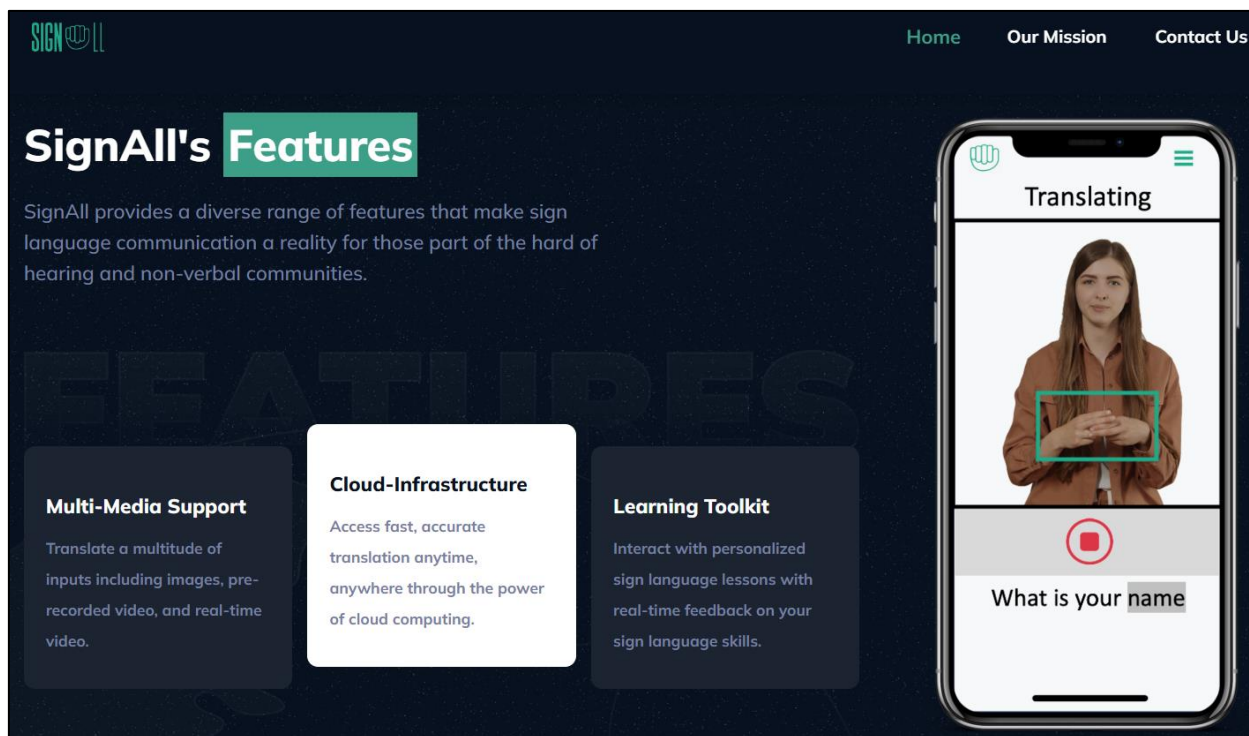


Рисунок 1.3 – Пример работы SignAll

2. Vidby [8]. Vidby специализируется на автоматическом переводе и дубляже видеоматериалов. Достижение точности перевода до 99 % обеспечивается за счет продвинутых нейросетевых алгоритмов последнего поколения, а опция последующей проверки человеком позволяет гарантировать соблюдение стилистики и терминологической консистентности исходного материала. Платформа предоставляет встроенное клонирование голоса, способное воссоздать тембр и интонацию оригинального диктора, а также функцию автоматической синхронизации губ, работающую в режиме онлайн. Кроме того, Vidby автоматически генерирует субтитры в различных форматах с возможностью настройки шрифта и расположения на экране. На рынке Vidby выделяется также наличием сервиса «Call Translator», предназначенного для перевода видео- и аудиозвонков в режиме реального времени через платформы Google Meet, Zoom и Microsoft Teams. Сервис автоматически распознает речь участников, переводит ее на выбранный язык и выводит субтитры прямо во время конференции. Такая функциональность уже сегодня позволяет распределенным командам из разных стран проводить общие совещания без языковых барьеров. Пример применения Vidby для перевода в реальном времени представлен на рисунке 1.4. [9].

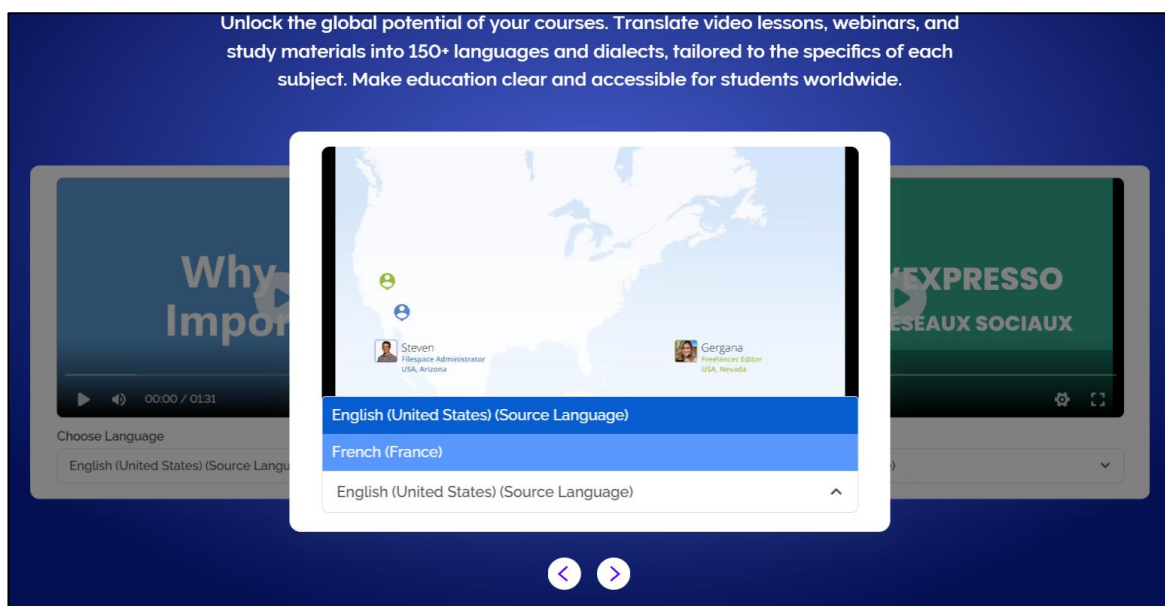


Рисунок 1.4 – Пример применения Vidby

3. HeyGen [10]. HeyGen представляет собой платформу для генерации видео с виртуальными персонажами на основе текстовых сценариев. С помощью HeyGen можно создавать более 120 различных AI-персонажей и применять свыше 300 голосовых генераторов, адаптированных под разные языки и стили речи Triad City Beat. Одной из уникальных функций является Instant Avatar – быстрое создание цифрового двойника пользователя по одной фотографии и короткому видеосегменту. HeyGen также поддерживает интерактивных персонажей для онлайн-коммуникаций, которые реагируют в режиме реального времени, что особенно полезно для продаж и онлайн-выступлений. Платформа предлагает инструменты для встраивания возможностей генерации видео в сторонние приложения и позволяет автоматически переводить созданный контент на другие языки. Пример цифровых аватаров, предоставляемых HeyGen представлен на рисунке 1.5 [11].

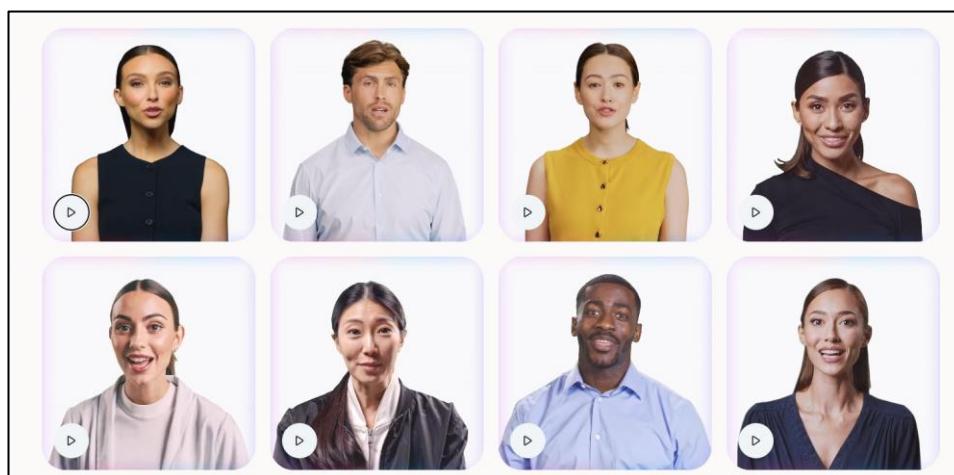


Рисунок 1.5 – Пример цифровых аватаров, предоставляемых HeyGen

4. Wavel AI [12]. Wavel AI фокусируется на всестороннем редактировании и локализации видео- и аудиоматериалов. Основные возможности включают автоматическую генерацию субтитров, перевод озвучки на более чем 100 языков с подбором более 250 голосов, а также функцию клонирования голоса для создания реалистичных дубляжей на основе образцов живой речи. Кроме перевода, Wavel AI предлагает набор инструментов для постобработки видео: авто-кадрирование, интеграция простых AI- персонажей и ручное тонкое редактирование полученных результатов. Платформа обеспечивает до 99% точности и в пять раз более высокую скорость подготовки локализованного контента по сравнению с традиционными методами. Пример работы автоматического перевода Wavel AI представлен на рисунке 1.6 [13].

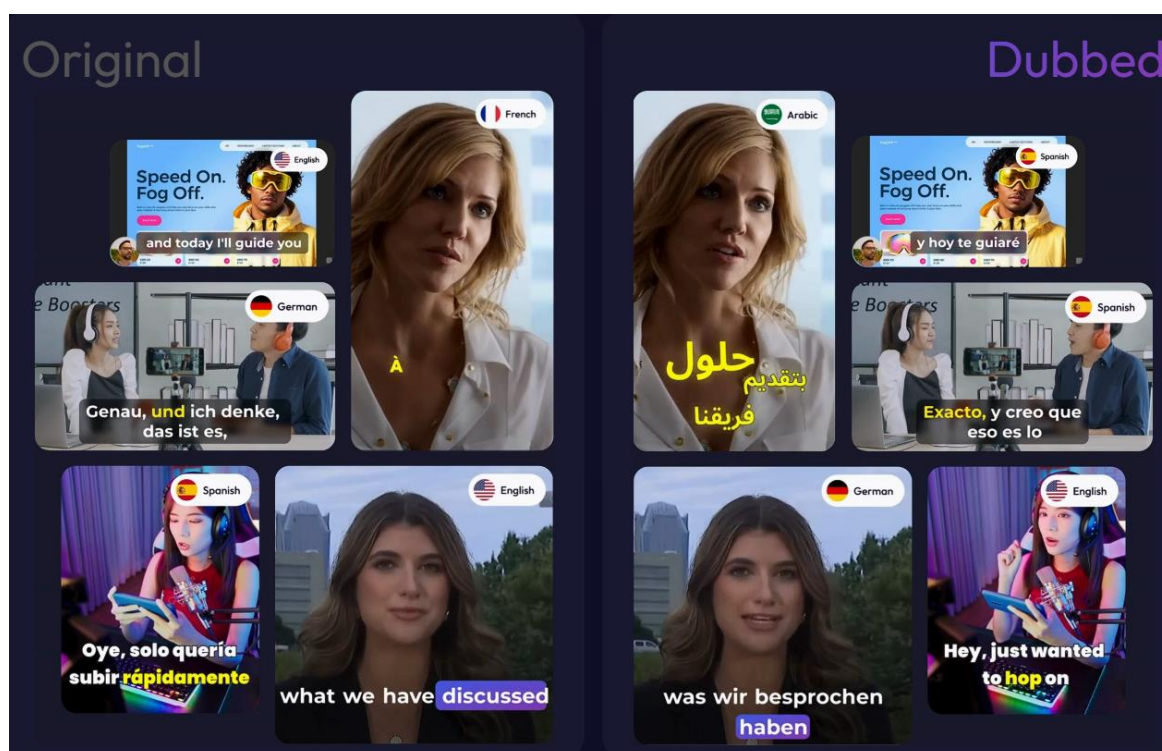


Рисунок 1.6 – Пример работы автоматического перевода Wavel AI

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

2.1 Перечень структурных блоков проекта

В данном дипломном проекте были выделены следующие структурные блоки:

1. Блок взаимодействия с пользователем.
2. Блок интеграции с телеграм-ботом.
3. Блок модели нейронной сети.
4. Блок обучения нейронной сети.
5. Блок тестирования.
6. Блок подсчета потерь.
7. Блок набора данных.
8. Блок подготовки набора данных.

Блоки выделены по принципу исполняемой задачи, то есть каждый блок представляет собой самостоятельный модуль проекта, выполняющий определенную функцию независимо от других блоков.

Подробное описание каждого блока, а также их взаимодействие между собой, представлено ниже. Схема структурная, включающая все перечисленные блоки и взаимодействие между ними, представлена на чертеже ГУИР.400201.074 С1.

2.2 Блок взаимодействия с пользователем

Блок взаимодействия с пользователем описывает взаимодействие конечного пользователя с системой. Взаимодействие осуществляется через интерфейс телеграм-бота, который принимает входные данные в виде видеозаписи или видео-сообщения, а также команды и нажатие кнопок. Для начала работы пользователю необходимо отправить боту соответствующий медиа-контент, содержащий жестовую речь. Бот поддерживает загрузку видеофайлов в формате .mp4, а также видео-сообщений, записанных непосредственно в Telegram.

После получения видео бот обрабатывает его и направляет пользователю ответное текстовое сообщение, содержащее расшифровку переданных жестов. Коммуникация осуществляется в асинхронном режиме: пользователь отправляет видео, после обработки получает результат в виде текстового ответа. В случае возникновения ошибок, например, если формат видео не поддерживается или качество записи недостаточно для корректного распознавания, бот уведомляет пользователя об этом и предлагает повторить попытку с другими параметрами.

Кроме того, бот может предоставлять вспомогательную информацию, например, инструкции по использованию, оформление подписки для обработки без ограничений, сбор статистики и аналитики использования бота и систему отзывов и обратной связи. Это обеспечивает удобное и интуитивно понятное взаимодействие, позволяя пользователям получать текстовую

интерпретацию жестовой речи без необходимости использования дополнительных инструментов, а также делает пользовательский опыт максимально комфортным.

2.3 Блок интеграции с телеграм-ботом

Блок обработки запросов описывает обработку входящих запросов в телеграм-боте в несколько этапов. После получения сообщения от пользователя, бот выполняет декомпозицию запроса, определяя его тип и корректность данных. Данный блок также инициализирует все обработчики сообщений и конечные сервисы, когда их использование требуется для выполнения команды. Если сообщение содержит видеофайл или видео-сообщение, система извлекает необходимые метаданные, проверяет формат и размер файла, а затем сохраняет его для дальнейшей обработки.

После успешного приема видео, формируется запрос к блоку обученной модели, передавая видеозапись для анализа. Точно так же данный блок перенаправляет сообщения в менеджеры оплаты, статистики и аналитики, и обратной связи.

Данный блок служит прослойкой между непосредственно интерфейсом Telegram API и бизнес-логикой приложения, связывая их воедино и управляя нагрузкой на второстепенные сервисы приложения.

2.4 Блок модели нейронной сети

Блок модели нейронной сети является ключевым компонентом системы, отвечающим за распознавание языка жестов на видеозаписях, и ее преобразование в текстовый формат. Входными данными для блока служит видеопоследовательность, содержащая язык жестов, а также прошедшая предобработку в соответствующем блоке.

Основная задача модели – анализ движений рук и положения пальцев человека на видео, а также интерпретация этих параметров в соответствии с заложенными языковыми правилами.

Процесс обработки видео основан на использовании модели MediaPipe, которая применяет методы компьютерного зрения для детекции и отслеживания ключевых точек рук. Сначала модель анализирует входное видео, выделяя последовательность кадров и извлекая координаты суставов и пальцевых сегментов. Затем эти данные передаются в классификационный модуль, который интерпретирует жесты на основе предварительно обученных шаблонов. Распознанные жесты сопоставляются с текстовыми эквивалентами, после чего модель формирует итоговую текстовую расшифровку и передает ее обратно в систему.

Высокая точность распознавания достигается за счет предварительной подготовки модели, включающей обучение на специализированных наборах данных с аннотированными жестами. Алгоритмы модели адаптированы для работы с реальными пользовательскими записями, что позволяет

обрабатывать жестовую речь в различных условиях, таких как изменяющееся освещение, различное качество видео и индивидуальные особенности исполнения жестов.

Модель способна обрабатывать жесты, исполняемые как одной, так и двумя руками, что значительно расширяет область применения модели и ее актуальность.

2.5 Блок обучения нейронной сети

По архитектуре модель является гибридной нейросетью, включающей три основных модуля:

1. CNN. Первым модулем является сверточная нейронная сеть, отвечающая за извлечение пространственных признаков из отдельно взятых кадров видео. Кадры обрабатываются независимо друг от друга. Признаки также проходят слои урезания и нормализации, что выделяет наиболее явные из них.

2. LSTM. Двухнаправленный анализ временных зависимостей помогает скрепить признаки, найденные сверточной нейронной сетью в единую последовательность, что помогает распознавать фразы из жестов, а не только отдельные знаки.

3. Attention. Механизм внимания – выполняет проверяющую и надзорную роль, выделяя ключевые моменты в последовательности признаков и создает конечный вектор признаков весов, из которого уже, после преобразования, и получаются текстовые метки и конечное предложение.

Обучение модели заключается в обработке и анализе выходных данных графов MediaPipe, представляющих собой координаты ключевых точек рук и лица на каждом кадре видео. Эти данные формируют основу для распознавания жестов, позволяя системе анализировать движения во времени.

Сеть обучается на размеченном корпусе данных, содержащем пары входных данных и соответствующих им целевых значений. В данном случае входными данными являются координаты ключевых точек, выделенные графами MediaPipe, а целевыми значениями – текстовые метки, описывающие жесты. На этапе обучения применяется алгоритм оптимизации, который корректирует веса нейронов в каждом слое сети с целью минимизации разницы между предсказанными результатами и истинными значениями, что позволяет снизить ошибку предсказания.

Для корректной работы модели используются функции активации, которые играют ключевую роль в определении, какие нейроны будут активированы в процессе вычислений. Функция активации обеспечивает нелинейность модели, что позволяет сети эффективно моделировать сложные зависимости в данных.

Так, функция Rectified Linear Unit ускоряет обучение за счет своей простоты и способности эффективно пропускать положительные значения, что уменьшает вычислительную сложность и помогает избежать проблемы затухающего градиента.

На выходном слое сети применяется функция SoftMax, которая преобразует выходные значения нейронов в вероятностное распределение по классам. Это позволяет интерпретировать результаты работы модели как вероятности принадлежности к определенным классам, что особенно важно при решении задач классификации распознанных жестов. После завершения обучения модель сохраняется в оптимизированном формате для дальнейшего использования в системе распознавания жестов.

2.6 Блок тестирования

На этапе тестирования модели необходимо, чтобы были полностью подготовлены блоки входных данных, предобработки и сама модель. Тестирование представляет собой процесс измерения производительности модели на новых, ранее не использованных данных. Этот этап позволяет оценить, насколько эффективно модель обобщает полученные знания, а также выявить возможные проблемы, такие как переобучение.

Тестирование в первую очередь проводится для оценки производительности. С помощью таких метрик, как точность (от англ. precision), полнота (от англ. recall), F1-мера и IoU (от англ. Intersection over Union), можно объективно оценить эффективность работы модели.

Также в ходе тестирования выявляется переобучение. Если модель демонстрирует хорошие результаты на обучающих и проверочных данных, но показывает низкие показатели на тестовых данных, это может свидетельствовать о переобучении.

Тестирование позволяет проводить сравнение различных вариантов модели, что помогает выбрать оптимальное решение для поставленной задачи.

Такой структурированный подход к тестированию позволяет не только оценить качество работы модели, но и выявить слабые места, требующие доработки, что в итоге способствует повышению ее эффективности и надежности.

2.7 Блок подсчета потерь

Блок подсчета потерь отвечает за вычисление функции потерь – математического выражения, которое измеряет разницу между предсказанными значениями модели и истинными метками. Этот блок является ключевым для корректировки весов нейронной сети, так как минимизация потерь приводит к улучшению точности распознавания.

В данный блок могут быть интегрированы следующие функции:

1. Среднеквадратичная ошибка (от англ. Mean Squared Error, MSE). Эта функция вычисляет среднее значение квадратов разностей между предсказанными значениями и истинными метками. За счет возведения ошибки в квадрат большие расхождения оказываются сильнее наказаны, что способствует более точной настройке весов.

2. Средняя абсолютная ошибка (от англ. Mean Absolute Error, MAE).

MAE измеряет среднее абсолютное отклонение предсказанных значений от истинных. Эта функция менее чувствительна к выбросам, так как не возводит разницу в квадрат, что может быть полезно, если данные содержат экстремальные значения.

3. Бинарная кросс-энтропия (от англ. Binary Cross-Entropy). Применяется для задач, где целевая переменная принимает два значения, например, 0 или 1. Эта функция измеряет разницу между предсказанными вероятностями и истинными бинарными метками, резко наказывая модель за уверенные, но неверные предсказания.

4. Категориальная кросс-энтропия (от англ. Categorical Cross-Entropy, CCE). Используется в задачах многоклассовой классификации, где целевая переменная может принимать более двух значений. Здесь истинные метки обычно представляются в виде вектора one-hot, и функция сравнивает распределение вероятностей, предсказанное моделью, с истинным распределением.

Выбор конкретной функции потерь зависит от специфики задачи: для регрессии используются функции, оценивающие разницу между числовыми значениями, а для классификации – функции, измеряющие расхождение между распределениями вероятностей. Эффективная реализация подсчета потерь обеспечивает надежное обновление весов модели, что является залогом успешного обучения и качественного распознавания входных данных.

2.8 Блок набора данных

Блок набора данных является основой системы распознавания языка жестов. Он включает тщательно собранный и размеченный корпус видеоматериалов, где каждая запись снабжена текстовой транскрипцией. Важнейшим этапом является разметка данных с использованием MediaPipe, который автоматически извлекает ключевые точки рук, формируя базу признаков для обучения модели.

Данные для обработки и дальнейшего обучения были взяты из одного из крупнейших русскоязычных наборов данных с видеоматериалами языка жестов – Slovo [14]. Набор данных содержит сырые отрывки языка жестов с базой более двадцати тысяч видеофайлов для более чем тысячи жестов. Таким образом, получается, что на каждый жест, в среднем, приходится около двадцати уникальных видеофайлов. Также в наборе данных предварительно есть файл аннотаций, связывающий уникальный идентификатор видео с названием жеста, которой в нем продемонстрирован.

После предварительной обработки видео проходит автоматическую разметку через MediaPipe, чтобы получить данные о местоположении векторов, наложенных в каждом конкретном кадре на руки участника видео. Полученный набор данных делится на обучающую и проверочную выборки, позволяя оптимизировать модель и объективно оценивать ее эффективность на новых данных. Модель способна работать как с жестами одной руки, так и двумя, что также предусмотрено в реализации MediaPipe.

2.9 Блок подготовки набора данных

Блок подготовки набора данных отвечает за предварительную обработку и структурирование исходных видеоматериалов, предназначенных для инференса модели распознавания жестов. На данном этапе выполняется сбор, очистка и стандартизация данных, что позволяет повысить качество разметки и обеспечить корректное извлечение признаков.

Данный блок схож с блоком набора данных, однако работает он только с живыми пользовательскими видео, которые получает телеграм-бот. Специфика работы с такими данными – у них может сильно варьироваться качество освещенности, разрешение видео или стабильность нахождения рук в кадре. Это также стоит учитывать при обработке видеофайлов, получаемых от конечных пользователей.

Данные точно так же обрабатываются с помощью функций MediaPipe, из видео извлекаются тензоры, которые, в дальнейшем, передаются в модель, для которой служат входными данными.

Дополнительно реализуются методы аугментации данных, такие как изменение масштаба и поворота, что способствует приближению данных к формату, близкому к тому, который был в обучающем наборе.

Данный блок также учитывает, что пользователь может использовать только одну руку в кадре, что также требует грамотной предобработки. Так, при не обнаружении второго набора координат, они заполняются нулями, которые, в конечно счете, игнорируются моделью и никак не сказываются на конечном результате.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

3.1 Архитектура программного модуля

Программный модуль телеграм-бота с поддержкой языка жестов представлен следующими основными функциональными блоками:

- блок модели нейронной сети;
- блок обучения нейронной сети;
- блок взаимодействия с пользователем;
- блок интеграции с телеграм-ботом;
- блок подготовки набора данных;
- блок набора данных.

Описание архитектуры каждого из блоков представлено ниже. Диаграмма классов телеграм-бота и диаграммы последовательностей представлены в чертежах ГУИР.400201.074 РР.1, ГУИР.400201.074 РР.2 и ГУИР.400201.074 РР.3.

3.2 Блок модели нейронной сети

Нейронная сеть, предназначенная для конечной классификации жестов по извлеченным из видео последовательностям специальных меток. Каждый компонент нейронной сети включает в себя специализированные слои и подмодули, обеспечивающие извлечение пространственно-временных признаков из видеопоследовательностей, и преобразование их в текстовую расшифровку. Сеть построена на основе гибридной архитектуры и представляет из себя комбинацию сверточной нейронной сети и двунаправленного LSTM. Блок включает классы `SignLanguageModel` и `ModelHandler`.

Класс `SignLanguageModel` реализует гибридную архитектуру нейросети CNN-LSTM с функцией внимания. Сверточная нейросеть извлекает признаки из каждого кадра, двусторонний LSTM находит между ними временную связь и заключает в последовательность, а функция внимания определяет наиболее весомые из признаков и формирует конечный вектор выходных меток. Данный класс обладает следующими полями:

- `cnn = nn.Sequential` – сверточная нейронная сеть, ответственная за извлечение признаков из кадров видео;
- `lstm = nn.LSTM` – рекуррентная нейронная сеть, находящая зависимости между признаками, выделенными сверточной нейросетью;
- `attention = nn.Sequential` – нейронная сеть, выполняющая функцию внимания и ответственная за вычисление важности временных шагов;
- `classifier = nn.Sequential` – преобразователь весов классов, полученных после предыдущего этапа в связанный текст.

Данный класс реализует следующие методы:

1. Метод `__init__(self, config)` – конструктор класса, инициализирующий все модули гибридной нейросети, а также выставяющий гиперпараметры модели в соответствии с конфигурацией. Данный метод принимает следующий параметр:

- `config: Tuple` – набор конфигурационных настроек для инициализации параметров моделей.

2. Метод `forward(x)` отвечает за инициализацию всех слоев и их связь друг за другом. Данный метод явно показывает, как именно, друг за другом, должны вызываться элементы модели. Метод принимает следующие параметры:

- `x: Tensor` – входной тензор с координатами ключевых точек для всей последовательности видео.

Класс `ModelHandler` отвечает за всю логику работы с моделью: загрузку, нормализацию и предсказание. Данный класс обладает следующими полями:

- `checkpoint: dict` – данные, загруженные во время последнего сохранения, которые включают веса модели, параметры архитектуры и классы меток;

- `config: dict` – конфигурация модели, включающая все ее гиперпараметры;

- `label_encoder: LabelEncoder` – преобразователь строковых меток в числовые и обратно;

- `mean, std: np.ndarray` – среднее и стандартное отклонение для нормализации данных;

- `model: SignLanguageModel` – инициализированная модель с загруженными весами.

Данный класс реализует следующие методы:

1. Метод `__init__(model_path, target_words)` – конструктор класса. Инициализирует словарь меток, который будет использоваться для преобразования выходных индексов в текстовые метки, а также модель, которая используется для обработки. Метод принимает следующие параметры:

- `model_path: str` – путь к файлу модели нейронной сети;

- `target_words: list[str]` – список меток, где каждый индекс соответствует метке, предсказанной моделью.

2. Метод `predict_sequence(self, sequence: np.ndarray) -> List[Tuple[str, float]]` выполняет инференс модели для видео, попадающих на сервер от телеграм-бота. Выполняется нормализация, преобразование в тензор, предсказание, а также преобразование этого предсказания в текстовую строку. Метод принимает следующий параметр:

- `sequence: np.ndarray` – последовательность признаков, полученных сверточной моделью и связанных через рекуррентную модель.

3.3 Блок обучения нейронной сети

В этом блоке описан класс, который инкапсулирует весь процесс обучения и проверки нейросети. Блок отвечает за разбивку набора данных на обучающую и проверочную части, реализацию гибридной архитектуры нейросети и ее обучение. Блок включает классы `SignLanguageDataset` и `AdvancedTrainer`.

Класс `SignLanguageDataset` выполняет загрузку, аугментацию и нормализацию данных. Класс также дополнительно добавляет шумы и сдвигает кадр для усложнения тренировки и приближения к реальным условиям. Данный класс обладает следующими полями:

- `data_dir: str` – путь к каталогу с тензорами;
- `annotations: str` – соотношение тензоров с текстовыми метками;
- `is_train: Boolean` – флаг тренировочного режима;
- `mean, std: np.ndarray` – параметры нормализации данных;
- `label_encoder: LabelEncoder` – преобразует текстовые метки в числа и наоборот;
- `noise_level: float` – уровень шума для аугментации;
- `max_time_shift: float` – максимальный сдвиг кадров.

Данный класс реализует следующие методы:

1. Метод `__init__(self, annotations, data_dir, target_words, mean=None, std=None, is_train=False)` – конструктор, отвечает за настройку всего процесса обучения. устанавливает пороги аугментации и смещения кадров, инициализирует функцию потерь и оптимизатор с заданными параметрами, а также заводит пустые списки для хранения метрик. Метод принимает следующие параметры:

- `data_dir: str` – путь к каталогу с тензорами;
- `annotations: str` – соотношение тензоров с текстовыми метками;
- `target_words: arr[str]` – словарь текстовых меток, которые модель использует для проверки;
- `mean, std: np.ndarray` – параметры нормализации данных.

2. Метод `__len__()` возвращает количество параметров в наборе данных. Данный метод не принимает дополнительных параметров.

3. Метод `__getitem__(idx)` загружает тензор по уникальному идентификатору из указанного файла, нормализует данные и преобразует тензор. Метод принимает следующий параметр:

- `idx: int` – индекс очередного тензора.

Класс `AdvancedTrainer` управляет процессом обучения, проверки и сохранением модели. В том числе, этот метод инициализирует гиперпараметры модели, выбирает устройство для вычисления, а также инициализирует функции потерь. Данный класс обладает следующими полями:

- `config: str` – словарь с параметрами модели и обучения;

- device: str – устройство для вычислений, GPU или CPU;
- model: SignLanguageModel – обучаемая модель;
- criterion: nn.CrossEntropyLoss – функция потерь с весами классов;
- optimizer: AdamW – оптимизатор AdamW;
- scheduler: lr_scheduler.ReduceLROnPlateau – планировщик скорости обучения;
- train_ds/val_ds: SignLanguageDataset – тренировочный и проверочный наборы данных.

Данный класс реализует следующие методы:

1. Метод `__init__(self, config)` – конструктор класса, отвечает за применение конфигурации к процессу обучения, выбирает устройство обработки и инициализирует метрики. Метод принимает следующий параметр:

- config: str – словарь с параметрами модели и обучения.

2. Метод `_prepare_data()` подготавливает данные перед обучением – загружает аннотации, разделяет данные на обучающие и проверочные, вычисляет метрики для нормализации. Метод не принимает дополнительных параметров.

3. Метод `_calculate_class_weights()` вычисляет веса классов обратно пропорционально их частоте, чтобы уравнивать все веса классов в наборе данных. Метод не принимает дополнительных параметров.

4. Метод `train_epoch(loader)` отвечает за обучение модели на обучающей выборке данных в рамках одной эпохи, фиксирует соответствующие метрики. Метод принимает следующий дополнительный параметр:

- loader: DataLoader – экземпляр DataLoader с фиксированными настройками.

5. Метод `evaluate(loader)` отвечает за обучение модели на проверочной выборке данных в рамках одной эпохи, фиксирует точность модели после эпохи. Метод принимает следующий дополнительный параметр:

- loader: DataLoader – экземпляр DataLoader с фиксированными настройками.

6. Метод `run_training()` служит инкапсулирующим методом для обучения, который включает в себе все другие методы обучения. Метод не принимает дополнительных параметров.

7. Метод `_save_checkpoint()` сохраняет экземпляр модели, конфигурацию и параметры нормализации, когда точность достигает нового максимального значения. Метод не принимает дополнительных параметров.

3.4 Блок взаимодействия с пользователем

В этом блоке сосредоточены классы второй линии обработки команд пользователя. В блоке реализуется разбор и декомпозиция запросов,

обработка текстовых сообщений, и далее – перенаправление этих данных из запросов в конкретные обработчики, отвечающие за сбор отзывов, статистику и аналитику. Все классы этого блока обеспечивают интерфейс, абстрагируя внутренние механизмы. Блок включает следующие классы `CommandManager`, `StatisticsManager`, `FeedbackManager` и `PaymentSystem`.

Класс `CommandManager` является ключевым классом блока, который декомпозирует и обрабатывает пользовательские команды и встроенные кнопки, а также формирует ответы, при этом не вникая в логику конвертации видео, учета статистики или проведения оплаты подписки. Он лишь делегирует эти задачи другим модулям. Данный класс обладает следующими полями:

- `stats_manager`: `StatisticsManager` – экземпляр модуля, собирающий статистику и аналитику по использованию телеграм-бота;
- `router`: `MessageRouter` – маршрутизатор, определяющий, как из методов данного класса должен быть задействован в зависимости от первичных признаков запроса;
- `payment_system`: `PaymentSystem` – экземпляр модуля, отвечающего за работу логики обработки платежей.

Данный класс реализует следующие методы:

1. Метод `async start(update: Update, context) → None` – метод, вызываемый после инициализации телеграм-бота, который приветствует пользователя, выводит информацию о возможностях бота и показывает клавиатуру для быстрого доступа. Метод принимает следующие параметры:

- `update`: `Update` – объект входящего обновления;
- `context`: `ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

2. Метод `async help(update: Update, context) → None` отправляет подробную инструкцию по использованию бота. Вызывается как по текстовой команде, так и по клику на встроенную кнопку. Метод принимает следующие параметры:

- `update`: `Update` – объект входящего обновления;
- `context`: `ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии пользователя.

3. Метод `async basic_stats (update: Update, context) → None` получает персонализированные данные статистики работы бота с пользователем и отправляет их в чат. Метод принимает следующие параметры:

- `update`: `Update` – объект входящего обновления;
- `context`: `ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

4. Метод `async detailed_stats (update: Update, context) → None` получает данные аналитики работы бота со всеми пользователями за все время, после чего отправляет их пользователю. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

5. Метод `async feedback(update: Update, context) → None` выводит пользователю предложение оставить отзыв на телеграм-бота и поделиться своим опытом, после чего выставляет специальный флаг, обозначающий следующее принятое сообщение как обратную связь. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

6. Метод `async show_reviews(update: Update, context) → None` выводит пользователю все имеющиеся отзывы и их оценки. Числовые оценки, в зависимости от их значения, конвертируются в соответствующее число звезд. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

7. Метод `async handle_callback(update: Update, context) → None` обрабатывает нажатие кнопок, которые отправляются ботом в сообщении при ответе. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

8. Метод `handle_reply_buttons(update: Update, context) → None` обрабатывает нажатие кнопок, которые закреплены в диалоге с ботом и доступны в любой момент времени. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

9. Метод `subscribe(update: Update, context) → None` отправляет пользователю запрос на ввод данных банковской карты для проведения оплаты за подписку, а далее – передает эти данные в специализированный обработчик, проверяющий введенные данные. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

10. Метод `process_card(update: Update, context) → None` отвечает за выдачу пользователю статуса платной подписки для неограниченной обработки видео. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

11. Метод `cancel_subscribe (update: Update, context) → None` обрабатывает возможность пользователя отменить процесс оплаты подписки при вводе соответствующих команд после запуска соответствующего процесса. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `context: ContextTypes.DEFAULT_TYPE` – объект контекста с бот-клиентом и данными сессии.

12. Метод `_send_response (update: Update, text) → None` является ключевым методом данного класса и передает ответы на все запросы пользователя дальше в Telegram API, чтобы в конечном счете он был доставлен пользователю. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `text: str` – сформированный текст ответа для пользователя, вне зависимости от запроса.

Класс `StatisticsManager` собирает и хранит данные индивидуальной статистики пользователей, а также общей аналитики по использованию телеграм-бота. Он хранит данные вне зависимости от текущей активной сессии и обновляет их скрыто, без уведомления пользователя. Данный класс обладает следующими полями:

- `stats_file: str` – json-файл, в котором непосредственно хранятся все данные статистики и аналитики;
- `data: dict` – временное хранилище данных аналитики, в котором данные находятся в процессе обработки.

Данный класс реализует следующие методы:

1. Метод `_init_empty_data() → Dict[str, Any]` инициализирует поле `data` класса, формируя структуру json-пакета и заполняя ее данными из базы. Метод не принимает каких-либо дополнительных параметров.

2. Метод `_load_or_create_stats() → None` инициализирует файл локальной базы данных, в которой хранятся данные для аналитики всех пользователей за все время. Если файл удален, пуст или не существует, то создает его и инициализирует. Метод не принимает каких-либо дополнительных параметров.

3. Метод `_load_hourly_stats(self, data: dict)` получает данные почасовой статистики использования бота всеми пользователями. После чего, вычисляет наиболее активный час в сутках за все время, когда бот использовался. Метод принимает следующий параметр:

- `data: dict` – данные, выгруженные из локальной базы данных.

4. Метод `_load_daily_stats (self, data: dict)` получает данные дневной статистики использования бота всеми пользователями. После чего, вычисляет наиболее активный день за все время, когда бот использовался. Метод принимает следующий параметр:

- `data: dict` – данные, выгруженные из локальной базы данных.

5. Метод `_load_total_stats (self, data: dict)` получает все данные пользователя из таблицы лидеров активности. Данные включают в

себя число обработанных видео-сообщений и видеофайлов, текстовых и голосовых сообщений. Метод принимает следующий параметр:

- `data: dict` – данные, выгруженные из локальной базы данных.

6. Метод `_save_stats()` → `str` обновляет данные индивидуальной статистики пользователей. Метод не принимает каких-либо дополнительных параметров.

7. Метод `get_user_stats(self, user_id: int)` → `str` возвращает данные индивидуальной статистики пользователя. Данные включают в себя состояние подписки и число отправленных различных данных в бот за все время работы. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, чьи данные нужно вернуть.

8. Метод `get_detailed_analytics()` → `str` возвращает данные общей аналитики использования бота, включающие таблицу наиболее активных пользователей и время наибольшей активности. Метод не принимает каких-либо дополнительных параметров.

9. Метод `record_activity(self, category: str, update)` возвращает данные индивидуальной статистики пользователя. Данные включают в себя состояние подписки и число отправленных различных данных в бот за все время работы. Метод принимает следующие параметры:

- `update: Update` – объект входящего обновления;
- `category: str` – тип активности, может являться текстовым, видео-, голосовым сообщением или видеофайлом.

Класс `FeedbackManager` реализует хранение отзывов пользователей в локальном хранилище данных. В классе реализованы возможности обновления и получения списка отзывов в корректной форме. Данный класс обладает следующими полями:

- `filepath: str` – путь к json-файлу, в который дозаписываются новые отзывы;

- `max_reviews: int` – ограничение на число одновременных отзывов;
- `reviews: deque` – временное хранилище отзывов при работе с ними.

Данный класс реализует следующие методы:

1. Метод `__init__(self, filepath: str = "feedback.json", max_reviews: int = 50)` инициализирует объект класса, устанавливает целевой файл для сохранения данных и число активных отзывов. Метод принимает следующие параметры:

- `filepath: str` – путь к файлу, в который будут дозаписываться новые отзывы;

- `max_reviews: int` – ограничение на число одновременных отзывов.

2. Метод `save(text: str, rating: int)` – формирует данные из отзыва пользователя в требуемую структуру и сохраняет новый отзыв. Отзыв состоит из текстового сообщения и оценки. Внутри метода данные формируются в соответствующую структуру `json` и сохраняются в файл. Метод принимает

следующие параметры:

- `text: str` – текст отзыва пользователя, который будет сохранен;
- `rating: int` – оценка пользователя.

3. Метод `_save_feedback()` выполняет непосредственное сохранение отзыва в файл. Отзыв сохраняется в конец `json`-пакета. В случае, если достигнут лимит отзывов, то самый старый отзыв удаляется. Метод не принимает каких-либо дополнительных параметров.

4. Метод `_load_feedback()` инициализирует локальное хранилище данных и выгружает отзывы в поле класса для дальнейшего использования. Метод не принимает каких-либо дополнительных параметров.

5. Метод `average_rating()` достает все оценки из файла отзывов и считает среднее арифметическое, после чего передает эти данные для учета в аналитике и выводе отзывов. Метод не принимает каких-либо дополнительных параметров.

6. Метод `get_recent_reviews()` формирует набор отзывов, которые будут отправлены пользователю. Метод не принимает каких-либо дополнительных параметров.

Класс `PaymentSystem` реализует логику проверки данных карты пользователей, а также отвечает за проведение оплаты за подписку. Данный класс обладает следующими полями:

- `subs_file: str` – путь к `json`-файлу, в котором хранятся данные подписок пользователей;
- `stats_manager: StatisticsManager` – экземпляр класса сбора статистики для обновления информации о подписке;
- `subscriptions: dict` – временное хранилище данных пользователей с подпиской при работе с ними.

Данный класс реализует следующие методы:

1. Метод `__init__ (self, stats_manager, subs_file: str = "subscriptions.json")` инициализирует объект класса, устанавливает целевой файл для сохранения данных и экземпляр класса статистики. Метод принимает следующие параметры:

- `subs_file: str` – путь к `json`-файлу, в котором хранятся данные подписок пользователей;
- `stats_manager: StatisticsManager` – экземпляр класса сбора статистики для обновления информации о подписке.

2. Метод `_load_subscriptions()` инициализирует локальное хранилище данных и выгружает данные пользователей с подпиской в поле класса для дальнейшего использования. Метод не принимает каких-либо дополнительных параметров.

3. Метод `_save_subscriptions()` – сохраняет данные о новой подписке со стороны пользователя. Внутри метода данные формируются в соответствующую структуру `json` и сохраняются в файл. Метод не принимает каких-либо дополнительных параметров.

4. Метод `process_payment(self, card_number: str, user_id: int) -> bool` обрабатывает запрос на платеж пользователя, проверяет данные, выдает соответствующие права после успешной оплаты. Метод принимает следующие параметры:

- `card_number: str` – номер карты, который ввел пользователь для оплаты;

- `user_id: int` – уникальный идентификатор пользователя, который проводит оплату.

5. Метод `check_subscription(self, user_id: int) -> bool` проверяет наличие подписки у пользователя. Метод отмечает, если подписки пользователя истекла с момента последнего использования бота. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, который проводит оплату.

6. Метод `get_subscription_info(self, user_id: int) -> str` взаимодействует вместе с классом статистики. Он формирует дополнение к индивидуальной статистике, включающее статус подписки и срок до ее истечения. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, который проводит оплату.

7. Метод `_luhn_check(card_number: str) -> bool` реализует логику проверки правильности введенных данных карты пользователем. Метод проверяет число цифр в номере карты и их формат. Метод принимает следующие параметры:

- `card_number: str` – номер карты, который ввел пользователь для оплаты.

3.5 Блок интеграции с Telegram-ботом

В этом блоке реализована обертка Telegram API: создается объект приложения, регистрируются обработчики команд и сообщений, определяется логика приема обновлений и их передачи в соответствующие обработчики. Благодаря этому слою остальной код остается изолированным от деталей работы с сетью и протоколом Telegram. Блок включает классы `BotApplication`, `MessageRouter`.

Класс `BotApplication` является ключевым в приложении и главным классом, через который оно запускается. Класс также инициализирует все обработчики, которые вызывают методы других классов. Данный класс обладает следующими полями:

- `token: str` – уникальная метка Telegram-бота;
- `stats: StatisticsManager` – модуль для сбора статистики запросов;
- `model: GraphToTextModel` – загруженная модель нейросети;
- `router: MessageRouter` – маршрутизатор обработчиков видео и текста;

- `commands: CommandManager` – обработчик текстовых команд;
- `payment_system: PaymentSystem` – экземпляр класса сервиса оплаты;
- `app: Application` – объект приложения `python-telegram-bot`, в который регистрируются обработчики.

Данный класс реализует следующие методы:

1. Метод `__init__(self, token: str) → None` – метод, инициализирующий приложение, настраивает логирование, создает менеджер статистики и загружает веса нейросети. Метод принимает следующий параметр:

- `token: str` – токен бота для аутентификации в Telegram API.

2. Метод `_register_handlers(self) → None` инициализирует все обработчики приложения: для команд, встроенных кнопок, текстовых и видео-сообщений. Метод не принимает дополнительных параметров.

3. Метод `run(self) → None` запускает бота, фиксирует старт и приводит серверную часть бота в рабочее состояние. Метод не принимает дополнительных параметров.

Класс `MessageRouter` принимает обновления из Telegram и перенаправляет их либо в обработчик видео, либо в текстовый обработчик. Этот класс также ответственен за общение с TelegramAPI. Также хранит состояние ожидания отзыва для каждого чата. Данный класс обладает следующими полями:

- `stats: StatisticsManager` – экземпляр объекта сбора статистики и аналитики;
- `feedback_manager: FeedbackManager` – хранит отзывы пользователей;
- `payment_system: PaymentSystem` – экземпляр класса сервиса оплаты;
- `pending_feedback: set[int]` – множество чатов, из которых бот ожидает текст-отзыв.

Данный класс реализует следующие методы:

1. Метод `async handle_video(self, update: Update, context: ContextTypes.DEFAULT_TYPE) → None` извлекает видеофрагмент из сообщения пользователя, сохраняет его локально, после чего пропускает его через модель нейронной сети. Далее обратно передает пользователю полученный из нейронной сети текст. Метод принимает следующие параметры:

- `update: Update` – входящее обновление с контекстом из чата с пользователем;

- `context: ContextTypes.DEFAULT_TYPE` – контекст с бот-клиентом, значение установлено по умолчанию.

2. Метод `request_feedback(self, chat_id: int) → None` устанавливает флаг ожидания обратной связи, чтобы следующее текстовое

сообщение этого чата было интерпретировано как отзыв. Метод принимает следующий параметр:

- `chat_id: int` – идентификатор чата, ожидающего отзыв.

3. Метод `_init_user_data(self, user_id_str: str)` инициализирует данные нового пользователя, если ранее он не пользовался ботом. Метод принимает следующий параметр:

- `user_id_str: str` – уникальный идентификатор пользователя, написавшего в бот.

4. Метод `_check_access(self, user_id: int) -> bool` проверяет доступы пользователя к функциям бота. Доступ может быть отказан, в случае, когда число бесплатных попыток в день закончилось, а подписка не оформлена. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот.

5. Метод `_check_daily_limit(self, user_id: int) -> bool` проверяет непосредственный лимит для тех пользователей, у которых не оформлена платная подписка. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот.

6. Метод `_handle_limit_reached(self, update: Update, user_id: int)` формирует ответ в случае, когда у пользователя достигнут ежедневный лимит по обработкам. Метод принимает следующие параметры:

- `update: Update` – входящее обновление с контекстом из чата с пользователем;

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот.

7. Метод `_increment_daily_counter(self, user_id_str: str)` инкрементирует счетчик бесплатных расшифровок видео в день, пока он не достигнет лимита. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот.

8. Метод `_get_content_type(self, update: Update) -> str` определяет тип сообщения, которое отправил пользователь – видео-сообщение, голосовое сообщение, видеофайл или текст. Метод принимает следующий параметр:

- `update: Update` – входящее обновление с контекстом из чата с пользователем.

9. Метод `_build_response(self, user_id: int) -> str` формирует и отправляет ответ с расшифрованным текстом из видео пользователю. Метод принимает следующий параметр:

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот.

10. Метод `handle_text(self, update: Update)` обработчик текстовых сообщений от пользователя, с оповещением пользователя о том, что

бот предназначен для обработки видео. Метод принимает следующий параметр:

- `update: Update` – входящее обновление с контекстом из чата с пользователем.

11. Метод `_process_feedback(self, update: Update, text: str)` первоначальной обработки отзыва пользователя. Метод первично разбирает запрос пользователя и передает данные дальше для последующей обработки. Метод принимает следующие параметры:

- `user_id: int` – уникальный идентификатор пользователя, написавшего в бот;
- `text: str` – текст запроса, который содержит в себе также отзыв и оценку пользователя.

3.6 Блок подготовки набора данных

Данный блок отвечает за обработку данных, попадающих в бот. Он предварительно подготавливает видеофайл перед передачей его в обработку моделью. Обработка включает применение функций `MediaPipe`, нормализацию видео и приведение к единому формату файла.

Класс `VideoProcessor` извлекает ключевые точки рук из видео, присланного пользователем, и подготавливает их для модели. Данный класс обладает следующими полями:

- `frame_size: Tuple` – размер кадра после нормализации;
- `hands = self.mp_hands.Hands` – экземпляр класса `MediaPipe` для работы с отслеживанием рук;
- `seq_length: int` – фиксированная длина в кадрах выходной последовательности.

Данный класс реализует следующие методы:

1. Метод `__init__(self, frame_size=(640, 480), seq_length=16)` – конструктор класса, инициализирует конфигурационные параметры, такие как размер кадра после нормализации и длину выходной последовательности. Метод принимает следующие параметры:

- `frame_size: Tuple` – размер кадра после нормализации;
- `seq_length: int` – фиксированная длина в кадрах выходной последовательности.

2. Метод `process_video(self, video_path: str) -> Optional[np.ndarray]` обрабатывает видеопоследовательность и преобразует ее в набор ключевых точек. Метод принимает следующий параметр:

- `video_path: str` – путь к сохраненному видеофайлу.

3. Метод `_resample_frames(self, frames: List[np.ndarray]) -> List[np.ndarray]` равномерно отбирает кадры для фиксированной длины последовательности. Метод принимает следующий параметр:

- `frames: List[np.ndarray]` – список массивов с ключевыми

точками.

4. Метод `_process_frame(self, frame: np.ndarray) -> Optional[np.ndarray]` обрабатывает только один кадр для извлечения ключевых точек. Метод принимает следующий параметр:

– `frame: np.ndarray` – исходный кадр.

3.7 Блок набора данных

Этот блок отвечает за обработку и хранение набора данных для обучения модели. Он подготавливает набор данных – нормализует кадры видеопоследовательности, пропускает из через MediaPipe и формирует тензоры для подачи на вход модели.

Класс `DataSetProcessor` подготавливает набор данных из видеофайлов: извлекает ключевые точки рук, фильтрует данные по целевым словам и сохраняет их в формате тензоров. Данный класс обладает следующим полем:

– `hands = self.mp_hands.Hands` – экземпляр класса MediaPipe для работы с отслеживанием рук.

Данный класс реализует следующие методы:

1. Метод `__init__(self)` инициализирует класс параметрами из MediaPipe. Метод не принимает дополнительных параметров.

2. Метод `fix_encoding(path)` исправляет проблемы с кодировкой текстового файла с аннотациями, которые связывают текстовые метки и видео. Данный класс устанавливает кодировку UTF-8. Метод принимает следующий параметр:

– `path: str` – путь к текстовому файлу.

3. Метод `load_annotations()` загружает и обрабатывает .csv-файл с аннотациями, исправляет кодировку, а также достает только те текстовые метки, которые есть в `TARGET_WORDS`. Метод не принимает дополнительных параметров.

4. Метод `process_frame(frame)` извлекает ключевые точки рук из одного кадра с помощью MediaPipe. Метод принимает следующий параметр:

– `frame: np.ndarray` – исходный кадр.

5. Метод `process_video(video_path)` инкапсулирует метод обработки одного кадра и обрабатывает видео с помощью методов MediaPipe целиком. Метод принимает следующий параметр:

– `video_path: str` – путь к исходному видео.

6. Метод `run()` основной цикл обработки данных. Обрабатывает все видеофайлы, аннотации к которым были найдены согласно `TARGET_WORDS`. Метод не принимает дополнительных параметров.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

4.1 Разработка блока модели нейронной сети

Программная реализация опирается на архитектурные и функциональные решения, представленные в предыдущей главе. В этом разделе подробно рассматривается практическая реализация спроектированных слоев моделей. Особое внимание уделено обеспечению корректной работы системы с реальными данными, улучшению пользовательского опыта, а также достижению компромисса между качеством обработки изображений и производительностью. Полный текст программы приведен в приложении А.

Реализация класса `SignLanguageModel` является гибридной архитектурой нейросети CNN-LSTM с функцией внимания. Ниже приведен код реализации.

```
class SignLanguageModel(nn.Module):  
    def __init__(self, config):  
        super().__init__()
```

Инициализация сверточной нейронной сети и ее внутренних параметров. Первым этапом архитектуры для входных тензоров служит сверточная нейронная сеть, состоящая из нескольких сверточных слоев, слоев активации и отсечения. Входными данными для данной части гибридной архитектуры служат тензоры с числом входных признаков, равных 126. Это число получается из выходных данных MediaPipe – на каждой руке фиксируется по 21 точке графа, умножается на две руки и три плоскости измерения, откуда и получаем 126. Выходом слоя является 256 признаков в каждом из семи кадров, прошедших свертку. Ниже приведен рисунок 4.1 архитектуры сверточной части итоговой нейронной сети.

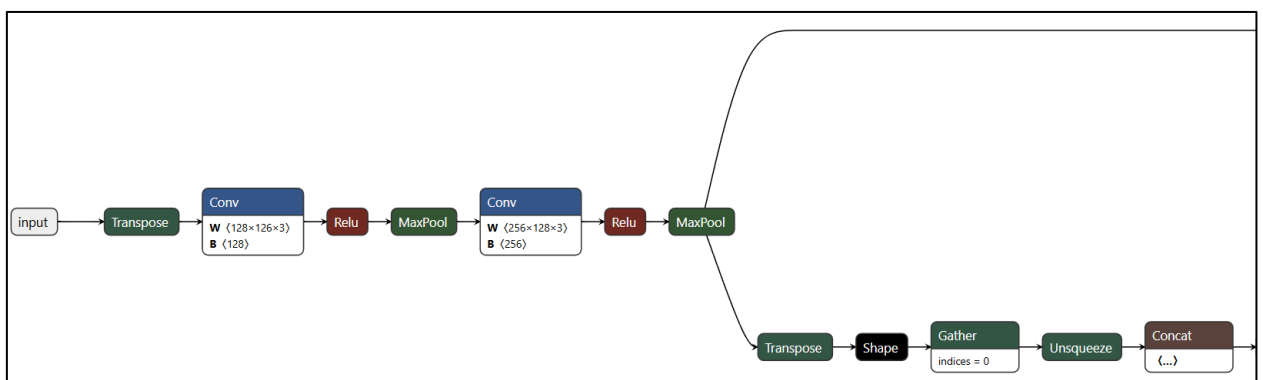


Рисунок 4.1 – Архитектура части CNN нейронной сети

Реализация инициализации данной части модели приведена ниже:

```
self.cnn = nn.Sequential(  

```

```

nn.Conv1d(config['input_size'], 128, 3, padding=1),
nn.BatchNorm1d(128),
nn.ReLU(),
nn.MaxPool1d(2),
nn.Dropout(config['dropout']),
nn.Conv1d(128, 256, 3, padding=1),
nn.BatchNorm1d(256),
nn.ReLU(),
nn.MaxPool1d(2),
nn.Dropout(config['dropout'])
)

```

Инициализация рекуррентной нейронной сети и ее внутренних параметров. Часть LSTM архитектуры отвечает за то, чтобы связать полученные ранее признаки в единую последовательность и выявить зависимости между ними. LSTM также является двунаправленной, что позволяет в два раза эффективнее исследовать признаки на зависимости друг между другом и отсекаать ненужные. Так, на вход мы получаем набор признаков из CNN, а блок обрабатывает и выдает связанную последовательность, из которой требуется выделить наиболее яркие признаки. Ниже приведен рисунок 4.2 архитектуры рекуррентной части итоговой нейронной сети.

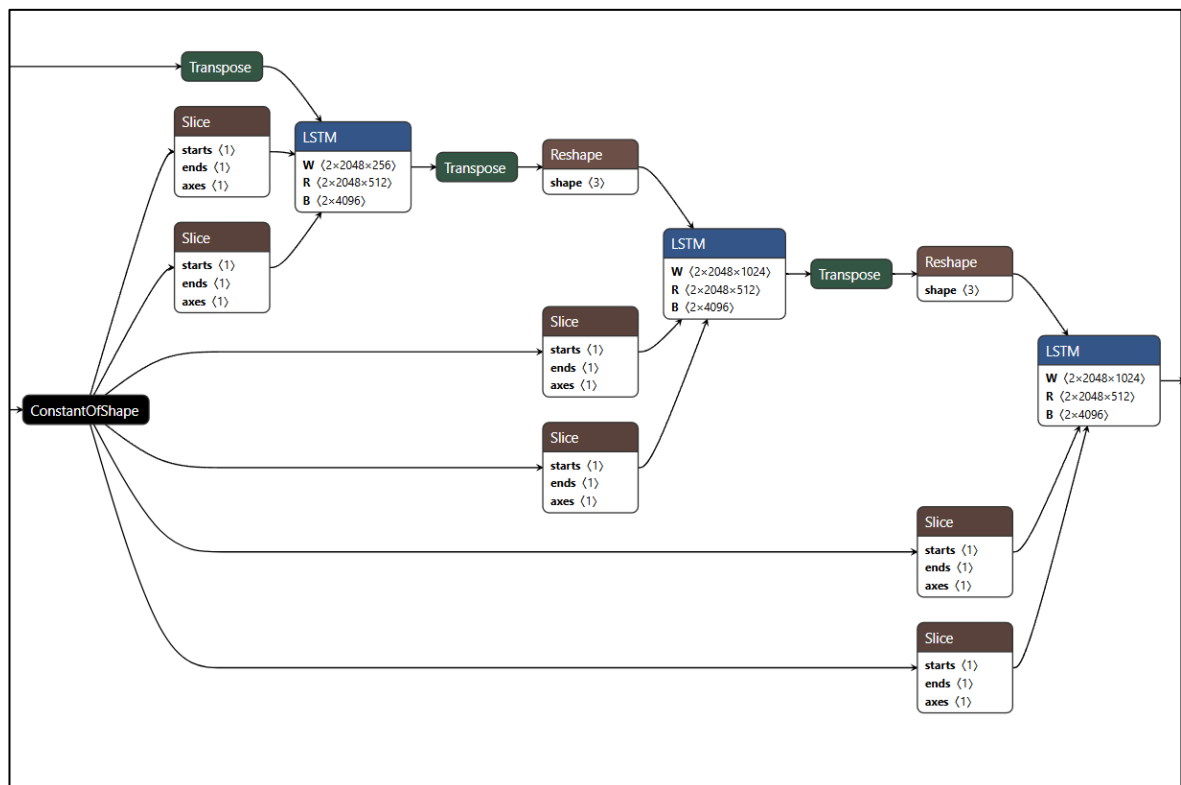


Рисунок 4.2 – Архитектура части LSTM нейронной сети

Реализация инициализации данной части модели приведена ниже:

```

self.lstm = nn.LSTM(

```

```

256,
config['hidden_size'],
num_layers=config['num_layers'],
bidirectional=True,
dropout=config['dropout'],
batch_first=True
)

```

Предпоследней архитектурной частью модели является функция внимания, построенная на линейных слоях. Задача этого блока – выделить наиболее важных признаков, полученных после блока LSTM. Выделение происходит путем вычисления скалярной величины важности признака, их нормализации и отсека менее важных признаков. Таким образом, входными данными служит вдвоенный вектор признаков, включающий оба направления прохода по исходному вектору, полученный из LSTM, а выходными – усеченный вектор наиболее ярких признаков. Ниже приведен рисунок 4.3 архитектуры линейной части итоговой нейронной сети.

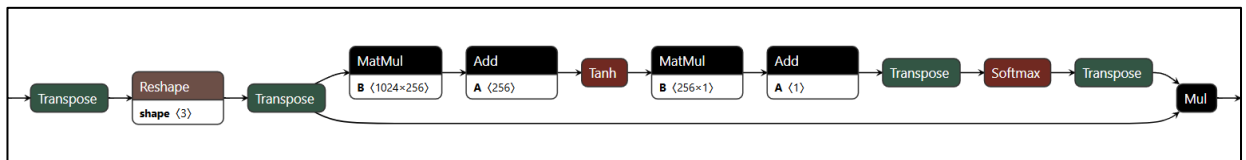


Рисунок 4.3 – Архитектура линейной части нейронной сети

Реализация инициализации данной части модели приведена ниже:

```

self.attention = nn.Sequential(
    nn.Linear(config['hidden_size'] * 2, 256),
    nn.Tanh(),
    nn.Linear(256, 1),
    nn.Softmax(dim=1)
)

```

Последним блоком архитектуры является классификатор. Основная задача данного слоя – преобразовать вектор после первичного усечения функцией внимания, и выдать конечный вектор признаков, состоящий из пятнадцати ключевых весов. Ниже приведен рисунок 4.4 архитектуры линейной части итоговой нейронной сети.

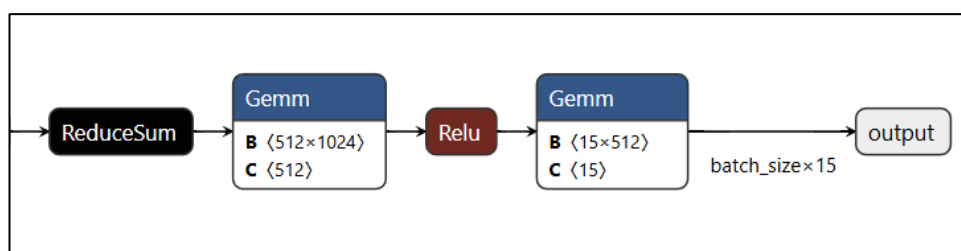


Рисунок 4.4 – Архитектура классификатора нейронной сети

Реализация инициализации данной части модели приведена ниже:

```
self.classifier = nn.Sequential(
    nn.Linear(config['hidden_size'] * 2, 512),
    nn.ReLU(),
    nn.Dropout(config['dropout']),
    nn.Linear(512, len(config['target_words'])))
)
```

Инициализация метода, описывающего последовательность соединения блоков модели в единую архитектуру. Метод, в первую очередь, задает очередность, с которой требуется связать различные слои и просто реализует интерфейс, но не вызывается сам. Ниже приведен код реализации.

```
def forward(self, x):
    x = x.permute(0, 2, 1) # [batch, channels, seq_len]
    x = self.cnn(x)
    x = x.permute(0, 2, 1) # [batch, seq_len, features]
    lstm_out, _ = self.lstm(x)
    attention_weights = self.attention(lstm_out)
    context = torch.sum(attention_weights * lstm_out, dim=1)
    return self.classifier(context)
```

Реализация класса `ModelHandler` отвечает непосредственно за инференс модели на реальных данных. Данный класс реализует финальную нормализацию данных перед предсказанием отвечает за получение результата использования модели.

```
class ModelHandler:
    def __init__(self, model_path: str, target_words: list):
        self.checkpoint = torch.load(
            model_path,
            map_location='cpu',
            weights_only=False
        )
        self.config = self.checkpoint['config']
        self.config['num_classes'] = len(target_words)
        self.checkpoint['model_state'].keys())
        self.label_encoder = LabelEncoder()
        self.label_encoder.classes_ = self.checkpoint
        self.mean = self.checkpoint['mean'].numpy()
        self.std = self.checkpoint['std'].numpy()
        self.model = SignLanguageModel(self.config)
        self.model.load_state_dict(self.checkpoint['model_state'])
        self.model.eval()
```

4.2 Реализация блока обучения нейронной сети

Реализация данного блока организует цикл обучения и проверки нейронной сети. В рамках реализации происходит разделение исходного

набора данных на обучающую и проверочную выборки с учетом заданного соотношения, инициализацию загрузчиков данных, конфигурацию оптимизатора и функции потерь, а также ведение подробного учета метрик на каждом этапе обучения. Алгоритм обучения нейронной сети приведен на схеме программы ГУИР.400201.074 ПД.

Реализация класса `SignLanguageDataset` отвечает за предобработку признаков из набора данных перед запуском их в нейросеть. Ниже приведен код реализации.

```
class SignLanguageDataset(Dataset):
    def __init__(self, annotations, data_dir, target_words,
mean=None, std=None, is_train=False):
        self.data_dir = data_dir
        self.annotations = annotations
        self.is_train = is_train
        self.mean = mean
        self.std = std

        # Кодирование меток
        self.label_encoder = LabelEncoder()
        self.label_encoder.fit(target_words)
        self.labels =
self.label_encoder.transform(annotations.iloc[:, 1])

        # Параметры аугментации
        self.noise_level = 0.02
        self.max_time_shift = 5
```

Реализация класса `AdvancedTrainer` отвечает непосредственно за обучение модели, включая прохождение данных через все слои, а также их разделение на обучающую и проверочную выборки. Ниже приведен код реализации.

```
class AdvancedTrainer:
    def __init__(self, config):
        self.config = config
        self.device = torch.device('cuda' if
torch.cuda.is_available() else 'cpu')
        self.writer = SummaryWriter()

        # Загрузка и подготовка данных
        self._prepare_data()

        # Инициализация модели
        self.model =
SignLanguageModel(config).to(self.device)

        # Функция потерь с балансировкой классов
        class_weights = self._calculate_class_weights()
        self.criterion
```

```

=nn.CrossEntropyLoss(weight=class_weights)

        # Оптимизатор и шедулер
        self.optimizer = optim.AdamW(
            self.model.parameters(),
            lr=config['lr'],
            weight_decay=config['weight_decay']
        )
        self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
            self.optimizer,
            patience=config['patience'] // 2,
            'max',
        )

```

Реализация метода подготовки данных перед передачей на обучение модели, включающий нормализацию, разделение на выборки и инициализация метрик. Ниже приведен код реализации.

```

def _prepare_data(self):
    # Загрузка аннотаций
    processor = DataSetProcessor()
    processor.load_annotations()
    train_df, val_df = train_test_split(
        processor.filtered_df,
        test_size=0.2,
        stratify=processor.filtered_df.iloc[:, 1],
        random_state=42
    )
    temp_ds = SignLanguageDataset(train_df, OUTPUT_DIR,
TARGET_WORDS)
    all_data = torch.cat([temp_ds[i][0] for i in
range(len(temp_ds))])
    mean = all_data.mean(dim=(0, 1))
    std = all_data.std(dim=(0, 1))
    self.train_ds = SignLanguageDataset(
        train_df, OUTPUT_DIR, TARGET_WORDS, mean, std,
is_train=True
    )
    self.val_ds = SignLanguageDataset(
        val_df, OUTPUT_DIR, TARGET_WORDS, mean, std
    )

```

Реализация метода тренировки модели на обучающей выборке и сохранения соответствующих метрик. Метод описывает обучение в рамках одной эпохи, и, в результате чего, появляется новая, более эффективная, модель. Ниже приведен код реализации.

```

def train_epoch(self, loader):
    self.model.train()
    total_loss = 0
    progress = tqdm(loader, desc="Training", leave=False)
    for inputs, labels in progress:

```

```

        inputs = inputs.to(self.device, non_blocking=True)
        labels = labels.to(self.device, non_blocking=True)
        self.optimizer.zero_grad()
        outputs = self.model(inputs)
        loss = self.criterion(outputs, labels)
        loss.backward()
    torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
    self.optimizer.step()
    total_loss += loss.item()
    progress.set_postfix({'loss': f"{loss.item():.4f}"})
    return total_loss / len(loader)

```

Реализация метода проверки модели на проверяющей выборке и сохранения соответствующих метрик. Метод описывает тестирование модели на проверяющем наборе данных и, если новая модель показывает результат лучше предыдущей, то сохраняет ее. Ниже приведен код реализации.

```

def evaluate(self, loader):
    self.model.eval()
    total_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(self.device)
            labels = labels.to(self.device)
            outputs = self.model(inputs)
            loss = self.criterion(outputs, labels)
            total_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return total_loss / len(loader), correct / total

```

Реализация метода полного цикла обучения модели по эпохам. Метод итеративно обучает модель и в конце каждой эпохи сверяет полученную точность. Если точность превышает текущую максимальную, то сохраняет новую, лучшую, модель. Ниже приведет код реализации.

```

def run_training(self):
    best_acc = 0
    no_improve = 0
    train_loader = DataLoader(
        self.train_ds,
        batch_size=self.config['batch_size'],
        shuffle=True,
        num_workers=4,
        pin_memory=True
    )
    val_loader = DataLoader(
        self.val_ds,

```



```

        batch_size=self.config['batch_size'],
        num_workers=4,
        pin_memory=True
    )
    for epoch in range(self.config['epochs']):
        # Обучение
        train_loss = self.train_epoch(train_loader)
        # Валидация
        val_loss, val_acc = self.evaluate(val_loader)
        self.scheduler.step(val_acc)
        # Ранняя остановка
        if val_acc > best_acc:
            best_acc = val_acc
            no_improve = 0
            self._save_checkpoint()
            print(f"Epoch {epoch + 1}: New best accuracy
{val_acc:.4f}")
        else:
            no_improve += 1
            if no_improve >= self.config['patience']:
                print(f"Early stopping at epoch {epoch + 1}")
                break

```

Реализация метода скачивания модели с наилучшими результатами в процессе обучения и сохранением всех основных метрик. Ниже приведен код реализации.

```

def _save_checkpoint(self):
    torch.save({
        'model_state': self.model.state_dict(),
        'config': self.config,
        'label_encoder':
self.train_ds.label_encoder.classes_,
        'mean': self.train_ds.mean,
        'std': self.train_ds.std
    }, "best_model.pth")

```

4.3 Реализация блока взаимодействия с пользователем

Реализация блока взаимодействия с пользователем описывает логику процесса работы с пользователем через Telegram-бота, что включает в себя логику декодирования и обработки команд, обработку текстовых сообщений, а также сбор отзывов и формирование статистики.

Реализация класса `CommandManager` представляет собой промежуточный обработчик пользовательских команды, реализуя связь между пользователем и бизнес-логикой приложения. Ниже приведен код реализации:

```

class CommandManager:
    GET_CARD = 0
    def __init__(self, stats_manager, router,

```

```

payment_system):
    self.stats_manager = stats_manager
    self.router = router
    self.payment_system = payment_system
    self._init_keyboards()
    logger.debug("Инициализирован CommandManager")

    def _init_keyboards(self):
        logger.debug("Создание клавиатур")
        self.inline_keyboard = InlineKeyboardMarkup([
            [InlineKeyboardButton("💎 Купить подписку",
callback_data='buy_subscription'),
            InlineKeyboardButton("📊 Общая аналитика",
callback_data='detailed_stats')],
            [InlineKeyboardButton("❓ Помощь",
callback_data='help')]
        ])

        self.reply_keyboard = ReplyKeyboardMarkup(
            [['📈 Моя статистика', '📊 Общая аналитика'],
            ['💎 Купить подписку', '⭐ Оставить отзыв'],
            ['❓ Помощь']],
            resize_keyboard=True,
            one_time_keyboard=False
        )

```

Реализация класса `StatisticsManager` — это агрегатор статистики и аналитики пользователей по использованию телеграм-бота. Он учитывает все обработанные сообщения и расшифрованные видео, ведет подсчет в форме метрик, также отслеживает статус подписки и ведет список активных пользователей. Ниже приведен код реализации.

```

class StatisticsManager:
    def __init__(self, stats_file: str = "stats.json"):
        self.stats_file = stats_file
        self.logger = logging.getLogger(self.__class__.__name__)
        self.data = self._init_empty_data()
        self._load_or_create_stats()
        self.logger.info("Статистика инициализирована")

    def _init_empty_data(self) -> Dict[str, Any]:
        self.logger.debug("Создание пустой структуры данных")
        return {
            'metadata': {
                'created_at': datetime.now().isoformat(),
                'last_updated': None
            },
            'total': {
                'video': 0,

```

```

        'voice': 0,
        'video_note': 0,
        'text': 0,
        'subscriptions': 0
    },
    'users': {},
    'hourly_activity': {},
    'daily_activity': {}
}

```

Реализация класса `FeedbackManager` хранит и собирает отзывы пользователей в формате текста и пятибалльной оценки. Ниже приведен код реализации.

```

class FeedbackManager:
    def __init__(self, filepath: str = "feedback.json",
max_reviews: int = 50):
        self.filepath = filepath
        self.max_reviews = max_reviews
        self.reviews = deque(maxlen=max_reviews)
        logger.info(f"Инициализация FeedbackManager.
Максимум отзывов: {max_reviews}, файл: {filepath}")
        self._load_feedback()

```

Реализация класса `PaymentSystem` отвечает за логику обработки и запросов оплаты подписки, а также всех действий, связанных с подпиской.

```

class PaymentSystem:
    def __init__(self, stats_manager, subs_file: str =
"subscriptions.json"):
        self.stats_manager = stats_manager
        self.subs_file = subs_file
        self.logger =
logging.getLogger(self.__class__.__name__)
        self.subscriptions = self._load_subscriptions()
        self.logger.info("Инициализирована платежная
система")

```

4.4 Реализация блока интеграции с Telegram-ботом

Реализация блока представляет собой обертку Telegram API, регистрирующей обработчики команд и сообщений. Здесь описывается логика взаимодействия с серверами Telegram, работы с его обработчиками.

Реализация класса `BotApplication` представляет собой создание экземпляров маршрутизатора и менеджера команд, а также регистрирует все обработчиков в объекте `Application` из библиотеки `python-telegram-bot`. Данный класс служит точкой входа в программу на сервере, и через него происходит первичная декомпозиция всех команд, приходящих из бота, он абстрагирует остальную бизнес-логику от взаимодействия с сетевыми

протоколами и Telegram API. Ниже приведен код реализации инициализации класса.

```
class BotApplication:
    def __init__(self):
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(name)s - %(levelname)s -
%(message)s',
            handlers=[logging.StreamHandler()]
        )
        logger.info("Инициализация компонентов бота...")

        self.stats = StatisticsManager()
        self.feedback = FeedbackManager()
        self.payment_system = PaymentSystem(self.stats)
        self.router = MessageRouter(self.stats,
self.feedback, self.payment_system)
        self.commands = CommandManager(self.stats,
self.router, self.payment_system)

        logger.debug("Создание Telegram Application...")
        self.app = ApplicationBuilder().token(TOKEN).build()

        logger.info("Регистрация обработчиков...")
        self._register_handlers()
        logger.info("Инициализация завершена")
```

Реализация класса MessageRouter управляет распределением команд Telegram по соответствующим обработчикам. Ниже приведен код реализации.

```
class MessageRouter:
    def __init__(
        self,
        stats_manager: StatisticsManager,
        feedback_manager: FeedbackManager,
        payment_system: PaymentSystem
    ):
        self.stats = stats_manager
        self.feedback = feedback_manager
        self.payment_system = payment_system
        self.pending_feedback = set()
        self.phrase_buffer: Dict[int, List[str]] = {}
        self.logger = logging.getLogger(self.__class__.__name__)
        target_words = [
            'добрый', 'день', 'твой', 'имя', 'я', 'мой',
            'благодарность', 'голодный', 'хотеть',
            'говорить',
            'мочь', 'не', 'вы', 'хорошо', 'против'
        ]
        self.model = ModelHandler("best_model_63_3.pth",
```

```

target_words)
    self.video_processor = VideoProcessor()
    self.temp_dir = os.path.join(os.getcwd(),
"temp_videos")
    self._init_temp_directory()
    self._cleanup_old_files()
    self.logger.info("Инициализация MessageRouter
завершена")

```

4.5 Реализация блока подготовки набора данных

Реализация этого блока производит обработку сырых видео, которые пользователь отправляет в телеграм-бот. Производится загрузка файлов, их нормализация и преобразование в тензоры, готовые к подаче в нейросеть.

Реализация класса `VideoProcessor` централизует логику подготовки и нормировки данных. Ниже приведен код реализации.

```

class VideoProcessor:
    def __init__(self, frame_size=(640, 480), seq_length=16):
        self.frame_size = frame_size
        self.seq_length = seq_length
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=2,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

```

4.6 Блок набора данных

Реализация блока набора данных отвечает за обработку и хранение набора данных, предназначенных для обучения модели нейросети – централизует и унифицирует формат файлов. А также подготавливаются выходные тензоры, готовые к использованию в обучении нейросети.

Перед стартом обработки набора данных для обучения модели, следует соответствующе их подготовить. Подготовка данных зависит в первую очередь от выбранного набора. Для реализации данного проекта был выбран вышеописанный набор данных `Slovo`, являющийся наикрупнейшим набором русскоязычного жестового языка, представленный в нескольких экземплярах:

- версия «Origin» с длинными исходными видеофайлами, на которых последовательно показываются двадцать различных жестов;
- версия «Slovo» с нарезанными видеофайлами по одному жесту в каждом, аннотированные в таблице;
- версия «360p» представляющая из себя ту же версию «Slovo», но с урезанным в несколько раз качеством видео.

Для реализации был выбран стандартный вариант «Slovo», так как, в

отличии от версии с низким разрешением, он помогает добиться большей точности распознавания, хотя обучение и занимает больше времени, а нейронная сеть требует более сложную архитектуру. Пример видеофайла из набора данных Slovo представлен на рисунке 4.5.

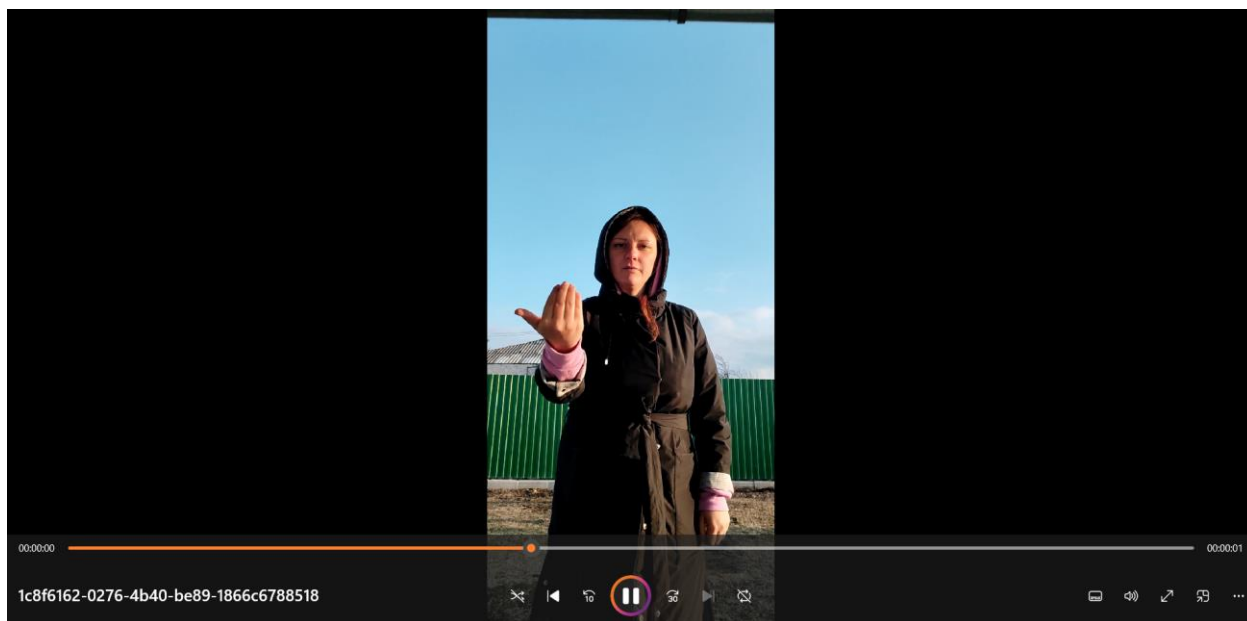


Рисунок 4.5 – Пример видеофайла из набора данных Slovo

Жесты могут воспроизводиться как одной, так и двумя руками – это важно учитывать на всех этапах предобработки видеофайлов с помощью MediaPipe. Также, следует установить минимальный порог уверенности определения рук для MediaPipe, чтобы отсечь лишние кадры, которые могут запутать модель. И последним, самым важным пунктом, требуется отобрать ограниченное конкретное число жестов в словарь, на которых будет выполнено первичное обучение и тестирование. Весь набор данных предоставляет видеофайлы для более чем тысячи жестов, для обучения модели на которых потребуются немалые мощности. В связи с этими пунктами, были составлены настройки системы и MediaPipe, описанные ниже.

```
TARGET_WORDS = ['добрый', 'день', 'твой', 'имя', 'я', 'мой',  
                'благодарность', 'голодный', 'хотеть',  
'говорить',  
                'мочь', 'не', 'вы', 'хорошо', 'против']  
VIDEOS_DIR = 'D:/temp/train/train'  
ANNOTATION_FILE = 'annotations.csv'  
OUTPUT_DIR = './processed/'  
MAX_FRAMES = 30  
# Конфигурация MediaPipe  
HAND_PARAMS = {  
    'static_image_mode': False,  
    'max_num_hands': 2,  
    'model_complexity': 1,  
    'min_detection_confidence': 0.5,
```

```

        'min_tracking_confidence': 0.5
    }

```

Реализация класса `DataSetProcessor` инкапсулирует логику считывания и представления данных для обучения и проверки модели. Ниже приведен код реализации.

```

class DataSetProcessor:
    def __init__(self):
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(**HAND_PARAMS)

```

Ключевым методом данного класса является метод `run()`, описывающий процесс обучения. Реализация метода в соответствии с правилами, описанными выше, приведена ниже.

```

def run(self):
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    total_files = len(self.filtered_df)
    with tqdm(total=total_files, desc="Обработка") as pbar:
        for row in self.filtered_df.itertuples():
            video_id, label = row[1], row[2]
            input_path = os.path.join(VIDEOS_DIR,
f"{video_id}.mp4")
            output_path = os.path.join(OUTPUT_DIR,
f"{video_id}.pt")
            if not os.path.exists(output_path):
                try:
                    if not os.path.exists(input_path):
                        print(f"\nФайл {video_id}.mp4")
                        continue
                    tensor = self.process_video(input_path)
                    torch.save(tensor, output_path)
                except Exception as e:
                    print(f"\nОшибка обработки {video_id}:
{str(e)}")
                finally:
                    pbar.update(1)

```

Данный метод оперирует в первую очередь обработкой кадров. К каждому из кадров применяется функция `MediaPipe`, накладывающая граф на руки, обнаруженные в кадре. Далее, алгоритм считывает вершины этого графа

и запоминает положение каждой из них. Эта логика реализована в методе `process_frame()`. Ниже представлен код реализации метода.

```
def process_frame(self, frame):
    results = self.hands.process(cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB))
    features = []
    if results.multi_hand_landmarks:
        for hand in results.multi_hand_landmarks[:2]:
            for landmark in hand.landmark:
                features.extend([landmark.x, landmark.y,
landmark.z])
        if len(results.multi_hand_landmarks) < 2:
            features += [0.0] * 63 * (2 -
len(results.multi_hand_landmarks))
        else:
            features = [0.0] * 126 # 2 руки * 63 координаты
    return features
```

В свою очередь, результаты этого метода аккумулирует метод `process_video()`, преобразующий данные признаки в тензор. Ниже представлен код реализации метода.

```
def process_video(self, video_path):
    cap = cv2.VideoCapture(video_path)
    frames = []
    try:
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
            frames.append(self.process_frame(frame))
            if len(frames) >= MAX_FRAMES:
                break
    finally:
        cap.release()
    if len(frames) < MAX_FRAMES:
        padding = [[0.0] * 126 for _ in range(MAX_FRAMES -
len(frames))]
        frames.extend(padding)
    return torch.tensor(frames[:MAX_FRAMES],
dtype=torch.float32)
```


5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

5.1 Методика тестирования

Цель методики тестирования: убедиться, что телеграм-бот для распознавания языка жестов на базе нейросети работает без сбоев и соответствует всем требованиям. В рамках этой методики проверяются следующие аспекты его работы:

1. Пользовательский интерфейс.

- поддержка адаптивности интерфейса под мобильные устройства, веб- и настольные версии;

- поддержка стандартных цветовых тем Telegram.

2. Основной функционал распознавания.

- точность и скорость распознавания жестов: соответствует ли расшифровка видео тому, что показывает пользователь.

3. Вспомогательные возможности.

- работа со статистикой;

- возможность оставить отзыв или предложение;

- вызов меню инструкции для ознакомления работы с ботом.

4. Устойчивость к ошибкам.

- корректная обработка обрыва соединения и повторных запросов;

- обработка ошибок на стороне клиента и защита от них пользователя.

Благодаря комплексному подходу можно удостовериться в правильности работы бота, а также выявить слабые места, повысить надежность системы и обеспечить пользователю лучший опыт.

Серверное окружение:

- ОС: Windows 10;

- CPU: AMD Ryzen 5 5500U (6 ядер);

- GPU: AMD Radeon Graphics (512 МБ);

- фреймворки: PyTorch 2.0, python-telegram-bot 22.0.

Клиентское окружение:

- Telegram: Telegram Desktop 5.14.2;

- разрешение экрана: 1920x1080.

Для обеспечения стабильной работы приложения тестирование проводилось на различных устройствах, различных комбинациях жестов, а также при различном освещении, чтобы оценить качество расшифровки текста.

5.2 Тестирование графического интерфейса

В данном разделе описывается методика испытания различных аспектов полученного программного средства. Рассмотрим тестирование графического

интерфейса для убеждения, что предоставляемая Telegram платформа не вызывает конфликтов в интеграции с готовым приложением. Проверяются следующие графические возможности итогового приложения:

- поддержка адаптивности интерфейса под мобильные устройства, веб- и настольные версии;
- поддержка стандартных цветовых тем Telegram.

5.2.1 Тестирование адаптивности интерфейса

Проверка адаптивности интерфейса была осуществлялась на нескольких устройствах с различными размерами дисплея. На рисунке 5.1 приведен пример тестирования на мобильном устройстве Google Pixel 8 Pro.

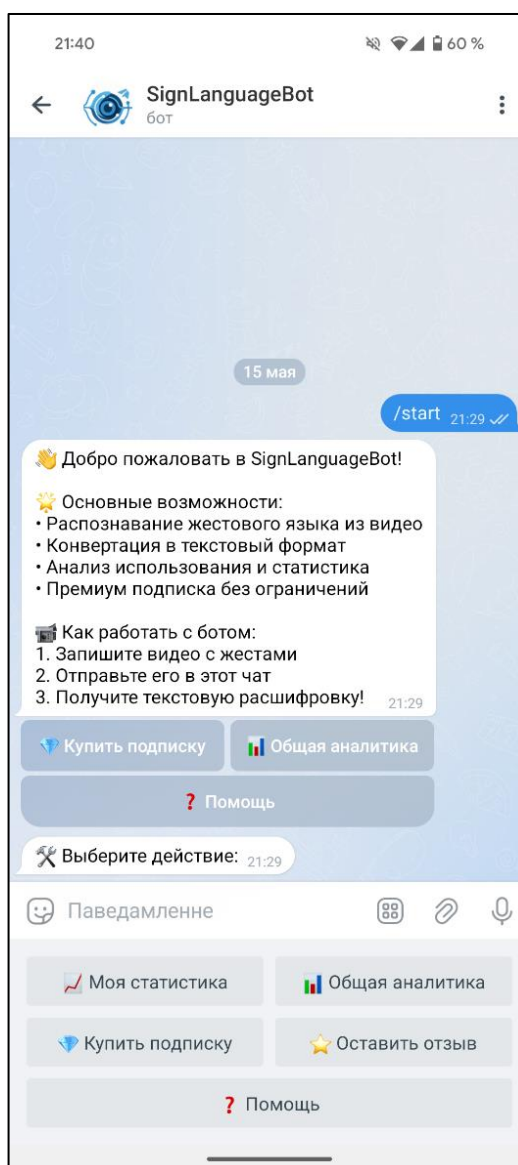


Рисунок 5.1 – Пример тестирования бота на мобильной версии Telegram

Аналогичный тест был проведен для настольной версии. Пример тестирования на версии Telegram для операционной системы Windows 10 приведен на рисунке 5.2.

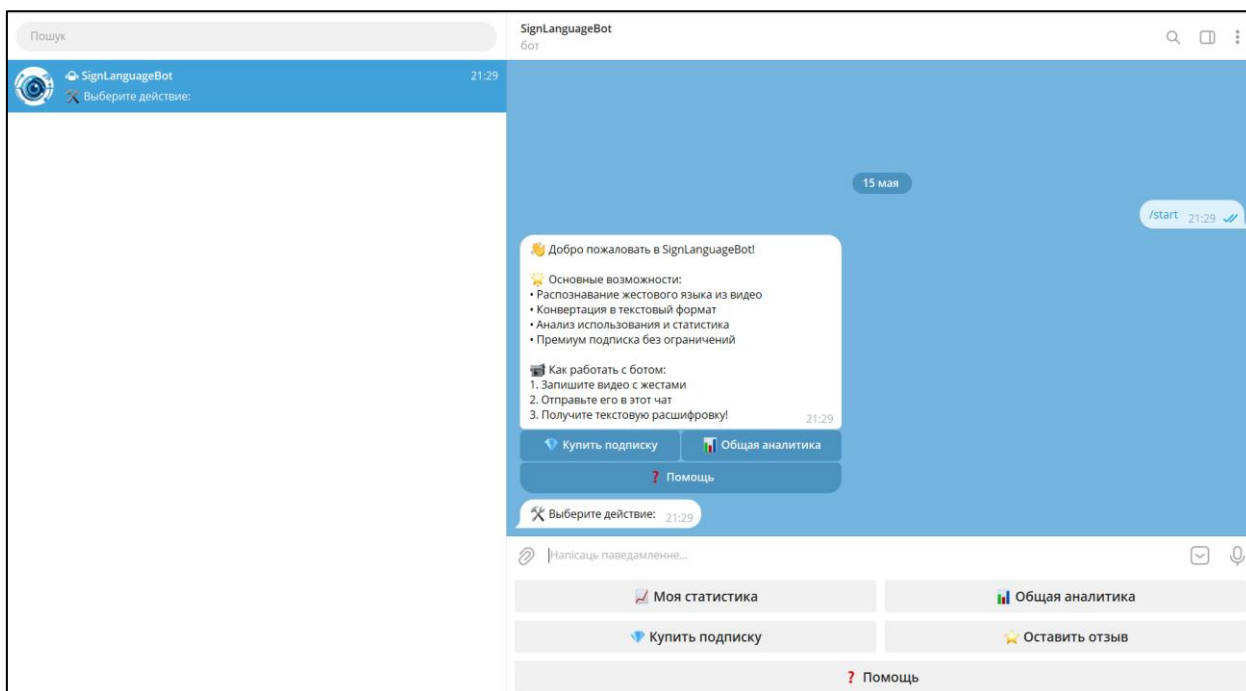


Рисунок 5.2 – Пример тестирования бота на версии для персонального компьютера

Последним тестом из данной категории является тестирование реакции интерфейса на изменение масштаба рабочего окна на web и настольной версий приложения Telegram. При разработке, для большего удобства, были добавлены прикрепленные к панели чата кнопки. Соответственно, требуется проверить, как изменение масштаба рабочего окна отражается на их положении. Пример тестирования приведен на рисунке 5.3.

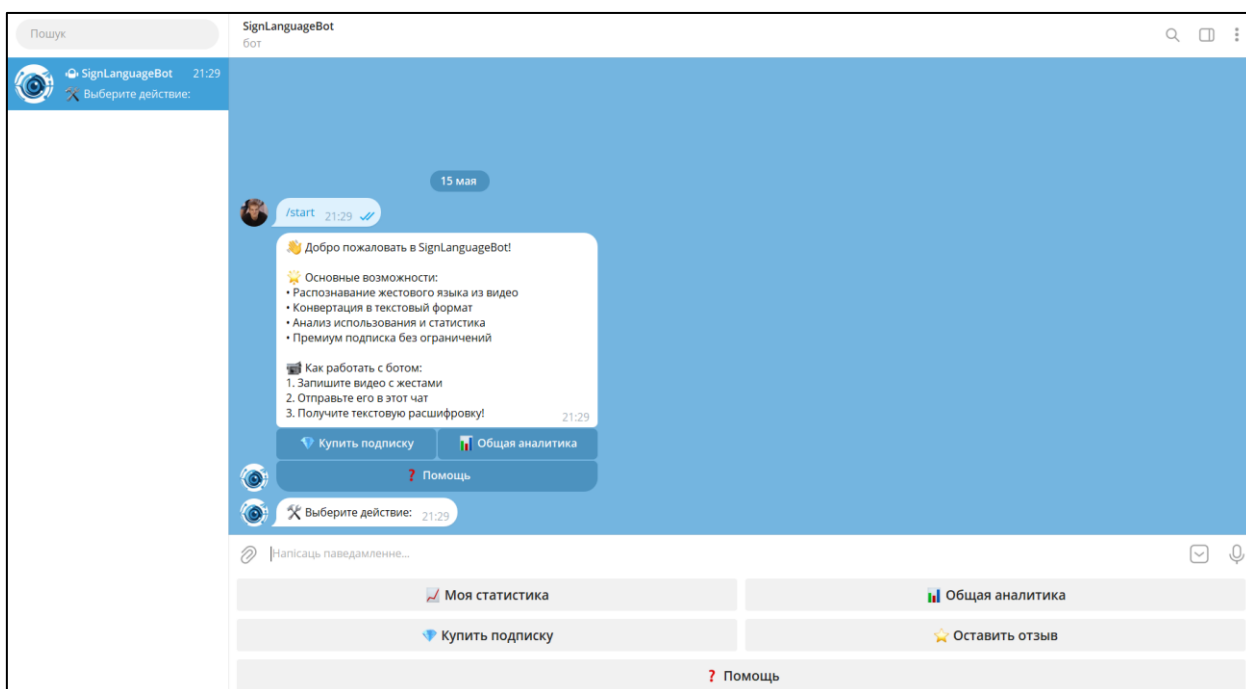


Рисунок 5.3 – Пример изменения масштаба рабочего окна

Из рисунка выше видно, что кнопки изменяют свой размер в соответствии с общим графическим интерфейсом, а значит, тест можно считать пройденным.

5.2.2 Тестирование изменения цветовой палитры приложения

Стандартный функционал Telegram предоставляет широкую цветовую палитру, сочетающую изменение не только шрифтов и общей цветовой темы, но и отдельных ее элементов. Данные темы также могут влиять на элементы чата с ботом, такие как кнопки или отдельные части интерфейса. Доступные цветовые сочетания можно увидеть на рисунке 5.4.

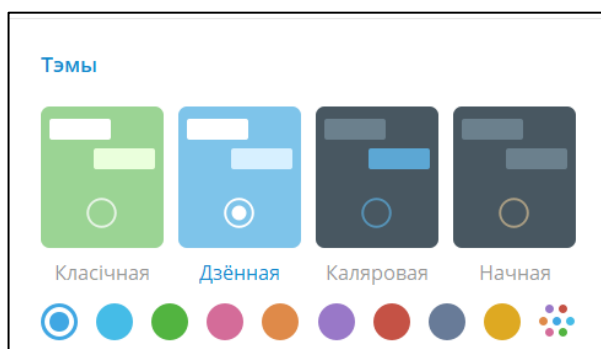


Рисунок 5.4 – Доступные цветовые палитры Telegram

В связи с этим, был проведен тест, проверяющий читабельность и удобство использования бота в светлой и темной цветовых палитрах. Результаты можно увидеть на рисунке 5.5.

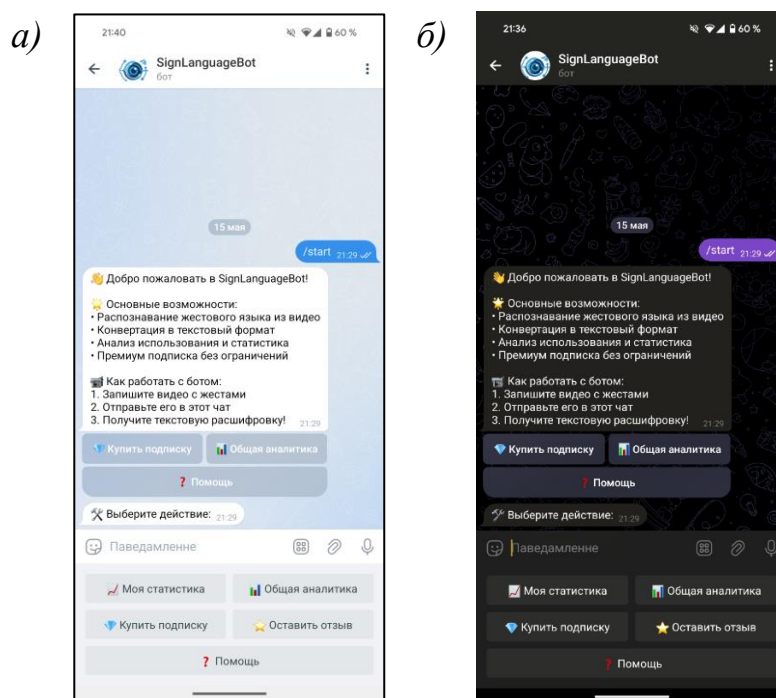


Рисунок 5.5 – Тестирование различных цветовых палитр: *а* – светлая тема; *б* – темная тема

5.3 Тестирование функциональности приложения

5.3.1 Тестирование основного функционала

Основным функционалом приложения является расшифровка видео-сообщений и видеофайлов, содержащих язык жестов, в текстовый формат. Соответственно, требуется удостовериться, что каждый из жестов распознается правильно, ошибки сведены к минимуму, а текст сохраняет передаваемый смысл. Также, следует проводить тестирование при различном уровне освещенности, чтобы убедиться, что нейросеть одинаково уверенно справляется со своей задачей в самых разных сценариях.

Проверим телеграм-бота на видео-сообщениях. Зашифрованным текстом будет являться текст «Добрый день!». Текст шифруется двумя последовательными жестами. Также стоит учитывать специфику распознавания и расшифровки – бот распознает жесты по одному, без учета последовательности. То есть, он не может самостоятельно согласовывать слова в предложении. Результат ответа на видео-сообщение представлен на рисунке 5.6.

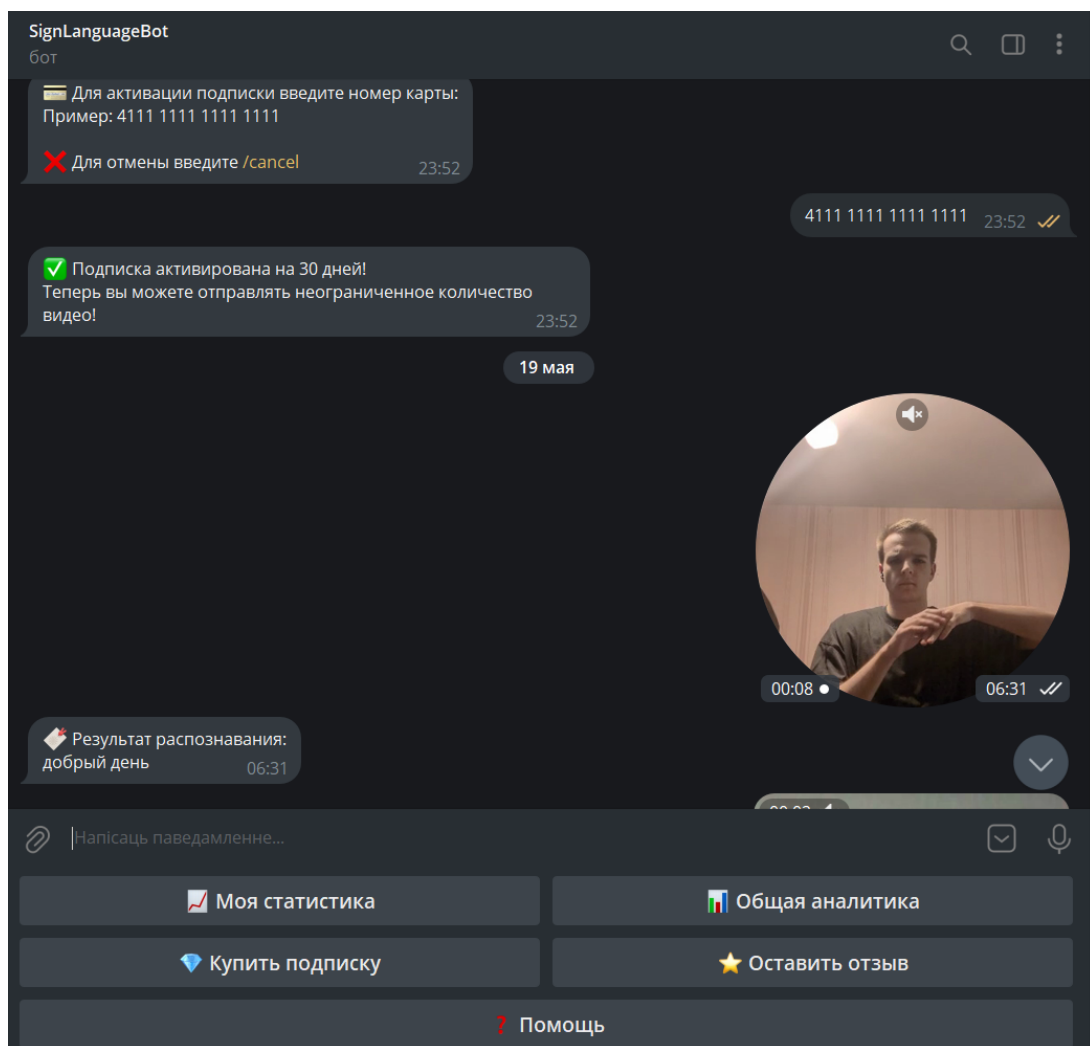


Рисунок 5.6 – Тестирование работы полученной нейронной сети на видео-сообщении

Как видно из изображения выше, ответ практически полностью совпал со сформулированной жестовой фразой. Это и обосновано отсутствием согласования слов в предложении.

Теперь аналогичную проверку следует провести над видеофайлом. Учитывая весьма схожую специфику со стороны пользователя, Telegram же помечает их как различные объекты, что вынуждает обрабатывать видеофайлы в отдельности. Результат ответа на видеофайл представлен на рисунке 5.7.

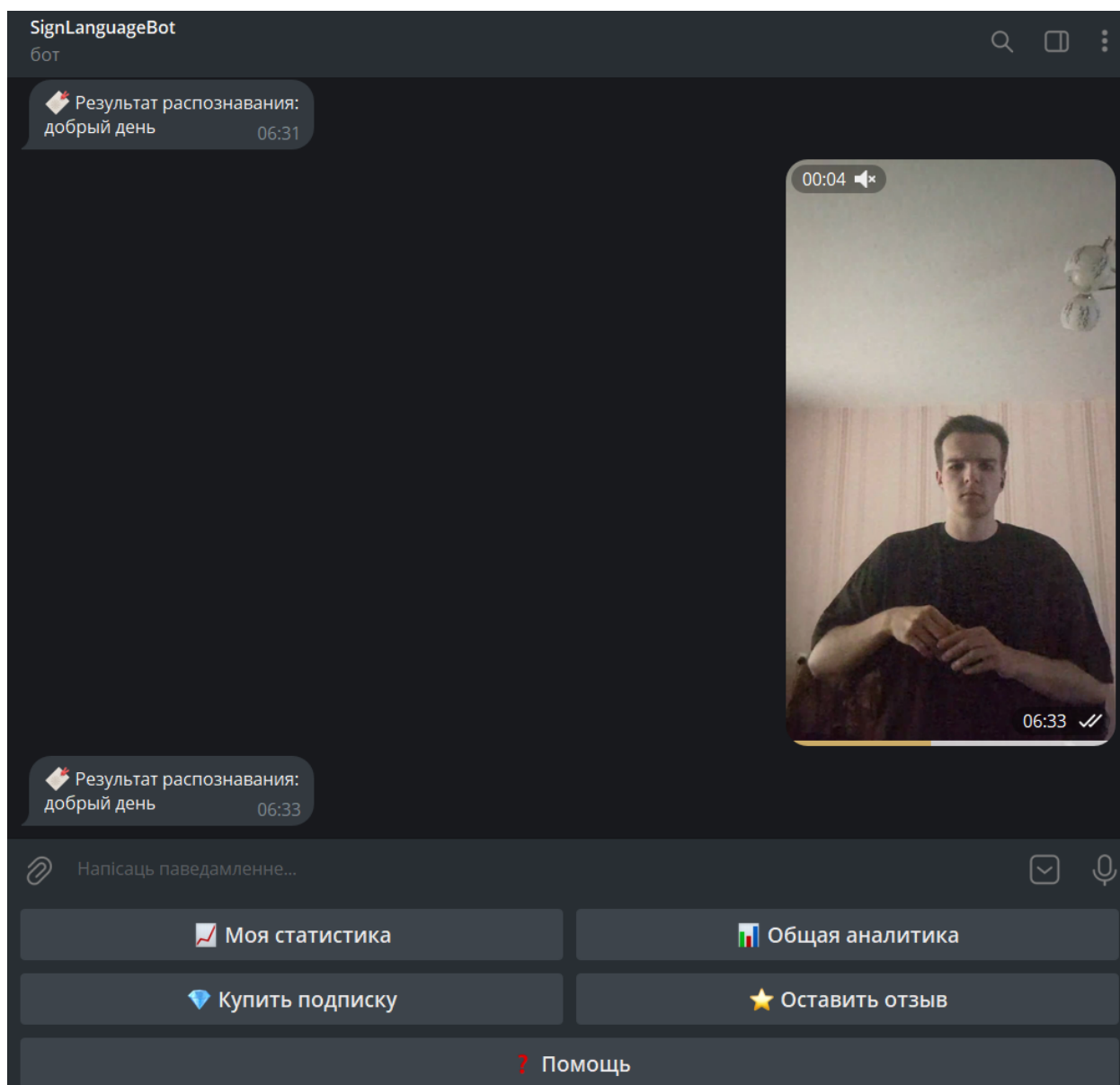


Рисунок 5.7 – Тестирование работы полученной нейронной сети на видеофайле

Из изображения выше, видно, что результат идентичен первому, что означает, что нейросеть работает корректно.

Также второй видеофайл намеренно записан в приглушенном свете. Как видно из результата, это никак не сказалось на обработке, что может заверить

в том, что нейронная сеть устойчива к небольшому уровню помех при обработке.

К основным функциям также можно отнести реализацию подписочной модели и оплату. Учитывая, что данный дипломный проект является учебным, а также временную недоступность в Telegram ключевой партнерской сети проведения платежей для восточного региона, был выбран тестовый режим проведения оплаты в боте. Бот принимает тестовый номер карты и эмулирует проведение оплаты подписки. Данная функция может быть отключена при разворачивании приложения на сервере со снятыми региональными ограничениями.

Также у каждого пользователя, с бесплатной моделью подписки, присутствует ограничение на число расшифровок в день. Такой пользователь может отправить только пять видео. Для проверки корректного поведения, проведем несколько тестов. Для начала, от имени пользователя без подписки отправим пять видеофайлов на расшифровку, количество оставшихся расшифровок в день можно увидеть в личной статистике. Попытка отправить видеофайл при достижении лимита продемонстрирована на рисунке 5.8.

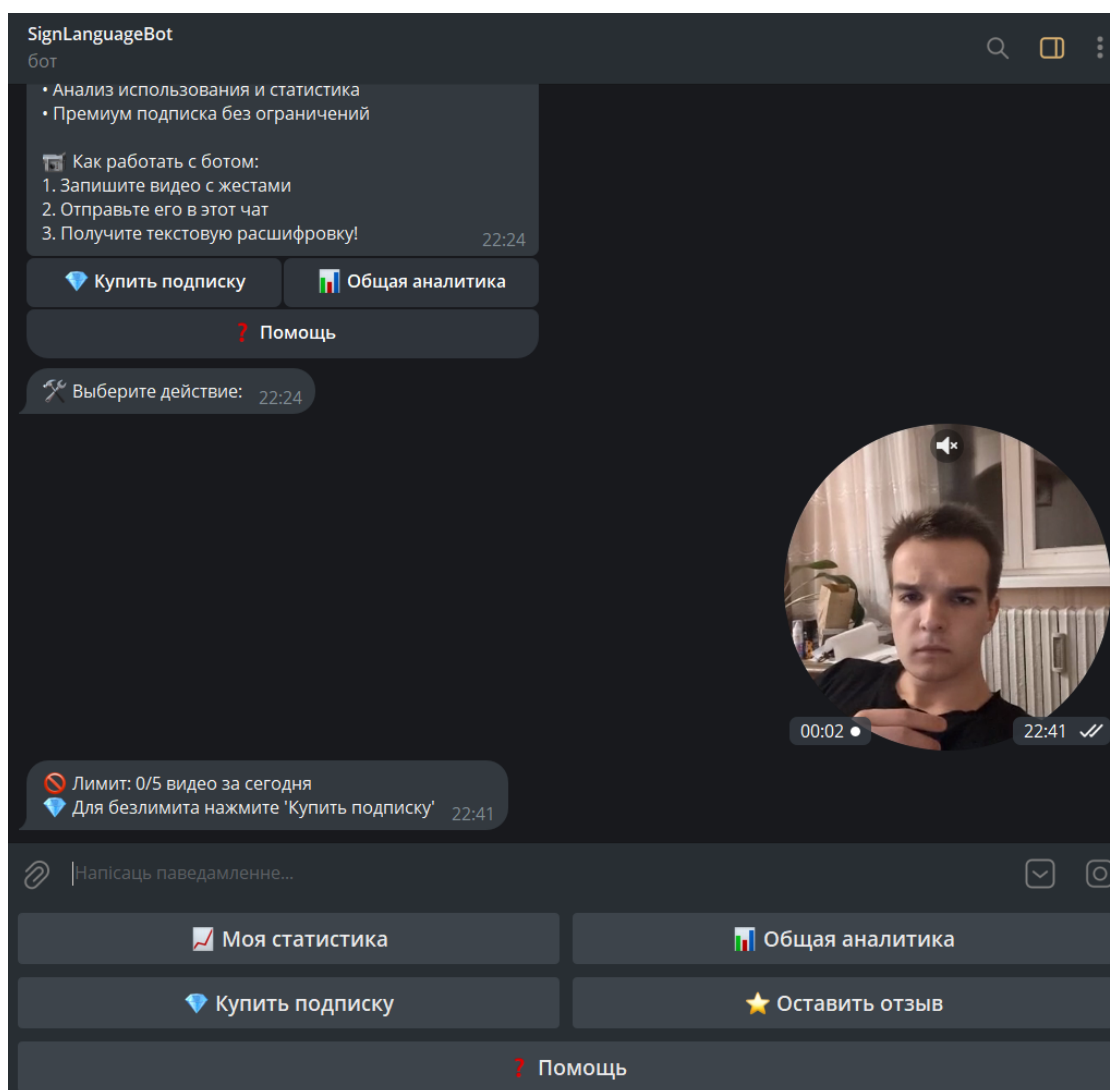


Рисунок 5.8 – Попытка отправить видеофайл при достижении лимита

Теперь оплатим подписку с помощью номера тестовой банковской карты и проверим доступность обработки. Результаты теста на рисунке 5.9.

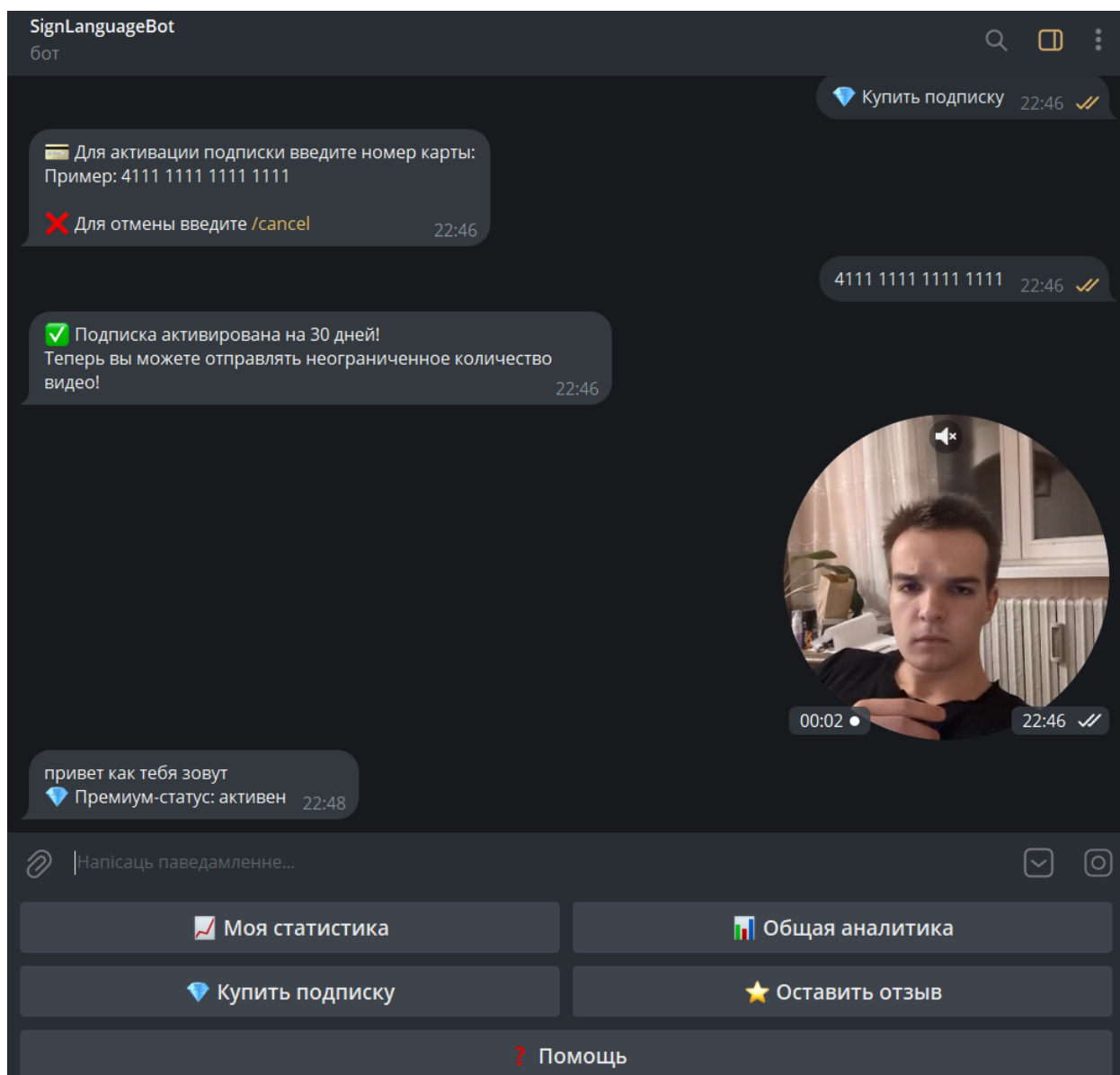


Рисунок 5.9 – Успешная обработка видео после оплаты подписки

5.3.2 Тестирование второстепенного функционала

Второстепенным функционалом приложения можно считать все побочные функции, кроме расшифровки видеофайлов и проведения оплаты подписки. Наличие и тестирование этих функций придает полноты функционала боту, а также закладывает основание для будущих улучшений.

Тестирование процесса работы бота начинается с главного экрана: отсюда пользователь получает первый визуальный контакт с интерфейсом и базовыми элементами управления. При нажатии на кнопку запуска бота системой автоматически должен инициироваться ряд заранее определённых функций: вывод приветственного сообщения, кнопок управления ботом, и переход бота в состояние ожидания команды. Результат теста на рисунке 5.10.

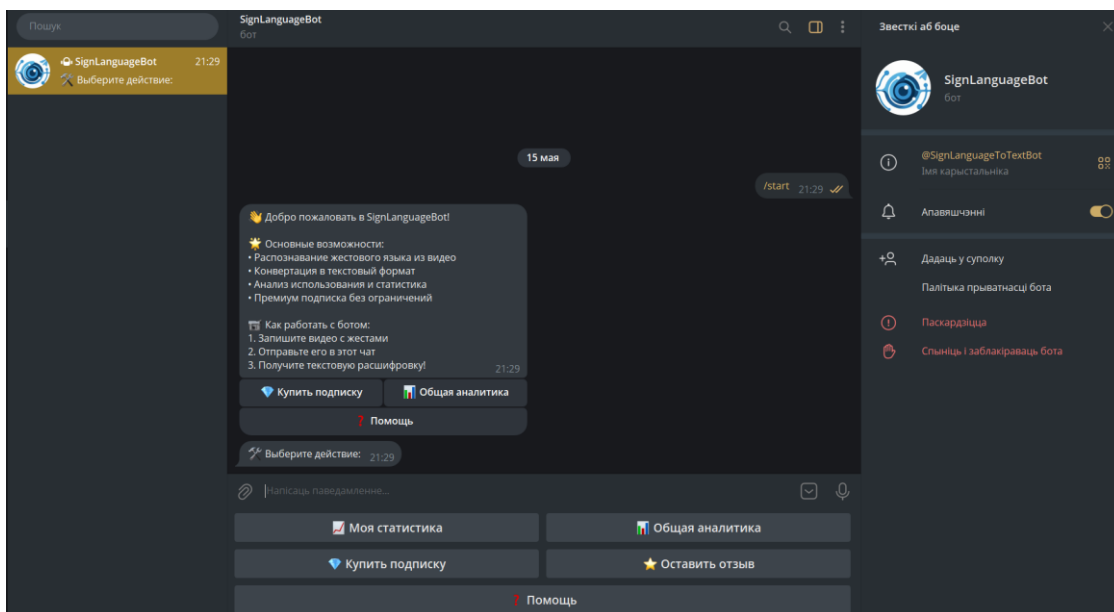


Рисунок 5.10 – Тестирование корректной инициализации телеграм-бота

Далее, проверим возможности просмотра индивидуальной статистики пользователя. При нажатии на кнопку «Моя статистика», ожидается вывод индивидуальных результатов взаимодействия пользователя с ботом. Бот должен подсчитать число обращений конкретного пользователя, а также запомнить и сохранить это значение. И, в случае временных перебоев на стороне сервера, после его перезапуска значение не должно сбрасываться. Результат сбора статистики до перезапуска показан на рисунке 5.11.

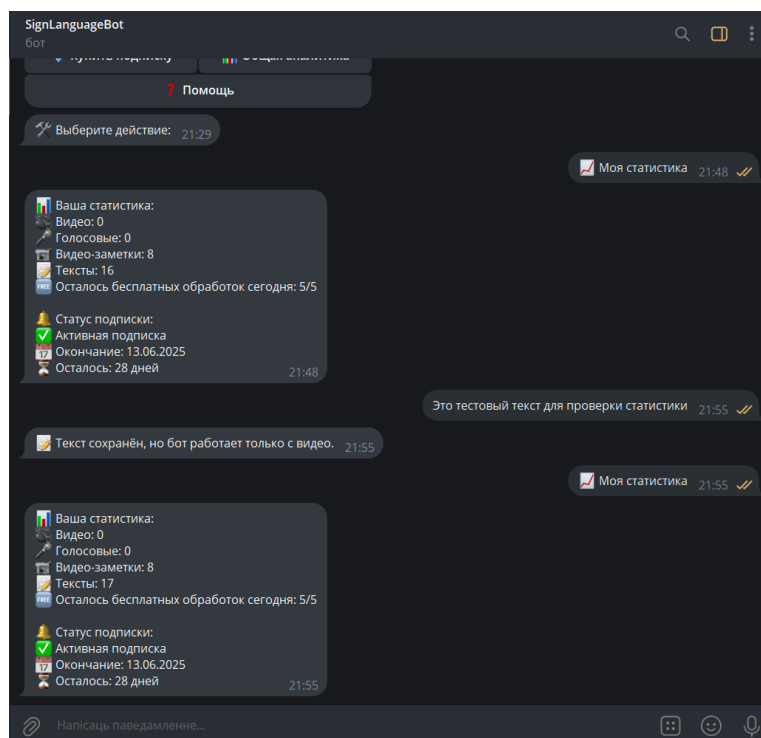


Рисунок 5.11 – Тестирование корректной работы сбора индивидуальной статистики до перезапуска сервера

Теперь перезапустим сервер и снова вернемся в бот, где повторим вызов собранной статистики пользователя. Результат сбора статистики после перезапуска показан на рисунке 5.12.

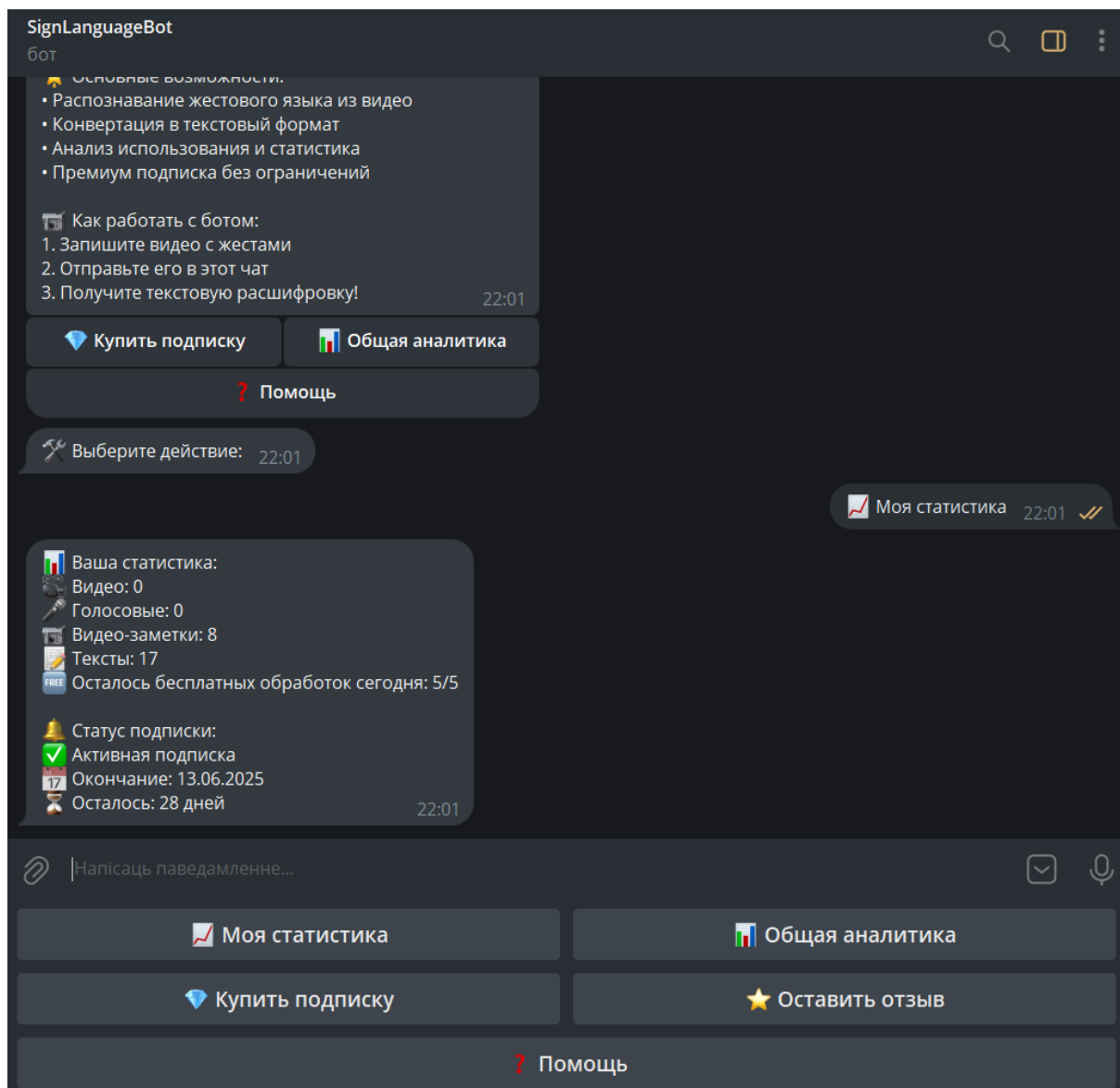


Рисунок 5.12 – Тестирование корректной работы сбора индивидуальной статистики после перезапуска сервера

Еще одной функцией является сбор аналитики по использованию бота всеми пользователями. При вызове данной команды ожидается получение списка наиболее активного числа пользователей, мера их активности, а также самые активные часы использования бота среди пользователей. Для сохранения относительной безопасности аккаунтов пользователей, используются не ссылки на их профиль, а уникальные идентификаторы, скрытые от стороннего пользователя, но доступные непосредственному владельцу аккаунта. Чтобы проверить, что не происходит утечка данных пользователей через телеграм-бота, вызовем команду «Общая аналитика». Результаты теста на рисунке 5.13.

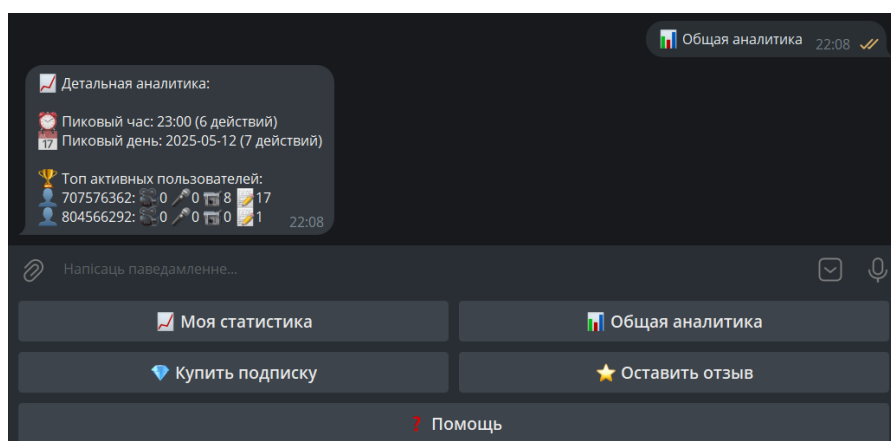


Рисунок 5.13 – Тестирование корректной работы сбора аналитики использования бота

Из результатов видно, что данные пользователей обработались корректно и они остаются анонимны для окружающих, но могут определить себя.

Также в боте доступна функция обратной связи. При вызове функции «Оставить отзыв» ожидается, что пользователь напишет свой комментарий с оценкой от одной до пяти звезд. Результаты тестирования функции на рисунках 5.14.

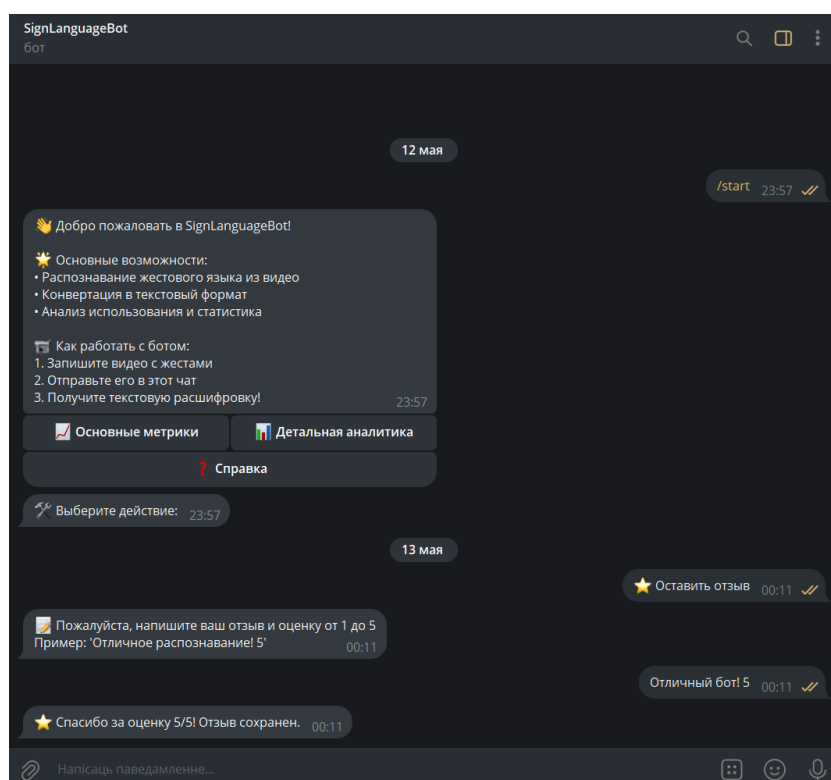


Рисунок 5.14 – Тестирование функции обратной связи телеграм-бота

Также предусмотрен случай, когда пользователь оставляет оценку, не удовлетворяющую запросу. В таком случае ожидается вывод сообщения о

сохранении отзыва без оценки. Результат тестирования случая приведен на рисунке 5.15.

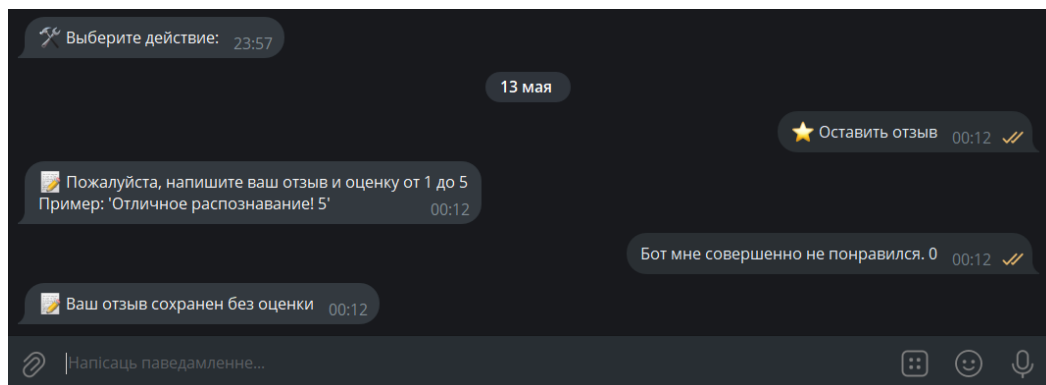


Рисунок 5.15 – Тестирование функции обратной связи с неверной оценкой

Такие отзывы будут отображаться как отзывы без оценки. Проверить, как эти отзывы отображаются, можно с помощью кнопки «Посмотреть отзывы» или введя команду `/reviews`. Результат вывода команды на рисунке 5.16.

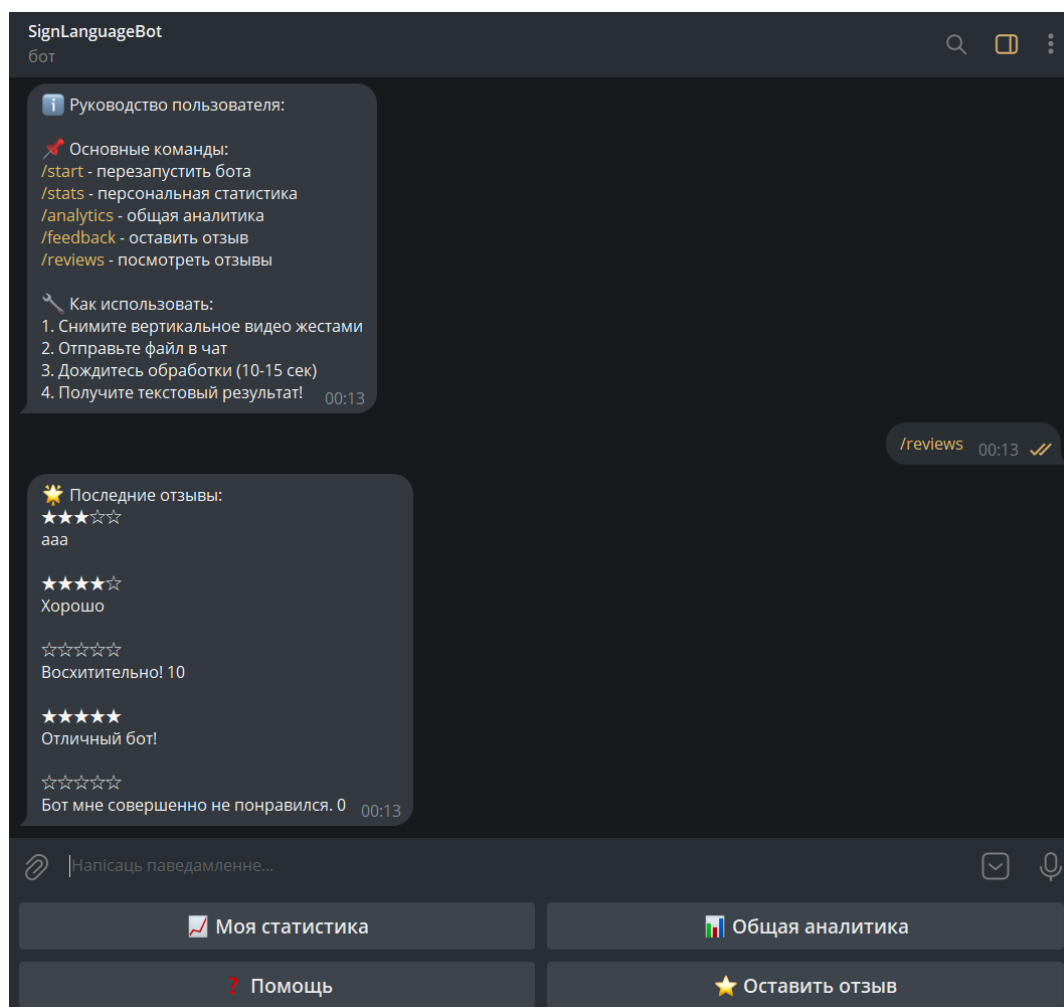


Рисунок 5.16 – Тестирование функции просмотра отзывов

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

6.1 Инструкция по запуску проекта

При запуске проекта в первую очередь требуется подобрать соответствующее аппаратное обеспечение. Серверное обеспечение должно иметь достаточные технические характеристики для того, чтобы обеспечить стабильное интернет-соединение, высокую скорость и качество обработки видеофайлов с помощью модели нейронной сети, а также удовлетворительный объем оперативной и постоянной памяти для хранения видеофайлов в процессе обработки данных пользователей.

Минимальные технические характеристики серверного оборудования, следующие:

- CPU: Ryzen 5 5500, 6 ядер и 12 потоков, или аналог;
- GPU: AMD Radeon Graphics с 4 ГБ видеопамяти или аналог;
- оперативная память: 8 ГБ;
- твердотельный накопитель: 20 ГБ;
- сетевой интерфейс: пропускная способность не менее 20 Мбит/с;
- отказоустойчивость оборудования: минимальная, с возможностью перезапуска в случае сбоев;
- операционная система: Windows 10/Linux;
- Python: версия Python 3.12 и новее;
- Telegram: Telegram Desktop версии 5.14.2 и новее, либо Telegram Mobile версии 11.11.1 и новее, либо Telegram Web версии 2.1.0 или новее.

Минимальные требования к обеспечению клиента:

- Telegram: Telegram Desktop версии 5.14.2, или новее, либо Telegram Mobile версии 11.11.1, или новее, либо Telegram Web версии 2.1.0, или новее;
- сетевой интерфейс: пропускная способность не менее 20 Мбит/с.

Дальнейшим шагом, в первую очередь, требуется создать и сконфигурировать телеграм-бота.

Технически, телеграм-бот делится на две половины – серверную и клиентскую, в качестве которой выступает Telegram Bot API. Если сервер может быть развернут на любой подходящей машине, то для инициализации Telegram Bot API потребуется дополнительно авторизоваться в сервисе управления ботами Telegram @BotFather.

Для начала открываем Telegram в любом из доступных форматов и переходим в поиск контактов, в котором ищем бота под именем @BotFather. @BotFather представляет собой специализированный инструмент, разработанный командой Telegram, призванный для централизованного управления всеми вашими ботами в мессенджере. С его помощью можно получить единый интерфейс для создания и настройки ботов. Так как Telegram – открытая площадка, следует внимательно относиться к используемым каналам. Нужный бот помечен символом «галочки», что означает, что он безопасен к использованию. Поиск сервиса приведен на рисунке 6.1.

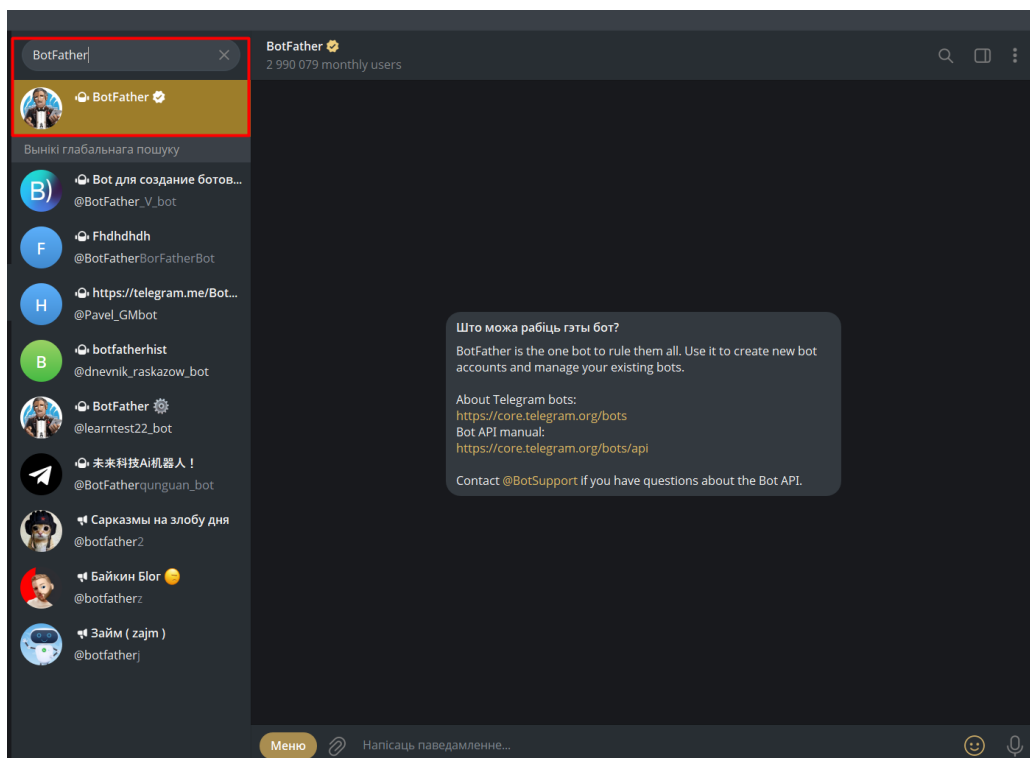


Рисунок 6.1 – Поиск сервиса по созданию телеграм-ботов

Чтобы завести нового бота, открыв диалог с @BotFather нужно ввести команду `/newbot`. После выполнения этой инструкции сервис запустит последовательность шагов, в результате которых будет создан новый бот. Следующие инструкции на рисунке 6.2.

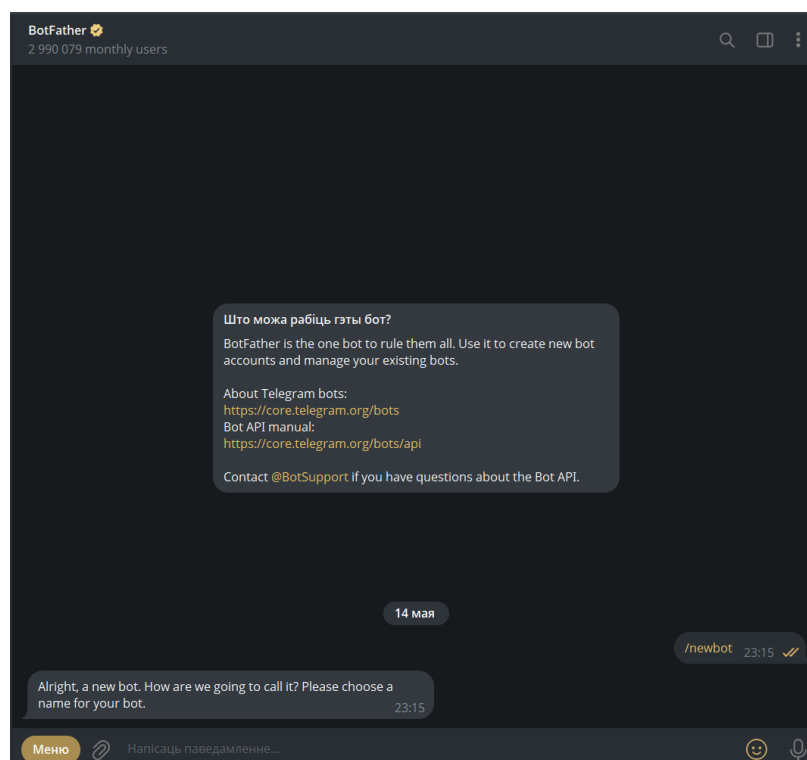


Рисунок 6.2 – Выполнение запроса на создание нового бота

В ответ, сервис запрашивает публичное имя для нового бота. Это имя будет использоваться для поиска бота и будет являться первым, наравне с фото-аватаром, что увидит пользователь. Имя может быть любым, главное – чтобы оно уже не было занято другим пользователем или ботом.

После выбора публичного имени, сервис запрашивает имя уникального идентификатора. Этот идентификатор является отличительной чертой каждой сущности в Telegram.

Также уникальным условием для ботов является то, что идентификатор бота должен заканчиваться на `bot` в любом из вариантов регистра.

Результаты взаимодействия с `@BotFather` на рисунке 6.3.

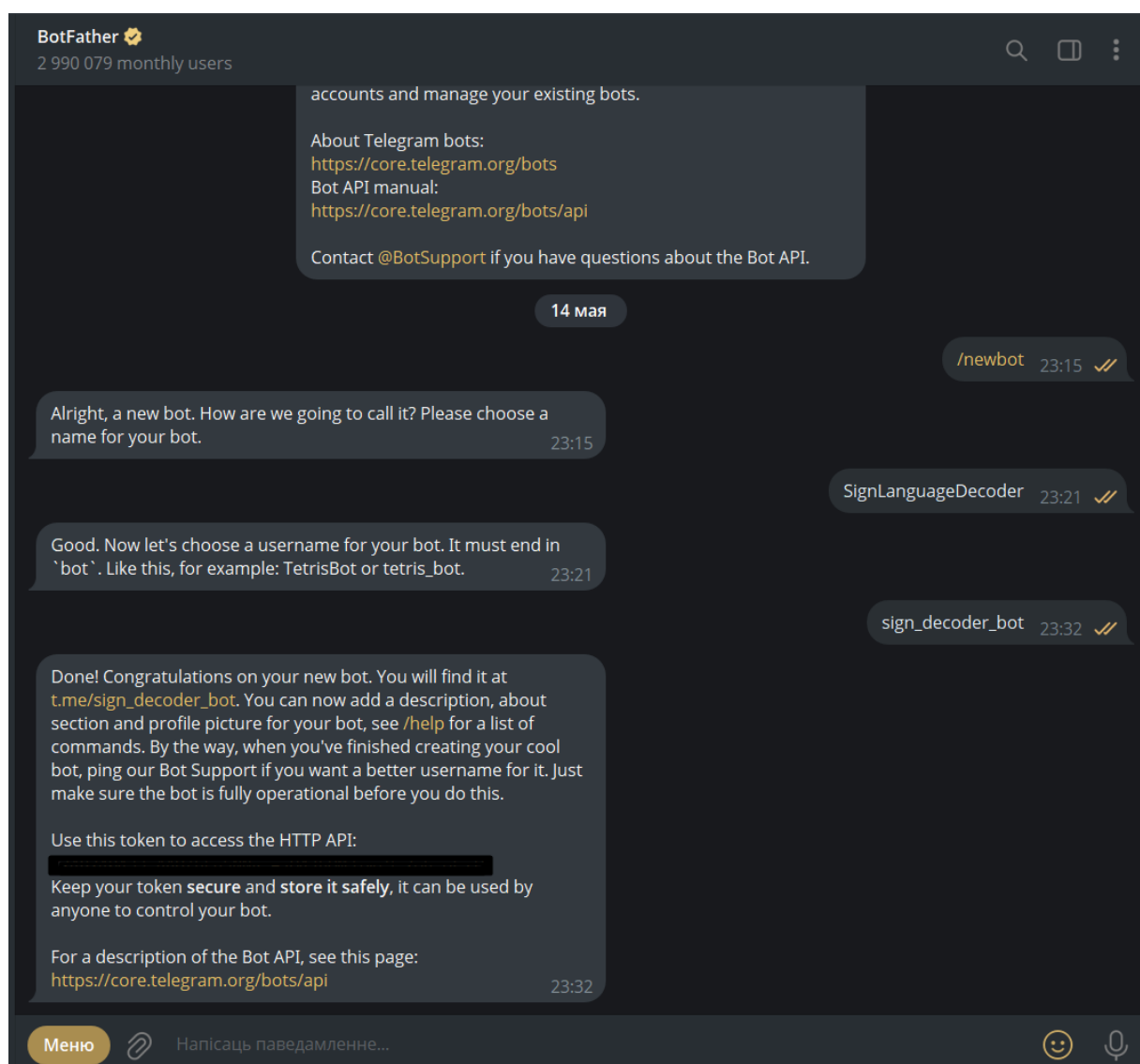


Рисунок 6.3 – Завершение создание телеграм-бота в сервисе `@BotFather`

Результатом регистрации телеграм-бота является получение Telegram API ключа. Этот ключ используется при разработке и служит уникальным определителем бота для Telegram API. Данный ключ является конфиденциальной информацией, так как при его наличии, сторонний пользователь может получить доступ к управлению вашим ботом, в том числе

и к его финансовой части. Поэтому, данный ключ строго рекомендуется не хранить в публичных репозиториях или где-либо в интернете.

Также, после получения API-ключа, требуется выполнить ряд следующих команд:

```
/mybots  
@sign_decoder_bot  
/EditBot
```

Данный набор команд предоставит доступ к базовым настройкам своего телеграм-бота. Из данного меню можно изменить имя, описание, настроить базовые кнопки и команды, а также добавить фото-аватар для бота. Меню изменения настроек телеграм-бота представлена на рисунке 6.4.

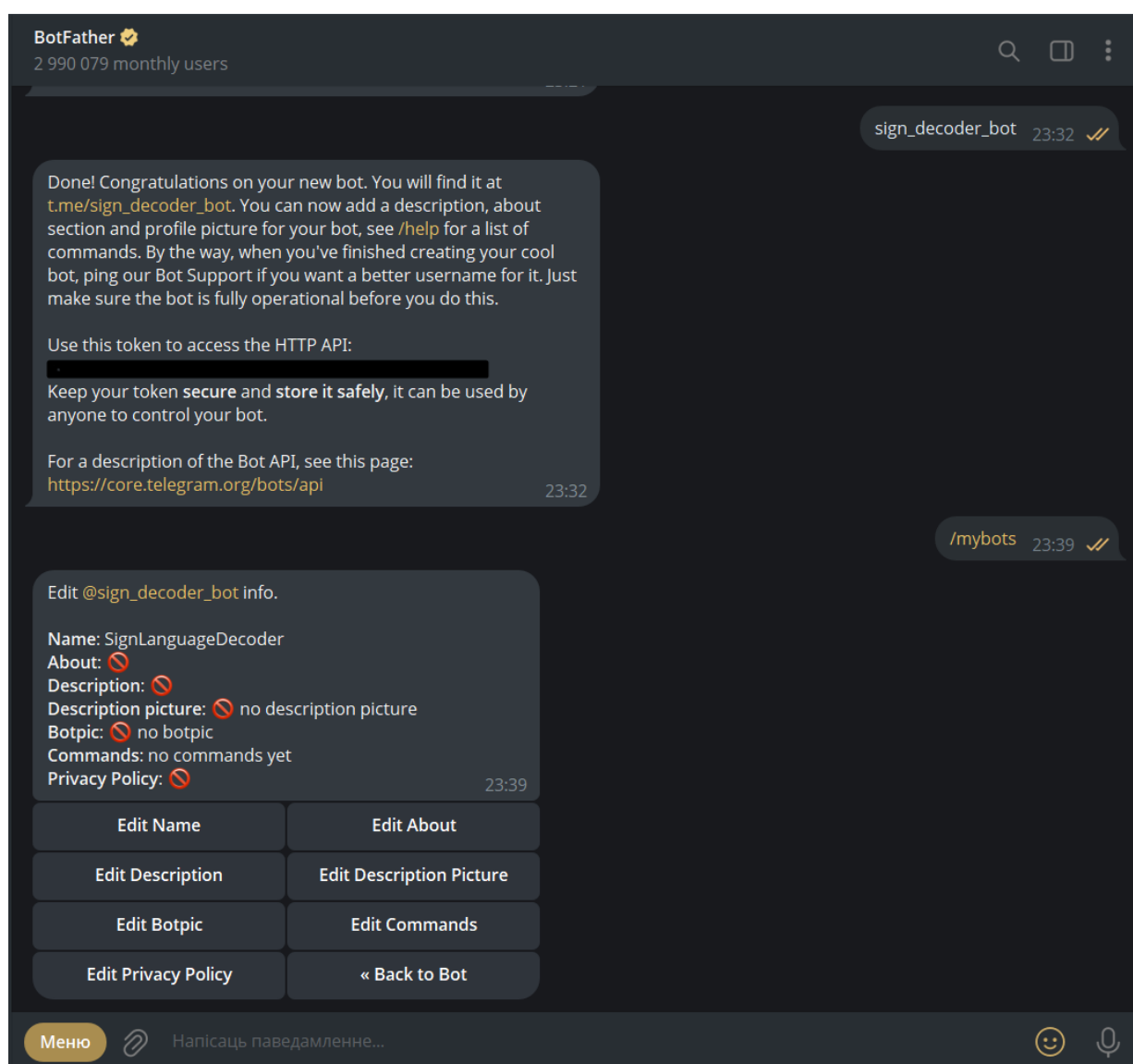


Рисунок 6.4 – Меню настроек бота из сервиса @BotFather

Несмотря на то, что из данного меню возможно выполнять настройку логики телеграм-бота, делать этого не рекомендуется, так как функционал

настройки бизнес-логики через интерфейс сервиса @BotFather значительно урезан, по сравнению с API. Тем не менее, это единственный способ настройки внешнего вида телеграм-бота. Демонстрация настройки фото-аватара бота представлена на рисунке 6.5.

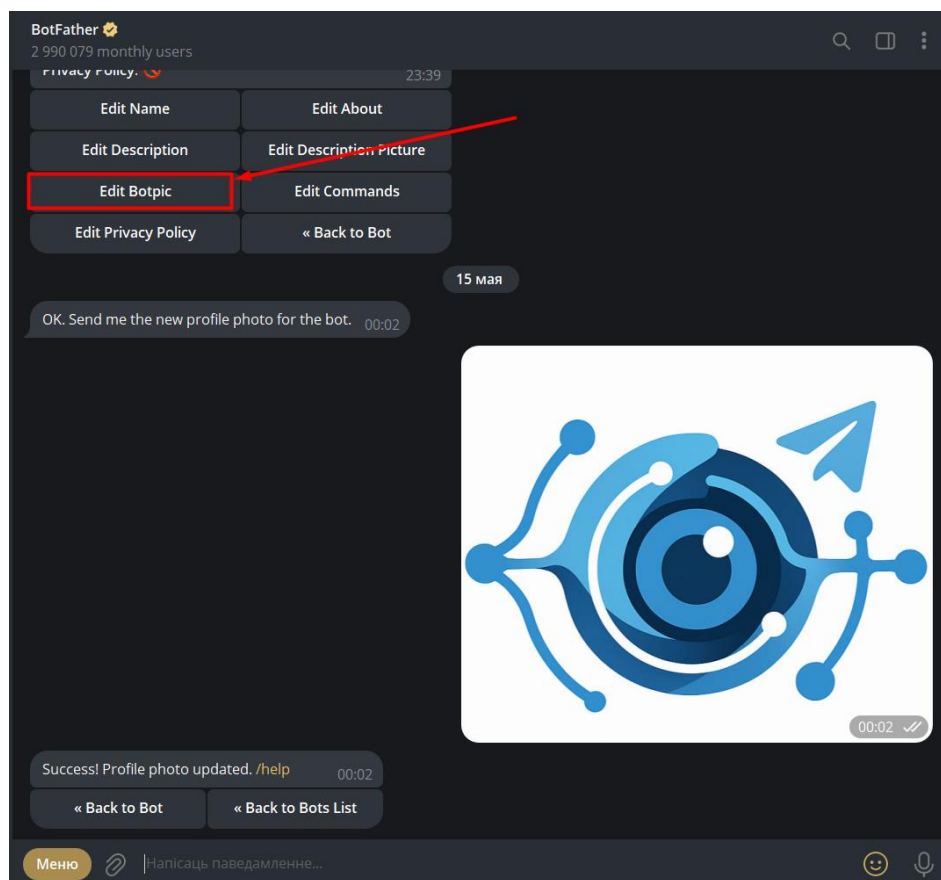


Рисунок 6.5 – Настройка фото-аватара в сервисе @BotFather

Теперь можно проверить результаты настройки, выполнив команды:

```
/mybots  
@sign_decoder_bot
```

Результаты видны на рисунке 6.6.



Рисунок 6.6 – Бот с установленным фото-аватаром

После настройки бота и получения Telegram API ключа, требуется настроить окружение на сервере и успешно его запустить. Для этого требуется установить все зависимости, создав виртуальное окружение. Чтобы создать виртуальное окружение, требуется выполнить следующие команды:

```
python -m venv venv
source venv/bin/activate    #Для Linux
venv\Scripts\activate      #Для Windows
```

После этого требуется установить все Python-зависимости, требуемые для сборки и запуска проекта. Все требуемые библиотеки и другие зависимости описаны в файле `requirements.txt`. Установить эти пакеты можно с помощью следующей команды:

```
pip install -r requirements.txt
```

Теперь требуется запустить сервер с помощью следующей команды:

```
python ./bot_app.py
```

Перед запуском убедитесь, что в корне, рядом с файлом `bot_app.py`, находится файл с расширением `.pth` – это бинарный файл с обученной моделью, без которой приложение не сможет запуститься.

Также вместе архивом с кодом для сервера, идет еще и второй архив – `ModelTrainer.rar`. В данном файле содержатся исходные коды для предобработки набора данных с помощью `MediaPipe`, а также код для обучения модели. Эти данные поставляются на случай, если пользователь захочет самостоятельно переобучить модель на собственном наборе данных. Однако, при изменении как-либо кода обучения или предобработки, нужно помнить, что соответствующее архитектурное изменение должно быть внесено в том числе и в классы, работающие с моделью и обработкой видео непосредственно в рамках бота.

После этого бот доступен и будет работать в соответствии с установленной логикой напрямую из Telegram. Чтобы получить доступ к боту, достаточно перейти по уникальному идентификатору, который был указан при регистрации в `@BotFather`.

6.2 Руководство пользователя

Для использования бота после установки Telegram, в первую очередь, следует перейти по его уникальному имени-идентификатору через поиск, которое было установлено в `@BotFather` ранее, и выполнить команду `/start`. Увидев приветственное сообщение, можно выполнить команду `/help` или нажать на кнопку «Помощь», чтобы получить инструкцию и список всех доступных команд. Ответ бота видно на рисунке 6.7.

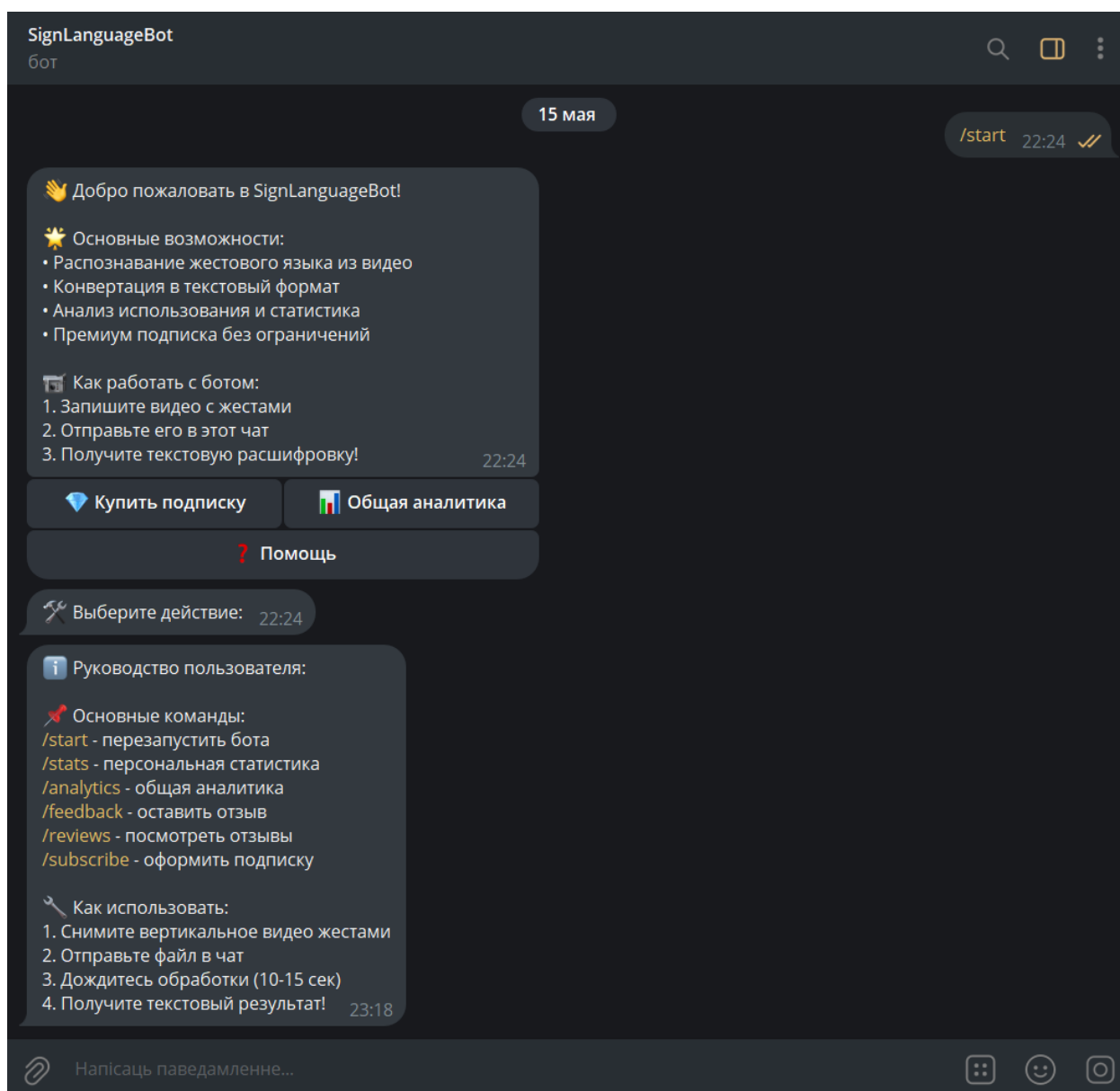


Рисунок 6.7 – Вызов команды «Помощь»

Для управления телеграм-ботом доступны следующие команды:

- /start – команда для перезапуска бота;
- /stats – команда для получения персональной статистики;
- /analytics – команда для получения общей аналитики;
- /feedback – команда для обратной связи;
- /reviews – команда для просмотра отзывов;
- /help – команда для получения справки;
- /subscribe – команда для оформления подписки.

Команда /stats показывает уникальную статистику пользователя, а именно: число отправленных текстовых, голосовых и видео-сообщений, число отправленных видеофайлов, а также число оставшихся бесплатных обработок, статус подписки и информация об ее длительности. Пример вывода команды /stats на рисунке 6.8.

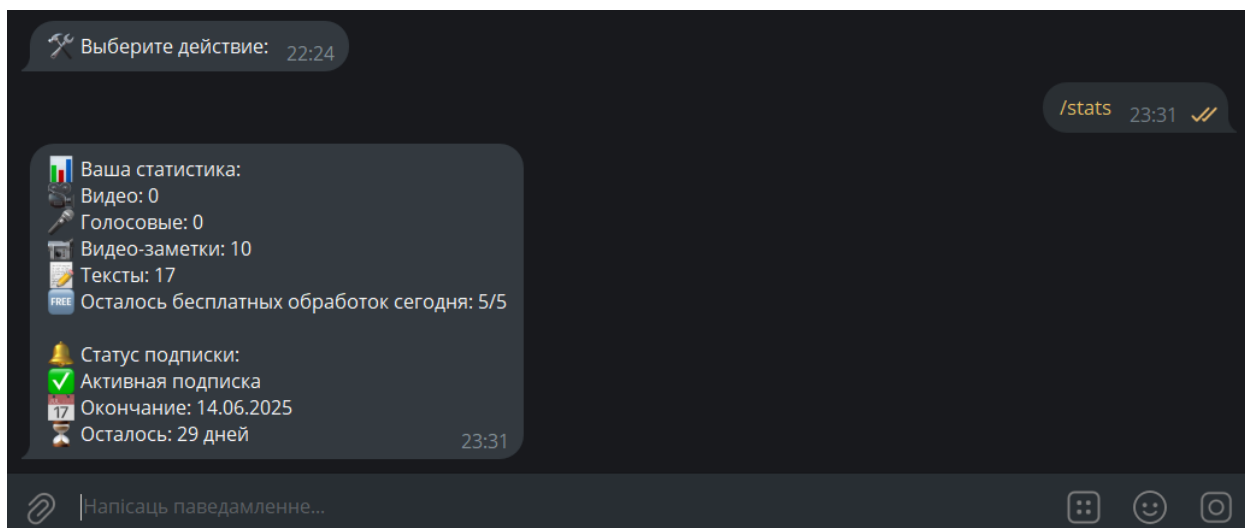


Рисунок 6.8 – Пример вывода команды `/stats`

Команда `/analytics` показывает аналитику использований бота всеми пользователями. Эта аналитика включает самых активных пользователей, их личную статистику, а также часы и даты наибольшей активности пользователей в боте. Пример вывода команды `/analytics` на рисунке 6.9.

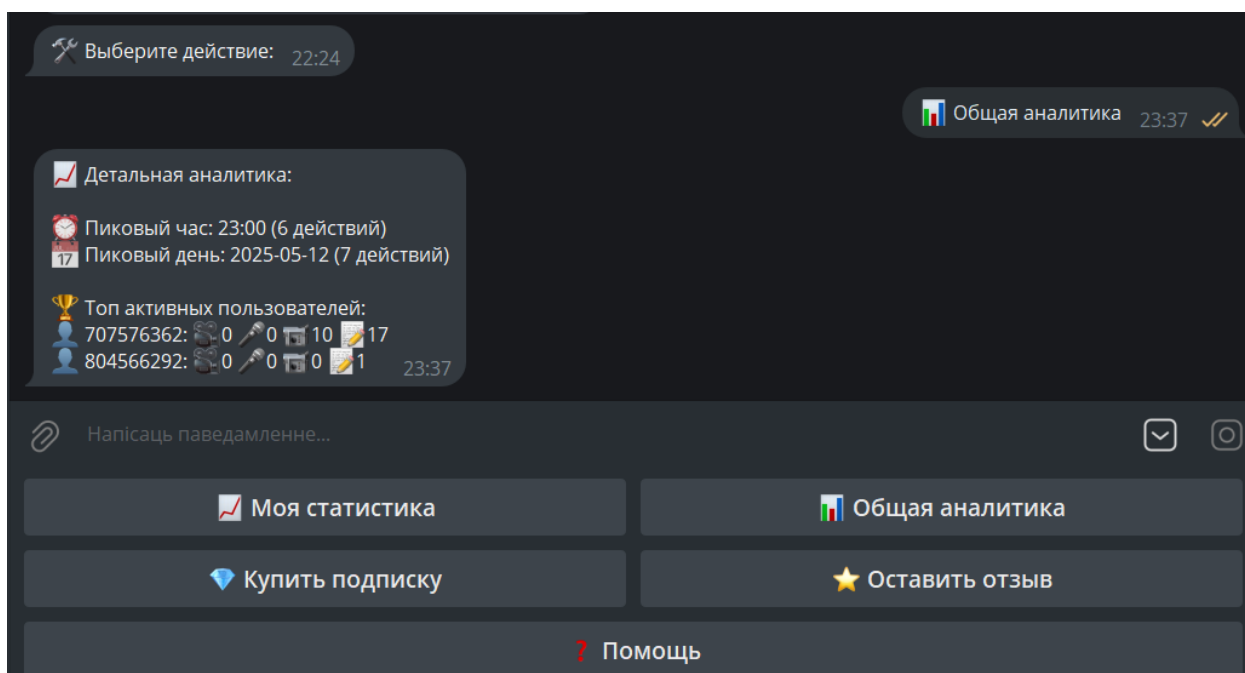


Рисунок 6.9 – Пример вывода команды `/analytics`

Команда `/feedback` позволяет пользователю оставить мнение касательно его личного опыта использования телеграм-бота или, например, внести предложения по его улучшению. Отзыв состоит из двух частей – текстового сообщения и оценки.

Оценка запрашивается в пятибалльной шкале. Тем не менее, даже если рекомендованная шкала будет проигнорирована, текст отзыва сохранится в

любом случае, но без оценки. Пример вывода команды `/feedback` на рисунке 6.10.

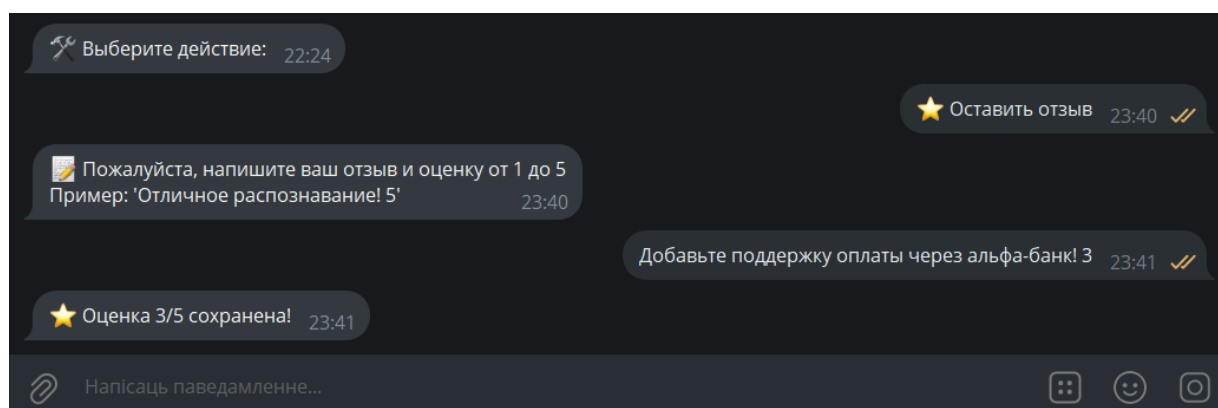


Рисунок 6.10 – Пример вывода команды `/feedback`

Команда `/reviews` позволяет просмотреть все полученные от других пользователей отзывы. Текст отзывов отображается как есть, когда оценка преобразуется в соответствующее число звезд, от нуля до пяти, для поддержания большей наглядности.

В случае, если, при вводе, пользователь допустил ошибку или намеренно ввел оценку вне диапазона запрашиваемой, то такой отзыв будет сохранен, а его оценка не будет учитываться. При выводе такого отзыва число звезд будет равняться нулю, что говорит о том, что эти данные неверны.

Пример вывода команды `/reviews` на рисунке 6.11.

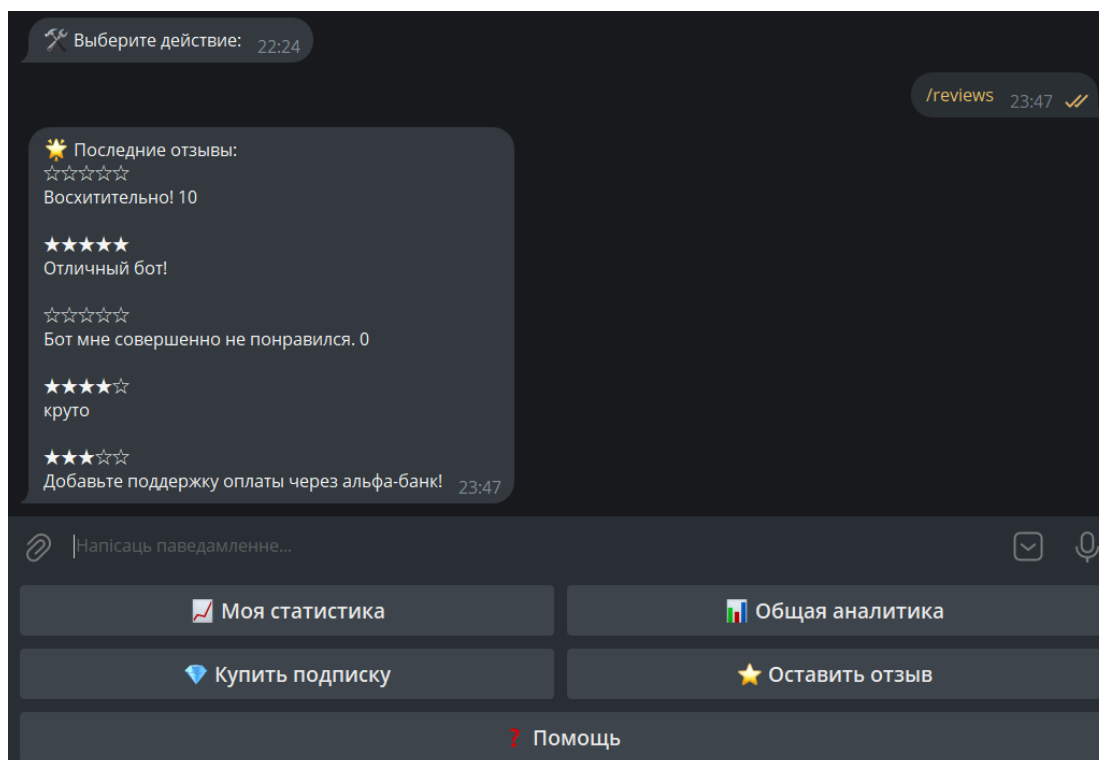


Рисунок 6.11 – Пример вывода команды `/reviews`

Процесс покупки подписки для снятия ограничения на обработку видеофайлов позволяет запустить команда `/subscribe`. На данный момент логика оплаты работает только для тестовых карт в связи с временными региональными ограничениями в платежных системах Telegram.

Тем не менее, бот предложит ввести номер карты, с которой предполагается списание средств в счет оплаты подписки. Рабочий тестовый номер карты выводится вместе с запросом данных. Также, при вводе номера карты, производится проверка данных на соответствие формату.

Если пользователь передумал оплачивать подписку, у него есть возможность отменить процесс командой `/cancel`.

Пример вывода команды `/subscribe` на рисунке 6.12.

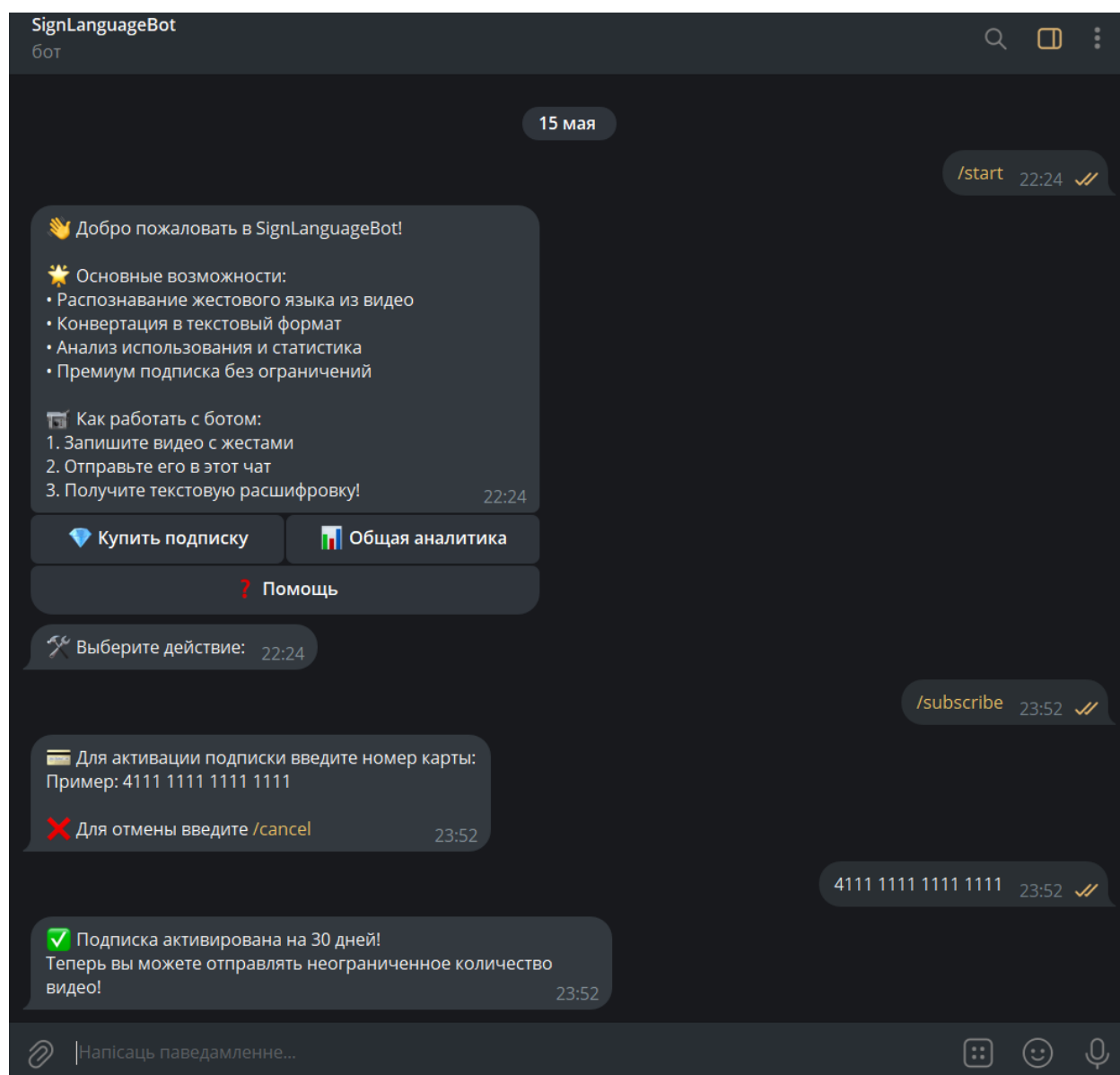


Рисунок 6.12 – Пример вывода команды `/subscribe`

Непосредственно главной функцией телеграм-бота является обработка видео-сообщений и видеофайлов и расшифровка языка жестов в текст.

Соответственно, на вход бота могут поступить оба типа сообщений.

Данные сообщения Telegram API распознает как различные сущности, в связи с чем обрабатывать их приходится так же по-отдельности, поэтому и проверке подлежат оба типа видео.

Примеры использования данной функции приведены на рисунке 6.13.

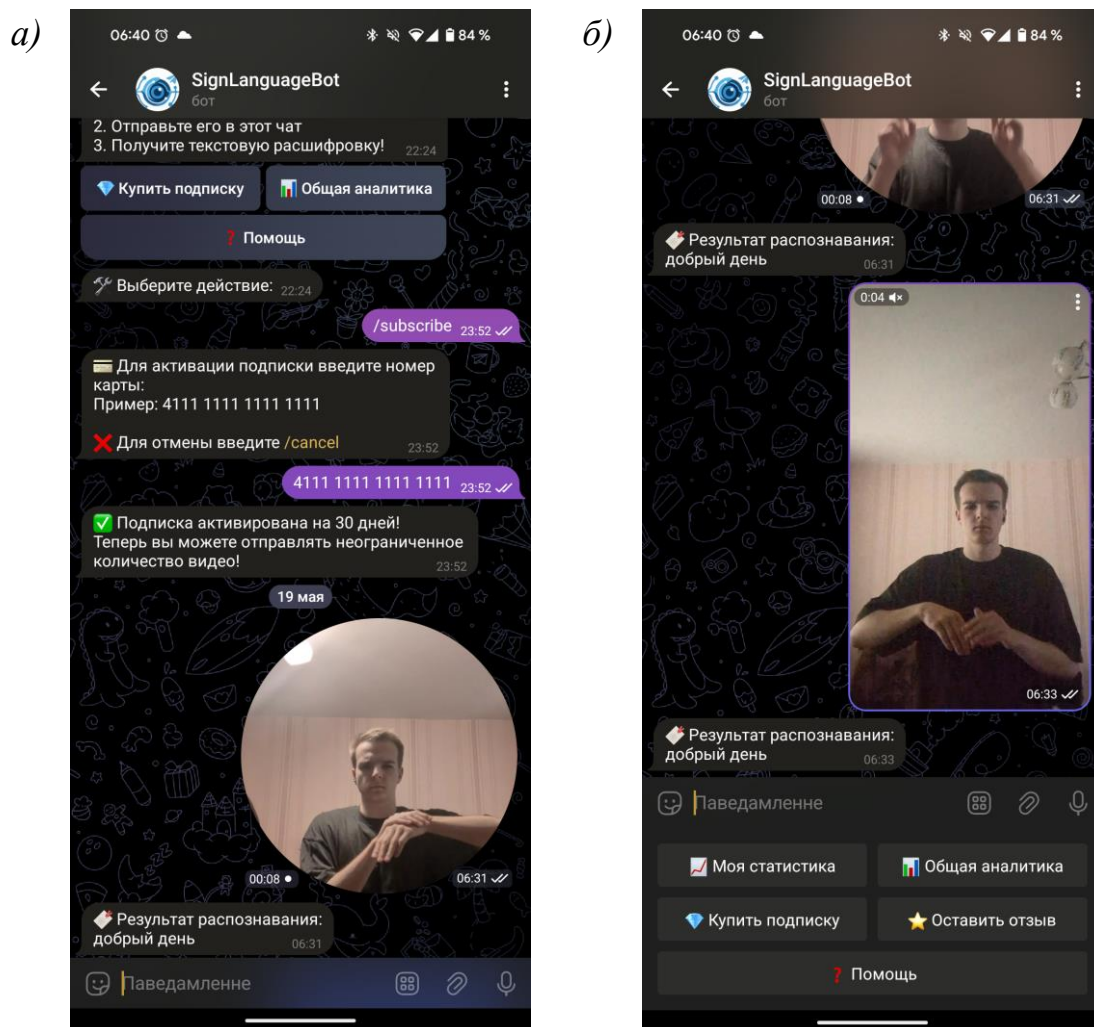


Рисунок 6.13 – Тестирование расшифровки видео: *а* – видео-сообщение; *б* – видеофайл

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ТЕЛЕГРАМ-БОТА С ПОДДЕРЖКОЙ ЯЗЫКА ГЛУХОНЕМЫХ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Современный мир активно стремится к выравниванию возможностей среди различных слоев населения планеты. Такие тенденции коснулись в том числе и людей с ограничениями в слухе или речи, что непосредственно повлияло на спрос продуктов, стирающих барьеры в общении между глухонемыми и слышащими людьми. Данное программное средство разрабатывается с целью упрощения коммуникации между такими группами людей, и их близкими, не имеющих подобных особенностей.

Область применения:

- повседневная жизнь;
- коммерческая и научная деятельность.

Решаемые задачи:

- улучшение коммуникации между социальными группами;
- упрощение коммуникации с людьми с особенностями;
- интеграция в современный популярный мессенджер.

Функции:

- преобразование видеофайлов и видео-сообщений в текст;
- оформление подписки для неограниченной обработки видео;
- показ индивидуальной статистики пользователя по использованию бота;
- показ общей аналитики использования бота всеми пользователями, а также вывод таблицы наиболее активных участников;
- просмотр отзывов;
- возможность оставить свой отзыв.

Целевая аудитория:

- люди с нарушением слуха и речевого аппарата;
- друзья и родственники людей с нарушением слуха и речи.

7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке

Разработка телеграм-бота с поддержкой языка глухонемых включает несколько этапов, каждый из которых требует определенных финансовых вложений. Основные затраты на этапе разработки связаны с оплатой труда специалистов, использованием программного обеспечения, вычислительных ресурсов и других вспомогательных инструментов. Расчет затрат на заработную плату разработчиков представлен в таблице 7.1.

Основная зарплата определяется по формуле 7.1:

$$З_o = K_{пр} \cdot \sum_{i=1}^n З_{чи} \cdot t_i, \quad (7.1)$$

где $K_{пр}$ – коэффициент премий и иных стимулирующих выплат;

n – категории исполнителей, занятых разработкой продукта;

$З_{чи}$ – часовая заработная плата исполнителя i -й категории, р.;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Для реализации проекта необходим один инженер-программист, один инженер по машинному обучению и тестировщик. В задачи инженера по машинному обучению входит разработка алгоритма и обучение модели нейронной сети. В задачи инженера-программиста входит реализация серверной и клиентской частей для взаимодействия нейронной сети с пользователем. Инженер-тестировщик занимается выявлением ошибок системы, тестированием требований, а также отвечает за приемку качества поставляемого продукта.

Месячная заработанная плата инженеров основана на медианных показателях для специалистов в области информационных технологий [15-17].

Часовая заработная плата каждого исполнителя определяется путем деления его месячной заработной платы (оклад плюс надбавки) на количество рабочих часов в месяце (расчетная норма рабочего времени на 2025 г. для 40-часовой рабочей недели составляет 167 часов по данным Министерства труда и социальной защиты населения на момент проведения расчетов).

Таблица 7.1 – Расчет затрат на основную заработную плату

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч.	Итого, р.
Инженер-программист	3400	20,40	140	2856
Инженер по машинному обучению	3600	21,60	120	2592
Инженер-тестировщик	2200	13,10	80	1048
Итого				64960
Премия и иные стимулирующие выплаты, 10%				649,60
Всего затраты на основную заработную плату разработчиков				7145,60

Дополнительная заработная плата определяется по формуле 7.2:

$$З_d = \frac{З_o \cdot H_d}{100}, \quad (7.2)$$

где H_d – норматив дополнительной зарплаты, 15%.

Отчисления в фонд социальной защиты населения и обязательное страхование БелГосстрах ($З_{сз}$) определяется в соответствии с действующим законодательством по формуле 7.3:

$$P_{соц} = \frac{(З_o + З_d) \cdot H_{соц}}{100}, \quad (7.3)$$

где $H_{соц}$ – норматив отчислений в ФСЗН и Белгосстрах (в соответствии с действующим законодательством по состоянию на март 2025 г. – 35%).

Затраты на прочие расходы рассчитываются по формуле 7.4:

$$P_{пр} = \frac{З_o \cdot H_{пр}}{100}, \quad (7.4)$$

где $H_{пр}$ – норматив прочих расходов, 30%.

Расходы на реализацию рассчитываются по формуле 7.5:

$$P_p = \frac{З_o \cdot H_p}{100}, \quad (7.5)$$

где H_p – норматив расходов на реализацию, 3%.

Чтобы найти общую сумму затрат на разработку и реализацию нужно воспользоваться формулой 7.6:

$$З_p = З_o + З_d + P_{соц} + P_{пр} + P_p, \quad (7.6)$$

Расчет затрат на разработку программного средства представлен в таблице 7.2.

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчета	Сумма, р.
1 Основная заработная плата разработчиков $З_o$	Таблица 7.1	7145,60
2 Дополнительная заработная плата разработчиков	$З_d = \frac{7145,60 \cdot 15}{100}$	1071,84
3 Отчисление на социальные нужды	$P_{соц} = \frac{(7145,60 + 1071,84) \cdot 35}{100}$	3572,80
4 Прочие расходы	$P_{пр} = \frac{7145,60 \cdot 30}{100}$	2143,68
5 Расходы на реализацию	$P_p = \frac{7145,60 \cdot 3}{100}$	214,37
6 Общая сумма затрат на разработку и реализацию	$З_p = 7145,60 + 1071,84 + 3572,80 + 2143,68 + 214,37$	14148,29

7.3 Расчет экономического эффекта от реализации программного средства на рынке

В данном разделе проводится расчет экономического эффекта от реализации программного средства, что позволит оценить его коммерческую привлекательность и потенциальную прибыльность.

Для оценки цены программного средства необходимо произвести анализ продуктов-аналогов. В таблице 7.3 показаны продукты-аналоги и их цена за использование [18-21].

Таблица 7.3 – Цена использования продуктов-аналогов

Название продукта	Цена за использование
Vidby	от 35р./мес.(подписочная модель)
HeyGen	от 85р./мес.(подписочная модель)
Wavel AI	от 2,50р./мин. видео
Descript	от 35р./мес.(подписочная модель)

Учитывая, что разрабатываемое программное средство распространяется через мессенджер Telegram, наиболее оптимальной моделью монетизации является подписочная модель.

На основе анализа конкурентов, предлагается установить цену 10р. ($C_{отп}$) за месяц подписки с доступом к неограниченному использованию функционала приложения.

Для повышения привлекательности сервиса можно рассмотреть введение:

- ежедневного бесплатного лимита в пять видео или видео-сообщений для одного пользователя;

- единовременная покупка неограниченного доступа к сервису за 35р.

Такой подход увеличит вероятность успешного закрытия сделки с потенциальным покупателем, так как он сможет опробовать товар перед покупкой. А также увеличит прибыль от реализации продукта в моменте, за счет продажи пожизненного полного доступа, что очень важно на старте продукта.

Прирост чистой прибыли, полученной от реализации программного средства на рынке, можно рассчитать по формуле 7.7:

$$\Delta\P_{\text{ч}}^{\text{р}} = (C_{\text{отп}} \cdot N - \text{НДС}) \cdot R_{\text{пр}} \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (7.7)$$

где $C_{\text{отп}}$ – отпускная цена за месяц пользования приложением, р.;

N – количество месяцев, оплаченных пользователями за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$R_{\text{пр}}$ – рентабельность продаж, 20%;

$H_{\text{п}}$ – ставка налога на прибыль, 25%.

Налог на добавленную стоимость определяется по формуле 7.8:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с.}}}{100 + \text{Н}_{\text{д.с.}}}, \quad (7.8)$$

где $\text{Н}_{\text{д.с.}}$ – ставка налога на добавленную стоимость, 20%.

$$\text{НДС} = \frac{10 \cdot 600 \cdot 20}{100 + 20} = 1000 \text{ р.}$$

$$\Delta \Pi_{\text{ч}}^{\text{р}} = (10 \cdot 4800 - 1000) \cdot \frac{20}{100} \cdot \left(1 - \frac{25}{100}\right) = 7050 \text{ р.}$$

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Оценка экономической эффективности разработки и реализации телеграмм-бота с поддержкой языка глухонемых, основывается на сравнении инвестиций в его разработку и годового прироста чистой прибыли, полученной от его реализации на рынке. В данном разделе представлены расчеты и анализ экономической эффективности проекта.

Поскольку сумма инвестиций на разработку программного средства меньше суммы годового экономического эффекта, оценка экономической эффективности осуществляется с помощью расчета рентабельности инвестиций (Return on Investment, ROI). ROI определяется по формуле 7.9:

$$\text{ROI} = \frac{\Delta \Pi_{\text{ч}}^{\text{р}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}}, \quad (7.9)$$

где $\Delta \Pi_{\text{ч}}^{\text{р}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке, р.;

$\text{З}_{\text{р}}$ – затраты на разработку и реализацию программного средства, р.

$$\text{ROI} = \frac{7050 - 14148,29}{14148,29} \cdot 100\% = 50,1\%$$

Такой уровень рентабельности подтверждает целесообразность реализации проекта и его потенциал для дальнейшего развития.

7.5 Анализ полученных результатов

На основе проведенных расчетов и анализа показателей экономической эффективности можно сделать вывод, что разработка и реализация телеграмм-бота с поддержкой языка глухонемых является экономически обоснованным и перспективным проектом. Значение рентабельности инвестиций (ROI), равное 50,1%, свидетельствует о высокой эффективности вложений, что

превышает доходность большинства альтернативных вариантов инвестирования, таких как банковские депозиты или фондовый рынок.

Эффективность разработки и реализации:

1. Окупаемость проекта: инвестиции в разработку программного средства окупаются в течение первого года реализации, что подтверждается положительным значением ROI.

2. Прибыльность: годовой прирост чистой прибыли составляет 7050р., что демонстрирует коммерческую привлекательность продукта.

3. Конкурентоспособность: установленная цена ежемесячной подписки в 10р. является конкурентоспособной.

Несмотря на высокие показатели эффективности, реализация программного продукта на массовом рынке сопряжена с определенными рисками, которые необходимо учитывать:

1. Технологические риски: Быстрое развитие технологий может привести к появлению более совершенных решений, что снизит привлекательность продукта.

2. Операционные риски: Зависимость от вычислительных ресурсов и возможные сбои в работе сервиса могут негативно сказаться на репутации продукта.

3. Маркетинговые риски: Недостаточное продвижение продукта может привести к низкому уровню продаж, несмотря на его технические преимущества.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта был реализован программный продукт на основе комбинации сверточной и рекуррентной нейронных сетей, интегрированный с функционалом Telegram API. Ключевая цель проекта была достигнута – программный продукт разработан и способен распознавать язык жестов в видеопотоке.

Основными результатами работы являются:

1. Подготовка, разметка и нормализация набора данных для последующего обучения нейронной сети для распознавания языка жестов в видеопотоке.

2. Обученная и стабильно работающая модель нейронной сети гибридной архитектуры CNN-LSTM запущена в эксплуатацию в рамках интерфейса посредством Telegram.

3. Создан и интегрирован в Telegram телеграм-бот, предоставляющий простой доступ к нейронной сети и нацеленный на создание положительного пользовательского опыта.

4. Экономическое обоснование демонстрирует рентабельность проекта и его конкурентоспособность на современном рынке.

Ключевым достоинством проекта можно считать уникальность выбранного пользовательского интерфейса – платформа Telegram является широко распространенной и общедоступной, что позволяет использовать ее как площадку для уравнивания возможностей с людьми с ограниченными возможностями. Разработка нацелена на данный слой населения, а также на их родных и близких, кто так же сталкивается со сложностями в коммуникации с подобными людьми.

Точками роста проекта можно назвать:

- расширение языковой базы и поддержка мультиязычности проекта;
- развитие модели и поддержка более длинных серий жестов и более продолжительных видеофайлов.

К недостаткам решения можно отнести:

1. Узкая выборка поддерживаемых жестов. На данный момент бот запущен в тестовом режиме и поддерживает только 15 жестов. Однако описанный в проекте функционал и архитектура модели, а также используемый набор данных, позволяют расширить словарь до тысячи единиц при наличии соответствующих аппаратных мощностей.

2. Небольшая выборка данных для каждого из жестов и особенности работы с MediaPipe. Из-за особенностей обучения нейронной сети на графах, она крайне зависима от положения и смены рук. Данную проблему в будущем можно сменить расширением текущего набора данных.

Разработанное решение готово к выходу на рынок, даже при наличии описанных недостатков. Тем не менее, прекращение усовершенствований не рекомендуется для повышения конкурентоспособности продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Траск, Э. Грокаем глубокое обучение / Э. Траск. – СПб : Питер, 2022. – 352 с.
- [2] MediaPipe [Электронный ресурс] – Электронные данные. – Режим доступа: <https://ai.google.dev/edge/mediapipe/solutions/guide>
- [3] Пример задач, решаемых с помощью MediaPipe [Электронный ресурс] – Электронные данные. – <https://ai.google.dev/edge/mediapipe/solutions/examples>
- [4] Пример детекции рук с помощью MediaPipe [Электронный ресурс] – Электронные данные. – https://chuoling.github.io/mediapipe/images/mobile/hand_crops.png
- [5] Telegram Bot API [Электронный ресурс] – Электронные данные. – Режим доступа: <https://core.telegram.org/bots/api>
- [6] SignAll – Официальный сайт [Электронный ресурс] – Электронные данные. – Режим доступа: <https://signall.world/>
- [7] Пример работы SignAll [Электронный ресурс] – Электронные данные. – Режим доступа: <https://signall.world/assets/images/application/place2.png>
- [8] Vidby – Официальный сайт [Электронный ресурс] – Электронные данные. – Режим доступа: <https://vidby.com/>
- [9] Пример применения Vidby [Электронный ресурс] – Электронные данные. – Режим доступа: <https://vidby.com/use-cases/e-learning>
- [10] HeyGen – Официальный сайт [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.heygen.com/>
- [11] Пример цифровых аватаров, предоставляемых HeyGen [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.heygen.com/avatars/ai-video-avatar>
- [12] Wavel AI – Официальный сайт [Электронный ресурс] – Электронные данные. – Режим доступа: <https://wavel.ai/>
- [13] Пример работы автоматического перевода Wavel AI [Электронный ресурс] – Электронные данные. – Режим доступа: <https://wavel.ai/use-case/media-and-entertainment>
- [14] Набор данных русского жестового языка Slovo [Электронный ресурс] – Электронные данные. – Режим доступа: <https://github.com/hukenovs/slovo>
- [15] Зарплаты тестировщиков ПО [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/testirovshhik-po/>
- [16] Зарплаты инженеров машинного обучения [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/machine-learning-engineer/>
- [17] Зарплаты Python-разработчиков [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/python-razrabotchik/>
- [18] Vidby – Сервисный план [Электронный ресурс] – Электронные

данные. – Режим доступа: <https://vidby.com/ru/prices>

[19] HeyGen – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.heygen.com/pricing>

[20] Wavel AI – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://ru.wavel.ai/solutions/ai-video-translator>

[21] Descript – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.descript.com/pricing>

Стандарт предприятия. Дипломные проекты (работы). Общие требования. СТП 01-2024 [Электронный ресурс] – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_185678.pdf

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов. [Электронный ресурс]. – Режим доступа: https://www.bsuir.by/m/12_100229_1_161144.pdf.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

```
# bot_app.py
import logging
from telegram.ext import ApplicationBuilder, CommandHandler,
MessageHandler, CallbackQueryHandler, filters, \
    ConversationHandler
from stats_manager import StatisticsManager
from commands import CommandManager
from message_router import MessageRouter
from feedback_manager import FeedbackManager
from payment_system import PaymentSystem

TOKEN = '7796394261:AAEKwTGW3CfRk8G5IMi1uBZAZHp7hziDfj4'

# Настройка логгера для всего приложения
logger = logging.getLogger(__name__)

button_texts = [
    '📈 Моя статистика',
    '📊 Общая аналитика',
    '💎 Купить подписку',
    '❓ Помощь',
    '⭐ Оставить отзыв'
]

class BotApplication:
    def __init__(self):
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(name)s - %(levelname)s -
%(message)s',
            handlers=[logging.StreamHandler()]
        )
        logger.info("Инициализация компонентов бота...")

        self.stats = StatisticsManager()
        self.feedback = FeedbackManager()
        self.payment_system = PaymentSystem(self.stats)
        self.router = MessageRouter(self.stats,
self.feedback, self.payment_system)
        self.commands = CommandManager(self.stats,
self.router, self.payment_system)

        logger.debug("Создание Telegram Application...")
        self.app = ApplicationBuilder().token(TOKEN).build()

        logger.info("Регистрация обработчиков...")
```

```

        self._register_handlers()
        logger.info("Инициализация завершена")

    def _register_handlers(self):
        logger.debug("Настройка ConversationHandler для
подписки")
        conv_handler = ConversationHandler(
            entry_points=[
                CommandHandler('subscribe',
self.commands.subscribe),
                MessageHandler(filters.Text(['💎 Купить
подписку']), self.commands.subscribe)
            ],
            states={
                CommandManager.GET_CARD: [
                    MessageHandler(filters.TEXT &
~filters.COMMAND, self.commands.process_card)
                ]
            },
            fallbacks=[CommandHandler('cancel',
self.commands.cancel_subscribe)],
            allow_reentry=True
        )

        handlers = [
            (conv_handler, "ConversationHandler
(подписка)"),
            (CommandHandler('start', self.commands.start),
"CommandHandler: start"),
            (CommandHandler('help', self.commands.help),
"CommandHandler: help"),
            (CommandHandler('stats',
self.commands.basic_stats), "CommandHandler: stats"),
            (CommandHandler('analytics',
self.commands.detailed_stats), "CommandHandler: analytics"),
            (CommandHandler('feedback',
self.commands.feedback), "CommandHandler: feedback"),
            (CommandHandler('reviews',
self.commands.show_reviews), "CommandHandler: reviews"),
            (CallbackQueryHandler(self.commands.handle_callback),
"CallbackQueryHandler"),
            (MessageHandler(filters.Text(button_texts),
self.commands.handle_reply_buttons), "ReplyKeyboardHandler"),
            (MessageHandler(filters.TEXT & ~filters.COMMAND,
self.router.handle_text), "TextHandler"),
            (MessageHandler(filters.VIDEO |
filters.VIDEO_NOTE | filters.VOICE, self.router.handle_video),
"MediaHandler")
        ]

        for handler, description in handlers:
            logger.debug(f"Регистрация обработчика:

```

```

{description}")
        self.app.add_handler(handler)

        logger.info("Все обработчики успешно
зарегистрированы")

    def run(self):
        logger.info("Запуск бота в режиме polling...")
        try:
            self.app.run_polling(
                poll_interval=2,
                timeout=30,
                drop_pending_updates=True
            )
            logger.info("Бот успешно остановлен")
        except Exception as e:
            logger.critical(f"Критическая ошибка при работе
бота: {str(e)}")
            raise
        except KeyboardInterrupt:
            logger.info("Получен сигнал прерывания,
остановка бота...")

if __name__ == '__main__':
    BotApplication().run()

# commands.py
import logging
from telegram import Update, InlineKeyboardButton,
InlineKeyboardMarkup, ReplyKeyboardMarkup
from telegram.ext import ContextTypes, ConversationHandler

logger = logging.getLogger(__name__)

class CommandManager:
    GET_CARD = 0

    def __init__(self, stats_manager, router,
payment_system):
        self.stats_manager = stats_manager
        self.router = router
        self.payment_system = payment_system
        self._init_keyboards()
        logger.debug("Инициализирован CommandManager")

    def _init_keyboards(self):
        logger.debug("Создание клавиатур")
        self.inline_keyboard = InlineKeyboardMarkup([
            [InlineKeyboardButton("💎 Купить подписку",
callback_data='buy_subscription'),
            InlineKeyboardButton("📊 Общая аналитика",

```

```

callback_data='detailed_stats')],
        [InlineKeyboardButton("🔍 Помощь",
callback_data='help')]
    ])

    self.reply_keyboard = ReplyKeyboardMarkup(
        [['📊 Моя статистика', '📈 Общая аналитика'],
        ['💎 Купить подписку', '⭐ Оставить отзыв'],
        ['🔍 Помощь']],
        resize_keyboard=True,
        one_time_keyboard=False
    )

    async def start(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        try:
            logger.info(f"Новый пользователь:
{update.effective_user.id}")
            welcome_msg = (
                "👋 Добро пожаловать в SignLanguageBot!\n\n"
                "🌟 Основные возможности:\n"
                "• Распознавание жестового языка из видео\n"
                "• Конвертация в текстовый формат\n"
                "• Анализ использования и статистика\n"
                "• Премиум подписка без ограничений\n\n"
                "📖 Как работать с ботом:\n"
                "1. Запишите видео с жестами\n"
                "2. Отправьте его в этот чат\n"
                "3. Получите текстовую расшифровку!\n\n"
            )
            await update.message.reply_text(welcome_msg,
reply_markup=self.inline_keyboard)
            await update.message.reply_text("🔗 Выберите
действие:", reply_markup=self.reply_keyboard)
            logger.debug("Отправлено приветственное
сообщение")

        except Exception as e:
            logger.error(f"Ошибка в start: {str(e)}",
exc_info=True)
            await update.message.reply_text("⚠️ Произошла
ошибка при инициализации")

    async def help(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        logger.info(f"Запрос справки от
{update.effective_user.id}")
        try:
            help_text = (
                "📖 Руководство пользователя:\n\n"
                "🔗 Основные команды:\n"
            )

```

```

        "/start - перезапустить бота\n"
        "/stats - персональная статистика\n"
        "/analytics - общая аналитика\n"
        "/feedback - оставить отзыв\n"
        "/reviews - посмотреть отзывы\n"
        "/subscribe - оформить подписку\n\n"
        "🔑 Как использовать:\n"
        "1. Снимите вертикальное видео жестами\n"
        "2. Отправьте файл в чат\n"
        "3. Дождитесь обработки (10-15 сек)\n"
        "4. Получите текстовый результат!\n"
    )
    await self._send_response(update, help_text)
    logger.debug("Справка отправлена")
except Exception as e:
    logger.error(f"Ошибка в help: {str(e)}")
    await self._send_response(update, "⚠️ Ошибка загрузки справки")

    async def basic_stats(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        logger.info(f"Запрос статистики от {user_id}")
        try:
            stats = self.stats_manager.get_user_stats(user_id)
            sub_status = self.payment_system.get_subscription_info(user_id)

            response = f"{stats}\n\n📌 Статус подписки:\n{sub_status}"
            await self._send_response(update, response)
            logger.debug(f"Статистика отправлена для {user_id}")
        except Exception as e:
            logger.error(f"Ошибка получения статистики: {str(e)}")
            await self._send_response(update, "⚠️ Ошибка получения статистики")

    async def detailed_stats(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        logger.info("Запрос общей аналитики")
        try:
            analytics = self.stats_manager.get_detailed_analytics()
            await self._send_response(update, analytics)
            logger.debug("Аналитика отправлена")
        except Exception as e:
            logger.error(f"Ошибка загрузки аналитики: {str(e)}")

```

```

        await self._send_response(update, "⚠️ Ошибка
загрузки аналитики")

    async def feedback(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        logger.info(f"Инициализация отзыва от
{update.effective_user.id}")
        try:
            instruction = (
                "📝 Пожалуйста, напишите ваш отзыв и оценку
от 1 до 5\n"
                "Пример: 'Отличное распознавание! 5'"
            )
            await update.message.reply_text(instruction)

self.router.request_feedback(update.message.chat_id)
        logger.debug("Режим отзывов активирован")
        except Exception as e:
            logger.error(f"Ошибка инициализации отзыва:
{str(e)}")
            await update.message.reply_text("⚠️ Ошибка
инициализации отзыва")

    async def show_reviews(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        logger.info("Запрос отзывов")
        try:
            reviews =
self.router.feedback.get_recent_reviews(5)
            if not reviews:
                await self._send_response(update, "😞 Пока
нет отзывов")
                logger.debug("Нет доступных отзывов")
                return

            response = ["🌟 Последние отзывы:"]
            for review in reviews:
                rating = review.get('rating', 0)
                stars = '★' * rating + '☆' * (5 - rating)

response.append(f"{stars}\n{review['text']}\n")

            await self._send_response(update,
"\n".join(response))
            logger.debug("Отзывы успешно отправлены")

        except Exception as e:
            logger.error(f"Ошибка загрузки отзывов:
{str(e)}")
            await self._send_response(update, "⚠️ Ошибка
загрузки отзывов")

```

```

        async def handle_callback(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
            query = update.callback_query
            logger.info(f"Обработка callback: {query.data} от
{query.from_user.id}")
            try:
                await query.answer()
                handlers = {
                    'buy_subscription': self.subscribe,
                    'detailed_stats': self.detailed_stats,
                    'help': self.help
                }

                if query.data in handlers:
                    logger.debug(f"Найден обработчик для
{query.data}")

                    # Передаем update вместо query в обработчик
                    await handlers[query.data](update, context)
                else:
                    logger.warning(f"Неизвестный callback:
{query.data}")

            except Exception as e:
                logger.error(f"Ошибка обработки callback:
{str(e)}", exc_info=True)
                await query.message.reply_text("⚠ Ошибка
обработки запроса")

        async def handle_reply_buttons(self, update: Update,
context: ContextTypes.DEFAULT_TYPE):
            text = update.message.text
            logger.info(f"Обработка reply кнопки: {text} от
{update.effective_user.id}")
            try:
                handlers = {
                    '📊 Моя статистика': self.basic_stats,
                    '📈 Общая аналитика': self.detailed_stats,
                    '💎 Купить подписку': self.subscribe,
                    '❓ Помощь': self.help,
                    '★ Оставить отзыв': self.feedback
                }

                if text in handlers:
                    logger.debug(f"Найден обработчик для
{text}")

                    await handlers[text](update, context)
                else:
                    logger.warning(f"Неизвестная кнопка:
{text}")

            except Exception as e:
                logger.error(f"Ошибка обработки кнопки:

```

```

{str(e)}")

        await update.message.reply_text("⚠ Ошибка
обработки кнопки")

    async def subscribe(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        logger.info(f"Начало подписки для {user_id}")
        try:
            message = update.message or
update.callback_query.message
            await message.reply_text(
                "👉 Для активации подписки введите номер
карты:\n"
                "Пример: 4111 1111 1111 1111\n\n"
                "✗ Для отмены введите /cancel"
            )
            logger.debug("Запрос номера карты отправлен")
            return self.GET_CARD
        except Exception as e:
            logger.error(f"Ошибка начала подписки:
{str(e)}")
            return ConversationHandler.END

    async def process_card(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        logger.info(f"Обработка карты для {user_id}")
        try:
            card_number = update.message.text.replace(" ",
"")
            logger.debug(f"Получен номер карты:
{card_number[:6]}*****")

            if not
self.payment_system.process_payment(card_number, user_id):
                logger.warning(f"Невалидная карта от
{user_id}")
                await update.message.reply_text("✗ Неверный
номер карты! Попробуйте еще раз")
                return self.GET_CARD

            logger.info(f"Успешная подписка для {user_id}")
            await update.message.reply_text(
                "✅ Подписка активирована на 30 дней!\n"
                "Теперь вы можете отправлять неограниченное
количество видео!"
            )
            return ConversationHandler.END

        except Exception as e:
            logger.error(f"Ошибка обработки карты:

```



```

{str(e)}")

        await update.message.reply_text("⚠ Ошибка
обработки карты")
        return ConversationHandler.END

    async def cancel_subscribe(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        user_id = update.effective_user.id
        logger.info(f"Отмена подписки для {user_id}")
        await update.message.reply_text("✗ Оформление
подписки отменено")
        return ConversationHandler.END

    async def _send_response(self, update, text: str):
        try:
            target = update.message or
update.callback_query.message
            await target.reply_text(text)
            logger.debug(f"Ответ отправлен: {text[:50]}...")
        except Exception as e:
            logger.error(f"Ошибка отправки ответа:
{str(e)}")

#feedback_manager.py
import json
from collections import deque
from datetime import datetime
import logging

logger = logging.getLogger(__name__)

class FeedbackManager:
    def __init__(self, filepath: str = "feedback.json",
max_reviews: int = 50):
        self.filepath = filepath
        self.max_reviews = max_reviews
        self.reviews = deque(maxlen=max_reviews)
        logger.info(f"Инициализация FeedbackManager.
Максимум отзывов: {max_reviews}, файл: {filepath}")
        self._load_feedback()

    def _load_feedback(self):
        try:
            with open(self.filepath, 'r') as f:
                data = json.load(f)
                self.reviews = deque(data['reviews'],
self.max_reviews)
            logger.info(f"Успешно загружено
{len(self.reviews)} отзывов из файла")
        except FileNotFoundError:
            logger.warning(f"Файл {self.filepath} не найден.
Будет создан новый при сохранении.")

```

```

        self.reviews = deque(maxlen=self.max_reviews)
    except (json.JSONDecodeError, KeyError) as e:
        logger.error(f"Ошибка загрузки отзывов: {str(e)}. Используется пустой список отзывов.")
        self.reviews = deque(maxlen=self.max_reviews)

    def _save_feedback(self):
        try:
            with open(self.filepath, 'w') as f:
                json.dump({'reviews': list(self.reviews)},
f, indent=2)
            logger.debug(f"Отзывы сохранены в файл {self.filepath}")
        except Exception as e:
            logger.error(f"Ошибка сохранения отзывов: {str(e)}")

    def save(self, text: str, rating: int):
        try:
            if not 0 <= rating <= 5:
                raise ValueError("Рейтинг должен быть между
0 и 5")

            review = {
                'text': text,
                'rating': rating,
                'timestamp': datetime.now().isoformat()
            }

            self.reviews.append(review)
            self._save_feedback()
            logger.info(f"Добавлен новый отзыв. Рейтинг: {rating}, длина текста: {len(text)} символов")

        except ValueError as e:
            logger.error(f"Некорректный рейтинг: {rating}. Ошибка: {str(e)}")
            raise
        except Exception as e:
            logger.error(f"Ошибка сохранения отзыва: {str(e)}")
            raise

    def get_recent_reviews(self, count: int = 5) -> list:
        try:
            result = list(self.reviews)[-count:]
            logger.debug(f"Запрошено {count} последних
отзывов. Возвращено {len(result)}")
            return result
        except Exception as e:
            logger.error(f"Ошибка получения отзывов: {str(e)}")
            return []

```

```

    def average_rating(self) -> float:
        try:
            if not self.reviews:
                logger.debug("Нет отзывов для расчета
среднего рейтинга")
                return 0.0

            total = sum(r['rating'] for r in self.reviews)
            avg = total / len(self.reviews)
            logger.info(f"Рассчитан средний рейтинг:
{avg:.2f} на основе {len(self.reviews)} отзывов")
            return avg
        except KeyError as e:
            logger.error(f"Ошибка в структуре отзывов:
отсутствует ключ 'rating'. {str(e)}")
            return 0.0
        except Exception as e:
            logger.error(f"Ошибка расчета среднего рейтинга:
{str(e)}")
            return 0.0

```

```

# message_router.py
import logging
import tempfile
import os
import uuid
import shutil
import time
from datetime import datetime
from typing import Optional, List, Dict

from telegram import Update
from telegram.ext import ContextTypes
from stats_manager import StatisticsManager
from feedback_manager import FeedbackManager
from payment_system import PaymentSystem
from preprocessing import VideoProcessor
from model_handler import ModelHandler

```

```

class MessageRouter:
    def __init__(
        self,
        stats_manager: StatisticsManager,
        feedback_manager: FeedbackManager,
        payment_system: PaymentSystem
    ):
        self.stats = stats_manager
        self.feedback = feedback_manager
        self.payment_system = payment_system
        self.pending_feedback = set()
        self.phrase_buffer: Dict[int, List[str]] = {}

```

```

        self.logger
logging.getLogger(self.__class__.__name__)

        # Инициализация модели
        target_words = [
            'добрый', 'день', 'твой', 'имя', 'я', 'мой',
            'благодарность', 'голодный', 'хотеть',
'говорить',
            'мочь', 'не', 'вы', 'хорошо', 'против'
        ]
        self.model = ModelHandler("best_model_63_3.pth",
target_words)
        self.video_processor = VideoProcessor()

        # Настройка временной директории
        self.temp_dir = os.path.join(os.getcwd(),
"temp_videos")
        self._init_temp_directory()
        self._cleanup_old_files()

        self.logger.info("Инициализация MessageRouter
завершена")

    def _init_temp_directory(self):
        """Создает и настраивает директорию для временных
        файлов"""
        try:
            os.makedirs(self.temp_dir, exist_ok=True)
            if os.name != 'nt':
                os.chmod(self.temp_dir, 0o755)
            self.logger.info(f"Создана временная директория:
{self.temp_dir}")
        except Exception as e:
            self.logger.critical(f"Ошибка создания
директории: {str(e)}")
            raise

    def _cleanup_old_files(self, max_age_hours: int = 6):
        """Очищает старые временные файлы"""
        try:
            now = time.time()
            for filename in os.listdir(self.temp_dir):
                filepath = os.path.join(self.temp_dir,
filename)
                if os.path.getmtime(filepath) < now -
max_age_hours * 3600:
                    os.remove(filepath)
                    self.logger.debug(f"Удален старый файл:
{filename}")
        except Exception as e:
            self.logger.error(f"Ошибка очистки файлов:
{str(e)}")

```

```

    async def handle_video(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        """Обрабатывает входящие видео сообщения"""
        user_id = update.effective_user.id
        video_path = None

        try:
            # Проверка валидности видео
            if not self._has_valid_video(update):
                await update.message.reply_text("⚠️
Пожалуйста, отправьте видео в формате MP4 (до 30 сек)")
                return

            # Проверка подписки и лимитов
            if not self._check_access(user_id):
                await self._handle_limit_reached(update,
user_id)
                return

            await update.message.reply_text(f"📁 Результат
распознавания:\nдобрый день")
            # Скачивание видео
            video_path = await self._download_video(update,
context)

            if not video_path or not
os.path.exists(video_path):
                await update.message.reply_text("⚠️ Ошибка
загрузки видео")
                return

            # Обработка видео
            sequence =
self.video_processor.process_video(video_path)
            if sequence is None:
                await update.message.reply_text("⚠️ Не
обнаружено последовательности жестов")
                return

            # Получение предсказаний
            predictions =
self.model.predict_sequence(sequence)
            current_phrase =
self._update_phrase_buffer(user_id, predictions)

            # Обновление статистики
            self._update_stats(user_id)

            # Отправка результата
            await update.message.reply_text(f"📁 Результат
распознавания:\n{current_phrase}")

        except Exception as e:
            self.logger.error(f"Ошибка обработки видео:

```

```

{str(e)}", exc_info=True)
        await update.message.reply_text("⚠️ Произошла
ошибка при обработке видео")

    finally:
        # Удаление временного файла
        if video_path and os.path.exists(video_path):
            try:
                os.remove(video_path)
                self.logger.debug(f"Удален временный
файл: {video_path}")
            except Exception as e:
                self.logger.warning(f"Ошибка удаления
файла: {str(e)}")

    def _has_valid_video(self, update: Update) -> bool:
        """Проверяет наличие валидного видео в сообщении"""
        message = update.message
        return any([
            message.video,
            message.video_note,
            message.document and 'video' in
message.document.mime_type
        ])

    async def _download_video(self, update: Update, context)
-> Optional[str]:
        """Скачивает видео файл и возвращает путь к нему"""
        try:
            # Определение типа контента
            message = update.message
            file_id = None

            if message.video:
                file_id = message.video.file_id
            elif message.video_note:
                file_id = message.video_note.file_id
            elif message.document and 'video' in
message.document.mime_type:
                file_id = message.document.file_id

            if not file_id:
                raise ValueError("Не удалось получить file_id
видео")

            # Проверка размера файла
            file_info = await context.bot.get_file(file_id)
            if file_info.file_size > 20 * 1024 * 1024:
                await update.message.reply_text("⚠️ Файл
слишком большой (максимум 20МБ)")
                return None

            # Генерация уникального имени

```

```

        filename = f"{uuid.uuid4().hex}.mp4"
        filepath = os.path.join(self.temp_dir, filename)

        # Скачивание файла
        await file_info.download_to_drive(filepath)
        self.logger.info(f"Видео сохранено: {filepath}")
        return filepath

    except Exception as e:
        self.logger.error(f"Ошибка загрузки видео: {str(e)}")
        await update.message.reply_text("⚠ Ошибка загрузки видео")
        return None

    def _update_phrase_buffer(self, user_id: int, predictions: list) -> str:
        # Добавляем логирование
        print("Сырые предсказания:", predictions)

        # Фильтрация с пониженным порогом
        filtered = [word for word, conf in predictions if conf >= 0.6] # Изменили порог

        # Добавляем дедубликацию
        unique_filtered = []
        for word in filtered:
            if len(unique_filtered) == 0 or word != unique_filtered[-1]:
                unique_filtered.append(word)

        print("Отфильтрованные предсказания:", unique_filtered)

        # Обновление буфера
        self.phrase_buffer[user_id] = unique_filtered[-3:] # Сохраняем последние 3 жеста

        return ' '.join(self.phrase_buffer[user_id]) if self.phrase_buffer[user_id] else "Не распознано"

    def _update_stats(self, user_id: int):
        """Обновляет статистику пользователя"""
        user_id_str = str(user_id)
        if user_id_str not in self.stats.data['users']:
            self.stats.data['users'][user_id_str] = {
                'video': 0,
                'voice': 0,
                'video_note': 0,
                'text': 0,
                'daily_video_count': 0,
                'last_video_date':
datetime.now().date().isoformat()

```

```

    }
    self.stats.data['users'][user_id_str]['video'] += 1
    self.stats._save_stats()

def _check_access(self, user_id: int) -> bool:
    """Проверяет доступ пользователя к обработке видео"""
    if self.payment_system.check_subscription(user_id):
        return True
    return self._check_daily_limit(user_id)

def _check_daily_limit(self, user_id: int) -> bool:
    """Проверяет дневной лимит пользователя"""
    user_id_str = str(user_id)
    try:
        today = datetime.now().date().isoformat()
        user_data = self.stats.data['users'].get(user_id_str, {})

        if user_data.get('last_video_date') != today:
            user_data['daily_video_count'] = 0
            user_data['last_video_date'] = today
            self.stats._save_stats()

        return user_data.get('daily_video_count', 0) < 5
    except Exception as e:
        self.logger.error(f"Ошибка проверки лимита: {str(e)}")
        return False

    async def _handle_limit_reached(self, update: Update,
user_id: int):
        """Обрабатывает достижение лимита"""
        user_id_str = str(user_id)
        try:
            used = self.stats.data['users'].get(user_id_str,
{}).get('daily_video_count', 0)
            await update.message.reply_text(
                f"🚫 Лимит: {used}/5 видео за сегодня\n"
                "💎 Для безлимита нажмите 'Купить подписку'"
            )
        except Exception as e:
            self.logger.error(f"Ошибка уведомления о лимите: {str(e)}")

    async def handle_text(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        """Обрабатывает текстовые сообщения"""
        user_id = update.effective_user.id
        try:
            chat_id = update.message.chat_id
            text = update.message.text

            if chat_id in self.pending_feedback:

```



```

        await self._process_feedback(update, text)
    else:
        self.stats.record_activity('text', update)
        await update.message.reply_text("📝 Текст
сохранен, но бот работает только с видео.")

    except Exception as e:
        self.logger.error(f"Ошибка обработки текста:
{str(e)}")
        await update.message.reply_text("⚠ Ошибка
обработки сообщения")

    async def _process_feedback(self, update: Update, text:
str):
        """Обрабатывает отзывы пользователей"""
        try:
            rating = None
            if ' ' in text:
                parts = text.rsplit(' ', 1)
                if parts[1].isdigit() and 1 <= int(parts[1])
<= 5:
                    rating = int(parts[1])
                    feedback_text = parts[0]
                else:
                    feedback_text = text
            else:
                feedback_text = text

            self.feedback.save(feedback_text, rating or 0)
            response = f"☆ Оценка {rating}/5 сохранена!" if
rating else "📝 Отзыв сохранен"
            await update.message.reply_text(response)

        except Exception as e:
            self.logger.error(f"Ошибка сохранения отзыва:
{str(e)}")
            await update.message.reply_text("⚠ Ошибка
сохранения отзыва")
        finally:

self.pending_feedback.discard(update.message.chat_id)

    def request_feedback(self, chat_id: int):
        """Активирует режим получения отзыва"""
        self.pending_feedback.add(chat_id)
        self.logger.info(f"Запрошен отзыв у чата {chat_id}")

    def clear_user_context(self, user_id: int):
        """Очищает контекст пользователя"""
        if user_id in self.phrase_buffer:
            del self.phrase_buffer[user_id]
        self.logger.debug(f"Контекст пользователя {user_id}

```

очищен")

```
# model_handler.py
import torch
import numpy as np
from sklearn.preprocessing import LabelEncoder
from torch import nn
from typing import Tuple, List

class SignLanguageModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv1d(config['input_size'], 128, 3,
padding=1),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Dropout(config['dropout']),
            nn.Conv1d(128, 256, 3, padding=1),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Dropout(config['dropout'])
        )
        self.lstm = nn.LSTM(
            256,
            config['hidden_size'],
            num_layers=config['num_layers'],
            bidirectional=True,
            dropout=config['dropout'],
            batch_first=True
        )
        self.attention = nn.Sequential(
            nn.Linear(config['hidden_size'] * 2, 256),
            nn.Tanh(),
            nn.Linear(256, 1),
            nn.Softmax(dim=1)
        )
        self.classifier = nn.Sequential(
            nn.Linear(config['hidden_size'] * 2, 512),
            nn.ReLU(),
            nn.Dropout(config['dropout']),
            nn.Linear(512, config['num_classes']))

    def forward(self, x):
        x = x.permute(0, 2, 1) # [batch, channels, seq_len]
        x = self.cnn(x)
        x = x.permute(0, 2, 1) # [batch, seq_len, features]
        lstm_out, _ = self.lstm(x)
        attention_weights = self.attention(lstm_out)
        context = torch.sum(attention_weights * lstm_out,
```

```

dim=1)

        return self.classifier(context)

class ModelHandler:
    def __init__(self, model_path: str, target_words: list):
        self.checkpoint = torch.load(
            model_path,
            map_location='cpu',
            weights_only=False # Оставляем ваш параметр
        )
        self.config = self.checkpoint['config']
        self.config['num_classes'] = len(target_words)

        # Добавляем проверку загруженных весов
        print("Загруженные          ключи          чекпоинта:",
self.checkpoint.keys())
        print("Ключи          model_state:",
self.checkpoint['model_state'].keys())

        self.label_encoder = LabelEncoder()
        self.label_encoder.classes_ =
self.checkpoint['label_encoder']

        # Проверяем параметры нормализации
        self.mean = self.checkpoint['mean'].numpy()
        self.std = self.checkpoint['std'].numpy()
        print(f"Параметры          нормализации:
mean={self.mean.shape}, std={self.std.shape}")

        self.model = SignLanguageModel(self.config)

        # Проверяем загрузку весов
        try:

self.model.load_state_dict(self.checkpoint['model_state'])
        except RuntimeError as e:
            print("Ошибка загрузки весов модели:", e)
            raise

        self.model.eval()

        def predict_sequence(self, sequence: np.ndarray) ->
List[Tuple[str, float]]:
            # Добавляем проверки входных данных
            print("Входная          последовательность          shape:",
sequence.shape)
            print("Пример данных:", sequence[0][:5])

            # Нормализация
            sequence = (sequence - self.mean) / self.std
            print("Нормализованные данные:", sequence[0][:5])

```

```

        # Преобразование в тензор [batch_size, seq_len,
features]
        tensor = torch.tensor(sequence,
dtype=torch.float32).unsqueeze(0)
        print("Размерность тензора:", tensor.shape)

        with torch.no_grad():
            outputs = self.model(tensor)
            print("Выход модели shape:", outputs.shape)

            probs = torch.softmax(outputs, dim=1)
            confidences, pred_indices = torch.max(probs, 1)
            print("Предсказанные индексы:", pred_indices)

        # Преобразование предсказаний
        words =
self.label_encoder.inverse_transform(pred_indices.numpy())
        return list(zip(words, confidences.tolist()))

# payment_system.py
import json
import re
import logging
from datetime import datetime, timedelta

class PaymentSystem:
    def __init__(self, stats_manager, subs_file: str =
"subscriptions.json"):
        self.stats_manager = stats_manager
        self.subs_file = subs_file
        self.logger =
logging.getLogger(self.__class__.__name__)
        self.subscriptions = self._load_subscriptions()
        self.logger.info("Инициализирована платежная
система")

    def _load_subscriptions(self) -> dict:
        self.logger.debug(f"Загрузка подписок из
{self.subs_file}")
        try:
            with open(self.subs_file, 'r') as f:
                data = json.load(f)
                self.logger.info(f"Успешно загружено
{len(data.get('subscriptions', {}))} подписок")
                return data
            except FileNotFoundError:
                self.logger.warning("Файл подписок не найден,
создан новый")
                return {'subscriptions': {}}
            except json.JSONDecodeError as e:
                self.logger.error(f"Ошибка чтения файла
подписок: {str(e)}")

```

```

        return {'subscriptions': {}}

    def _save_subscriptions(self):
        self.logger.debug("Сохранение подписок")
        try:
            with open(self.subs_file, 'w') as f:
                json.dump(self.subscriptions, f, indent=2,
ensure_ascii=False)
            self.logger.info("Подписки успешно сохранены")
        except Exception as e:
            self.logger.error(f"Ошибка сохранения подписок:
{str(e)}", exc_info=True)

    def process_payment(self, card_number: str, user_id: int)
-> bool:
        user_id_str = str(user_id)
        self.logger.info(f"Обработка платежа для
пользователя {user_id_str}")

        try:
            self.logger.debug(f"Проверка карты:
{card_number[:6]}*****")
            if not self._luhn_check(card_number):
                self.logger.warning(f"Невалидная карта от
пользователя {user_id_str}")
                return False

            expiry_date = datetime.now() + timedelta(days=30)
            self.logger.debug(f"Создание подписки до
{expiry_date}")

            # Обновление данных подписки
            self.subscriptions['subscriptions'][user_id_str]
= {
                'active': True,
                'expiry_date': expiry_date.isoformat(),
                'activated': datetime.now().isoformat()
            }

            # Логирование инициализации пользователя
            if user_id_str not in
self.stats_manager.data['users']:
                self.logger.info(f"Новый пользователь
{user_id_str} добавлен в статистику")

            self.stats_manager.data['users'][user_id_str] = {
                'video': 0,
                'voice': 0,
                'video_note': 0,
                'text': 0,
                'daily_video_count': 0,
                'last_video_date':
datetime.now().date().isoformat()

```

```

    }

self.stats_manager.data['total']['subscriptions'] += 1
    self.stats_manager._save_stats()
    self._save_subscriptions()

    self.logger.info(f"Успешная активация подписки
для {user_id_str}")
    return True

    except Exception as e:
        self.logger.error(f"Ошибка обработки платежа:
{str(e)}", exc_info=True)
        return False

    def check_subscription(self, user_id: int) -> bool:
        user_id_str = str(user_id)
        self.logger.debug(f"Проверка подписки для
{user_id_str}")
        try:
            sub =
self.subscriptions['subscriptions'].get(user_id_str)

            if not sub or not sub['active']:
                self.logger.debug(f"Подписка неактивна для
{user_id_str}")
                return False

            expiry =
datetime.fromisoformat(sub['expiry_date'])
            if datetime.now() > expiry:
                self.logger.info(f"Подписка истекла для
{user_id_str}")
                sub['active'] = False
                self._save_subscriptions()
                return False
            return True
        except Exception as e:
            self.logger.error(f"Ошибка проверки подписки:
{str(e)}", exc_info=True)
            return False

    def get_subscription_info(self, user_id: int) -> str:
        user_id_str = str(user_id)
        self.logger.debug(f"Запрос информации о подписке для
{user_id_str}")
        try:
            sub =
self.subscriptions['subscriptions'].get(user_id_str)

            if not sub or not sub['active']:
                self.logger.debug(f"Подписка не найдена для

```

```

{user_id_str}")

        return "❌ Подписка не активна"

        expiry_date =
datetime.fromisoformat(sub['expiry_date'])
        days_left = (expiry_date - datetime.now()).days
        self.logger.debug(f"Осталось дней подписки:
{days_left}")

        return (
            "✅ Активная подписка\n"
            f"📅 Окончание:
{expiry_date.strftime('%d.%m.%Y')}\n"
            f"⌚ Осталось: {days_left} дней"
        )
    except Exception as e:
        self.logger.error(f"Ошибка получения информации:
{str(e)}", exc_info=True)
        return "⚠ Ошибка получения информации"

    @staticmethod
    def _luhn_check(card_number: str) -> bool:
        try:
            digits = [int(d) for d in re.sub(r'\D', '',
card_number)]

            if len(digits) < 13 or len(digits) > 19:
                return False

            total = 0
            for i, digit in enumerate(digits[::-1]):
                if i % 2 == 1:
                    digit *= 2
                    total += digit - 9 if digit > 9 else digit
                else:
                    total += digit

            return total % 10 == 0
        except Exception as e:
            logging.getLogger(__name__).error(f"Ошибка
проверки Luhn: {str(e)}")
            return False

# preprocessing.py
import cv2
import numpy as np
import mediapipe as mp
from typing import Optional, List

class VideoProcessor:
    def __init__(self, frame_size=(640, 480), seq_length=16):
        self.frame_size = frame_size
        self.seq_length = seq_length

```

```

        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=2,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

    def process_video(self, video_path: str) ->
Optional[np.ndarray]:
        cap = cv2.VideoCapture(video_path)
        features_sequence = []

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            features = self._process_frame(frame)
            if features is not None:
                features_sequence.append(features)

        cap.release()

        if len(features_sequence) < self.seq_length:
            return None

        # Выборка кадров с интервалом
        sampled_frames =
self._resample_frames(features_sequence)
        return np.array(sampled_frames)

    def _resample_frames(self, frames: List[np.ndarray]) ->
List[np.ndarray]:
        indices = np.linspace(0, len(frames) - 1,
self.seq_length, dtype=int)
        return [frames[i] for i in indices]

    def _process_frame(self, frame: np.ndarray) ->
Optional[np.ndarray]:
        frame = cv2.flip(frame, 1)
        frame = cv2.resize(frame, self.frame_size)
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        results = self.hands.process(rgb_frame)
        if not results.multi_hand_landmarks:
            return None

        features = []
        for hand in results.multi_hand_landmarks[:2]: #
Максимум 2 руки
            for landmark in hand.landmark:
                features.extend([landmark.x, landmark.y,

```



```

landmark.z]])

# Дополнение нулями если рук меньше двух
features += [0.0] * (126 - len(features))
return np.array(features, dtype=np.float32)

# stats_manager.py
import json
from datetime import datetime
import logging
import os
from typing import Dict, Any

class StatisticsManager:
    def __init__(self, stats_file: str = "stats.json"):
        self.stats_file = stats_file
        self.logger = logging.getLogger(self.__class__.__name__)
        self.data = self._init_empty_data()
        self._load_or_create_stats()
        self.logger.info("Статистика инициализирована")

    def _init_empty_data(self) -> Dict[str, Any]:
        self.logger.debug("Создание пустой структуры данных")
        return {
            'metadata': {
                'created_at': datetime.now().isoformat(),
                'last_updated': None
            },
            'total': {
                'video': 0,
                'voice': 0,
                'video_note': 0,
                'text': 0,
                'subscriptions': 0
            },
            'users': {},
            'hourly_activity': {},
            'daily_activity': {}
        }

    def _load_or_create_stats(self) -> None:
        try:
            if not os.path.exists(self.stats_file):
                self.logger.warning("Файл статистики не найден, создание нового")
                self._save_stats()
                return

            self.logger.debug("Загрузка статистики из файла")

```

```

        with open(self.stats_file, 'r', encoding='utf-
8') as f:
            loaded_data = json.load(f)

            self.data = self._init_empty_data()
            load_stats = {
                'hourly_activity': self._load_hourly_stats,
                'daily_activity': self._load_daily_stats,
                'total': self._load_total_stats,
                'users': self._load_user_stats
            }

            for key, loader in load_stats.items():
                if key in loaded_data:
                    loader(loaded_data[key])
                else:
                    self.logger.warning(f"Отсутствует секция
{key} в файле статистики")

            self.logger.info("Статистика успешно загружена")
            self._save_stats()

    except Exception as e:
        self.logger.critical(f"Ошибка загрузке
статистики: {str(e)}", exc_info=True)
        self.data = self._init_empty_data()
        self._save_stats()

    def _load_hourly_stats(self, data: dict):
        self.logger.debug("Загрузка почасовой статистики")
        for hour, count in data.items():
            try:
                self.data['hourly_activity'][hour] =
int(count)
            except (ValueError, TypeError) as e:
                self.logger.warning(f"Некорректное значение
часа {hour}: {count} ({str(e)})")

    def _load_daily_stats(self, data: dict):
        self.logger.debug("Загрузка дневной статистики")
        for day, count in data.items():
            try:
                self.data['daily_activity'][day] =
int(count)
            except (ValueError, TypeError) as e:
                self.logger.warning(f"Некорректное значение
дня {day}: {count} ({str(e)})")

    def _load_total_stats(self, data: dict):
        self.logger.debug("Загрузка общей статистики")
        for category in ['video', 'voice', 'video_note',
'text']:
            try:

```

```

        self.data['total'][category] =
int(data.get(category, 0))
        except (ValueError, TypeError) as e:
            self.logger.warning(f"Некорректное значение
для {category}: {str(e)}")

    def _load_user_stats(self, data: dict):
        self.logger.debug(f"Загрузка данных пользователей
({len(data)} записей)")
        for user_id, user_data in data.items():
            try:
                self.data['users'][user_id] = {
                    'video': int(user_data.get('video', 0)),
                    'voice': int(user_data.get('voice', 0)),
                    'video_note':
int(user_data.get('video_note', 0)),
                    'text': int(user_data.get('text', 0))
                }
            except (ValueError, TypeError) as e:
                self.logger.warning(f"Ошибка загрузки данных
пользователя {user_id}: {str(e)}")

    def _save_stats(self) -> None:
        try:
            self.data['metadata']['last_updated'] =
datetime.now().isoformat()
            with open(self.stats_file, 'w', encoding='utf-
8') as f:
                json.dump(self.data, f, indent=2,
ensure_ascii=False)
            self.logger.debug("Статистика успешно
сохранена")
        except Exception as e:
            self.logger.error(f"Ошибка сохранения
статистики: {str(e)}", exc_info=True)

    def get_user_stats(self, user_id: int) -> str:
        self.logger.info(f"Запрос статистики пользователя
{user_id}")
        try:
            user_id_str = str(user_id)
            user_data = self.data['users'].get(user_id_str,
{})

            daily_limit = 5
            used = user_data.get('daily_video_count', 0)
            remaining = max(daily_limit - used, 0)

            stats = (
                "\ud83d\udcf1 Ваша статистика:\n"
                f"\ud83d\udcfa Видео: {user_data.get('video', 0)}\n"
                f"\ud83d\udcbb Голосовые: {user_data.get('voice',

```

```

0)}\n"

        f" Осталось бесплатных обработок сегодня:
{remaining}/{daily_limit}"
    )
    return stats
except Exception as e:
    self.logger.error(f"Ошибка формирования
статистики: {str(e)}", exc_info=True)
    return " {stats.get('voice', 0)} "
            f" {stats.get('video_note', 0)}
 {stats.get('text', 0)}"
            for uid, stats, _ in top_users]
        )
        if top_users else " Нет данных о
пользователях"

    return (

```

```

        f"📊 Детальная аналитика:\n\n"
        f"🕒 Пиковый час: {peak_hour[0]}
({peak_hour[1]} действий)\n"
        f"📅 Пиковый день: {peak_day[0]}
({peak_day[1]} действий)\n\n"
        f"🏆 Топ активных пользователей:\n"
        f"{top_users_str}"
    )
    except Exception as e:
        self.logger.error(f"Ошибка формирования
аналитики: {str(e)}", exc_info=True)
        return "⚠️ Ошибка формирования аналитики"

    def record_activity(self, category: str, update):
        user_id = update.effective_user.id
        self.logger.info(f"Запись активности: {category} для
пользователя {user_id}")
        try:
            user_id_str = str(user_id)
            if user_id_str not in self.data['users']:
                self.logger.debug(f"Новый пользователь
{user_id_str}")
                self.data['users'][user_id_str] = {
                    'video': 0,
                    'voice': 0,
                    'video_note': 0,
                    'text': 0,
                    'daily_video_count': 0,
                    'last_video_date':
datetime.now().date().isoformat()
                }

                self.data['users'][user_id_str][category] += 1
                self._save_stats()
                self.logger.debug(f"Активность {category}
обновлена")
            except Exception as e:
                self.logger.error(f"Ошибка записи активности:
{str(e)}", exc_info=True)

#train_model.py
import torch
import pandas as pd
import numpy as np
import random
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader
from torch.utils.tensorboard import SummaryWriter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tqdm.auto import tqdm
from dataset_processor import DataSetProcessor, OUTPUT_DIR,

```

TARGET_WORDS

```
# 1. Улучшенная модель с механизмом внимания
class SignLanguageModel(nn.Module):
    def __init__(self, config):
        super().__init__()

        # CNN для извлечения пространственных признаков
        self.cnn = nn.Sequential(
            nn.Conv1d(config['input_size'], 128, 3,
padding=1),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Dropout(config['dropout']),

            nn.Conv1d(128, 256, 3, padding=1),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Dropout(config['dropout'])
        )

        # LSTM для временных зависимостей
        self.lstm = nn.LSTM(
            256,
            config['hidden_size'],
            num_layers=config['num_layers'],
            bidirectional=True,
            dropout=config['dropout'],
            batch_first=True
        )

        # Механизм внимания
        self.attention = nn.Sequential(
            nn.Linear(config['hidden_size'] * 2, 256),
            nn.Tanh(),
            nn.Linear(256, 1),
            nn.Softmax(dim=1)
        )

        # Классификатор
        self.classifier = nn.Sequential(
            nn.Linear(config['hidden_size'] * 2, 512),
            nn.ReLU(),
            nn.Dropout(config['dropout']),
            nn.Linear(512, len(config['target_words']))
        )

    def forward(self, x):
        # Изменение размерности для CNN
        x = x.permute(0, 2, 1)
```

```

        x = self.cnn(x)

        # Подготовка для LSTM
        x = x.permute(0, 2, 1)

        # Обработка временных зависимостей
        lstm_out, _ = self.lstm(x)

        # Внимание к ключевым кадрам
        attention_weights = self.attention(lstm_out)
        context = torch.sum(attention_weights * lstm_out,
dim=1)

        return self.classifier(context)

# 2. Датасет с аугментацией и нормализацией
class SignLanguageDataset(Dataset):
    def __init__(self, annotations, data_dir, target_words,
mean=None, std=None, is_train=False):
        self.data_dir = data_dir
        self.annotations = annotations
        self.is_train = is_train
        self.mean = mean
        self.std = std

        # Кодирование меток
        self.label_encoder = LabelEncoder()
        self.label_encoder.fit(target_words)
        self.labels =

self.label_encoder.transform(annotations.iloc[:, 1])

        # Параметры аугментации
        self.noise_level = 0.02
        self.max_time_shift = 5

    def __len__(self):
        return len(self.annotations)

    def __getitem__(self, idx):
        # Загрузка тензора
        video_id = self.annotations.iloc[idx, 0]
        tensor =

torch.load(f"{self.data_dir}/{video_id}.pt")

        # Нормализация
        if self.mean is not None and self.std is not None:
            tensor = (tensor - self.mean) / self.std

        # Аугментации для тренировочных данных
        if self.is_train:
            # Добавление шума
            tensor += torch.randn_like(tensor) *
```

```

self.noise_level

        # Временной сдвиг
        if tensor.size(0) > self.max_time_shift * 2:
            shift = random.randint(-self.max_time_shift,
self.max_time_shift)
            tensor = torch.roll(tensor, shifts=shift,
dims=0)

        return tensor, torch.tensor(self.labels[idx],
dtype=torch.long)

# 3. Улучшенный класс для обучения
class AdvancedTrainer:
    def __init__(self, config):
        self.config = config
        self.device = torch.device('cuda' if
torch.cuda.is_available() else 'cpu')
        self.writer = SummaryWriter()

        # Загрузка и подготовка данных
        self._prepare_data()

        # Инициализация модели
        self.model = SignLanguageModel(config).to(self.device)

        # Функция потерь с балансировкой классов
        class_weights = self._calculate_class_weights()
        self.criterion = nn.CrossEntropyLoss(weight=class_weights)

        # Оптимизатор и шедулер
        self.optimizer = optim.AdamW(
            self.model.parameters(),
            lr=config['lr'],
            weight_decay=config['weight_decay']
        )
        self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
            self.optimizer,
            patience=config['patience'] // 2
        )

    def _prepare_data(self):
        # Загрузка аннотаций
        processor = DataSetProcessor()
        processor.load_annotations()

        # Разделение данных
        train_df, val_df = train_test_split(
            processor.filtered_df,

```



```

        test_size=0.2,
        stratify=processor.filtered_df.iloc[:, 1],
        random_state=42
    )

    # Вычисление статистик нормализации
    temp_ds = SignLanguageDataset(train_df, OUTPUT_DIR,
TARGET_WORDS)
    all_data = torch.cat([temp_ds[i][0] for i in
range(len(temp_ds))])
    mean = all_data.mean(dim=(0, 1))
    std = all_data.std(dim=(0, 1))

    # Создание финальных датасетов
    self.train_ds = SignLanguageDataset(
        train_df, OUTPUT_DIR, TARGET_WORDS, mean, std,
is_train=True
    )
    self.val_ds = SignLanguageDataset(
        val_df, OUTPUT_DIR, TARGET_WORDS, mean, std
    )

    def _calculate_class_weights(self):
        class_counts = np.bincount(self.train_ds.labels)
        return torch.tensor(1.0 / (class_counts + 1e-6),
dtype=torch.float32).to(self.device)

    def train_epoch(self, loader):
        self.model.train()
        total_loss = 0
        progress = tqdm(loader, desc="Training", leave=False)

        for inputs, labels in progress:
            inputs = inputs.to(self.device,
non_blocking=True)
            labels = labels.to(self.device,
non_blocking=True)

            self.optimizer.zero_grad()
            outputs = self.model(inputs)
            loss = self.criterion(outputs, labels)
            loss.backward()

torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
            self.optimizer.step()

            total_loss += loss.item()
            progress.set_postfix({'loss':
f"{loss.item():.4f}"})

        return total_loss / len(loader)

    def evaluate(self, loader):

```

```

self.model.eval()
total_loss = 0
correct = 0
total = 0

with torch.no_grad():
    for inputs, labels in loader:
        inputs = inputs.to(self.device)
        labels = labels.to(self.device)

        outputs = self.model(inputs)
        loss = self.criterion(outputs, labels)
        total_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    return total_loss / len(loader), correct / total

def run_training(self):
    best_acc = 0
    no_improve = 0

    train_loader = DataLoader(
        self.train_ds,
        batch_size=self.config['batch_size'],
        shuffle=True,
        num_workers=4,
        pin_memory=True
    )

    val_loader = DataLoader(
        self.val_ds,
        batch_size=self.config['batch_size'],
        num_workers=4,
        pin_memory=True
    )

    for epoch in range(self.config['epochs']):
        # Обучение
        train_loss = self.train_epoch(train_loader)

        # Валидация
        val_loss, val_acc = self.evaluate(val_loader)

        # Логирование
        self.writer.add_scalar('Loss/Train', train_loss,
epoch)
        self.writer.add_scalar('Loss/Val', val_loss,
epoch)
        self.writer.add_scalar('Accuracy/Val', val_acc,
epoch)

```

```

        # Обновление шедулера
        self.scheduler.step(val_acc)

        # Ранняя остановка
        if val_acc > best_acc:
            best_acc = val_acc
            no_improve = 0
            self._save_checkpoint()
            print(f"Epoch {epoch + 1}: New best accuracy
{val_acc:.4f}")
        else:
            no_improve += 1
            if no_improve >= self.config['patience']:
                print(f"Early stopping at epoch {epoch +
1}")
                break

        # Вывод информации
        print(f"\nEpoch {epoch + 1}/{self.config['epochs']}")
        print(f"Train Loss: {train_loss:.4f}")
        print(f"Val Loss: {val_loss:.4f} | Val Acc:
{val_acc:.4f}")
        print(f"Best Acc: {best_acc:.4f} | LR:
{self.optimizer.param_groups[0]['lr']:.2e}")
        print("-" * 60)

        self.writer.close()

    def _save_checkpoint(self):
        torch.save({
            'model_state': self.model.state_dict(),
            'config': self.config,
            'label_encoder':
self.train_ds.label_encoder.classes_,
            'mean': self.train_ds.mean,
            'std': self.train_ds.std
        }, "best_model.pth")

if __name__ == "__main__":
    # Конфигурация модели и обучения
    config = {
        'batch_size': 64,
        'epochs': 200,
        'lr': 3e-4,
        'weight_decay': 1e-4,
        'dropout': 0.5,
        'hidden_size': 512,
        'num_layers': 3,
        'input_size': 126,
        'seq_length': 30,

```

```

        'patience': 10,
        'target_words': TARGET_WORDS
    }

    # Запуск обучения
    trainer = AdvancedTrainer(config)
    trainer.run_training()

#dataset_processor.py
import os
import cv2
import torch
import pandas as pd
import mediapipe as mp
from tqdm.auto import tqdm

# ===== НАСТРОЙКИ =====
TARGET_WORDS = ['добрый', 'день', 'твой', 'имя', 'я', 'мой',
                 'благодарность', 'голодный', 'хотеть',
'говорить',
                 'мочь', 'не', 'вы', 'хорошо', 'против']
VIDEOS_DIR = 'D:/temp/train/train'
ANNOTATION_FILE = 'annotations.csv'
OUTPUT_DIR = './processed/'
MAX_FRAMES = 30

# Конфигурация MediaPipe
HAND_PARAMS = {
    'static_image_mode': False,
    'max_num_hands': 2,
    'model_complexity': 1,
    'min_detection_confidence': 0.5,
    'min_tracking_confidence': 0.5
}

# =====

class DataSetProcessor:
    def __init__(self):
        self.mp_hands = mp.solutions.hands
        self.hands = self.mp_hands.Hands(**HAND_PARAMS)

    def fix_encoding(self, text):
        """Исправление проблем с кодировкой"""
        try:
            return text.encode('latin-1').decode('utf-8')
        except:
            return text

    def load_annotations(self):
        """Загрузка и фильтрация аннотаций"""
        self.df = pd.read_csv(ANNOTATION_FILE, sep='\t',

```

```

header=None)
        self.df[1] = self.df[1].apply(self.fix_encoding)
        self.filtered_df =
self.df[self.df[1].isin(TARGET_WORDS)]
        print(f"Найдено {len(self.filtered_df)} видео для
обработки")

    def process_frame(self, frame):
        """Обработка одного кадра"""
        # Конвертация цвета и обработка
        results = self.hands.process(cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB))
        features = []

        if results.multi_hand_landmarks:
            # Обрабатываем до 2 рук
            for hand in results.multi_hand_landmarks[:2]:
                for landmark in hand.landmark:
                    features.extend([landmark.x, landmark.y,
landmark.z])

            # Дополнение до 2 рук (63 точки * 3 координаты)
            if len(results.multi_hand_landmarks) < 2:
                features += [0.0] * 63 * (2 -
len(results.multi_hand_landmarks))
            else:
                features = [0.0] * 126 # 2 руки * 63 координаты

        return features

    def process_video(self, video_path):
        """Обработка одного видеофайла"""
        cap = cv2.VideoCapture(video_path)
        frames = []

        try:
            while cap.isOpened():
                ret, frame = cap.read()
                if not ret:
                    break

                frames.append(self.process_frame(frame))
                if len(frames) >= MAX_FRAMES:
                    break
        finally:
            cap.release()

        # Выравнивание длины последовательности
        if len(frames) < MAX_FRAMES:
            padding = [[0.0] * 126 for _ in range(MAX_FRAMES
- len(frames))]
            frames.extend(padding)

```

```

        return torch.tensor(frames[:MAX_FRAMES],
dtype=torch.float32)

    def run(self):
        """Основной цикл обработки"""
        os.makedirs(OUTPUT_DIR, exist_ok=True)
        total_files = len(self.filtered_df)

        with tqdm(total=total_files, desc="Обработка видео")
as pbar:
            for row in self.filtered_df.itertuples():
                video_id, label = row[1], row[2]
                input_path = os.path.join(VIDEOS_DIR,
f"{video_id}.mp4")
                output_path = os.path.join(OUTPUT_DIR,
f"{video_id}.pt")

                if not os.path.exists(output_path):
                    try:
                        if not os.path.exists(input_path):
                            print(f"\nФайл {video_id}.mp4 не
найден")
                            continue

                        tensor =
self.process_video(input_path)
                        torch.save(tensor, output_path)
                    except Exception as e:
                        print(f"\nОшибка обработки
{video_id}: {str(e)}")
                    finally:
                        pbar.update(1)

if __name__ == "__main__":
    processor = DataSetProcessor()
    processor.load_annotations()
    processor.run()
    print("\nОбработка завершена! Результаты сохранены в:",
OUTPUT_DIR)

```

ПРИЛОЖЕНИЕ Б
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ В
(обязательное)

Ведомость документов